TRABAJO DE GRADO

MVP DE UN SISTEMA WEB PARA LA VERIFICACIÓN DE INTEGRIDAD Y TRAZABILIDAD DOCUMENTAL EN CONTRATOS LABORALES MEDIANTE BLOCKCHAIN PERMISIONADA

LAURA SOFIA DIAZ SUAREZ (67000991)
lsdiaz91@ucatolica.edu.co
SOFIA ORTIZ DAZA (67001001)
sortiz01@ucatolica.edu.co

UNIVERSIDAD CATÓLICA DE COLOMBIA
FACULTAD DE INGENIERÍA PROGRAMA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN
BOGOTÁ D.C
2026

TRABAJO DE GRADO

MVP DE UN SISTEMA WEB PARA LA VERIFICACIÓN DE INTEGRIDAD Y TRAZABILIDAD DOCUMENTAL EN CONTRATOS LABORALES MEDIANTE BLOCKCHAIN PERMISIONADA

LAURA SOFIA DIAZ SUAREZ (67000991)
lsdiaz91@ucatolica.edu.co
SOFIA ORTIZ DAZA (67001001)
sortiz01@ucatolica.edu.co

Trabajo de grado presentado para optar al título de Especialista en Ingeniería de sistemas y computación

Docente

SOLANGIE PAOLA GARAVITO MENDIVELSO
MSc
spgaravito@ucatolica.edu.co

UNIVERSIDAD CATÓLICA DE COLOMBIA
FACULTAD DE INGENIERÍA PROGRAMA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN
BOGOTÁ D.C
2026

Nota de aceptación:

_____
_____
_____
_____
_____

_____
Firma del presidente del jurado

_____
Firma del jurado

_____
Firma del jurado

CONTENIDO

GLOSARIO 10
RESUMEN 13
ABSTRACT 14
INTRODUCCIÓN 15
1. GENERALIDADES 16
1.1 ANTECEDENTES DEL PROBLEMA 16
1.2 PLANTEAMIENTO DEL PROBLEMA 16
1.3 PREGUNTA PROBLEMA 18
1.4 JUSTIFICACIÓN 18
Contribución a los Objetivos de Desarrollo Sostenible 20
1.5 OBJETIVOS 21
1.5.1 Objetivo General. 21
1.5.2 Objetivos Específicos. 21
1.6 MARCO REFERENCIAL 21
1.6.1 Marco Conceptual 21
1.6.2 Marco Teórico. 22
1.6.2.1 Verificación de integridad documental mediante criptografía 22
1.6.2.2 Blockchain permisionada como infraestructura de registro inmutable 23
1.6.2.3 Hyperledger Fabric como plataforma de implementación 24
1.6.2.4 Chaincode y automatización de lógica documental 25
1.6.2.5 Patrones de diseño en sistemas documentales con blockchain 25
1.6.2.6 Plan de pruebas y evaluación del sistema 26
1.6.3 Marco Jurídico. 26
1.6.3.1 Estándares internacionales 27
1.6.3.2 Marco normativo colombiano 27
1.7 DISEÑO METODOLÓGICO 28
1.7.1 Diseño de la Investigación 28
1.7.2 Ciclo de Vida del Software (SDLC): Prototipado Evolutivo 29
Fases del Proyecto y Correspondencia con Objetivos 29
Fase 1. Análisis, Benchmarking y Definición de Requerimientos (Relacionado con OE1) 29
Fase 2. Diseño de Arquitectura y Modelado del Sistema (Relacionado con OE2) 30
Fase 3. Desarrollo e Integración del MVP (Relacionado con OE3) 30
Fase 4. Evaluación, Pruebas de Integridad y Rendimiento (Relacionado con OE4) 31

2. DESARROLLO DEL PRIMER OBJETIVO 32
2.1 Planteamiento del objetivo y correspondencia metodológica 32
2.2 Análisis normativo y matriz de trazabilidad 32
2.3 Consulta con actores clave 33
2.4 Benchmarking metodológico 35
2.5 Levantamiento y especificación de requerimientos 36
2.6 Backlog del MVP 37
2.6.1 Cronograma de Sprints: 37
2.7 Resultados frente a los indicadores 38
3. DESARROLLO DEL SEGUNDO OBJETIVO 39
3.1 Planteamiento del objetivo y correspondencia metodológica 39
3.2 Modelado funcional y drivers de arquitectura 39
3.2.1 Diagrama de casos de uso 39
3.2.2 Matriz de requerimientos arquitectónicamente significativos (ASR) 40
3.2.3 Utility Tree 41
3.3 Arquitectura de software 41
3.3.1 Diagrama de componentes 41
3.3.2 Modelo entidad-relación (MER) 42
3.3.3 Diagramas de secuencia 42
3.3.4 Patrones de diseño y estilos arquitectónicos 42
3.4 Arquitectura de infraestructura 44
3.4.1 Topología de despliegue 44
3.4.2 Red Hyperledger Fabric 44
3.4.3 Separación de exposición 45
3.4.4 Trazabilidad de los RNF con los componentes de infraestructura 45
3.5 Diseño UX/UI 46
3.5.1 Wireframes de baja fidelidad 46
3.5.2 Prototipo de alta fidelidad 46
3.6 Decisiones de diseño y diferencias documentadas 47
3.7 Resultados frente a los indicadores 48
4. DESARROLLO DEL TERCER OBJETIVO 49
4.1 Planteamiento del objetivo y correspondencia metodológica 49
4.2 Código fuente del MVP 49
4.3 Script de despliegue de la red 52

4.4 API de integración funcional 52
4.5 Evidencia de integración con Hyperledger Fabric 53
4.6 Prototipado evolutivo y control de calidad por sprint 54
4.7 Registro de cambios respecto al diseño 55
4.8 Resultados frente a los indicadores 56
5. DESARROLLO DEL CUARTO OBJETIVO 58
5.1 Planteamiento del objetivo y correspondencia metodológica 58
5.2 Plan de Pruebas de Software 58
5.3 Entornos y ciclos de prueba 59
5.4 Pruebas funcionales de caja negra 59
5.5 Pruebas de integridad por mutación documental 66
5.6 Pruebas de trazabilidad e inmutabilidad 68
5.7 Pruebas de rendimiento 69
5.8 Pruebas del despliegue en Azure 70
5.9 Defectos y acciones correctivas 71
5.10 Matriz de cumplimiento y manual de usuario 71
5.11 Resultados frente a los indicadores 72
CONCLUSIONES 74
6. RECOMENDACIONES 76
Alcances y problemas no abarcados en este trabajo 76
Trabajos futuros sobre la misma temática 77
BIBLIOGRAFÍA 78
ANEXOS 82

LISTA DE TABLAS

Tabla 1. Resultados del KPI-1.2 33
Tabla 2. Stakeholders 34
Tabla 3. Benchmarking 35
Tabla 4. Backlog 37
Tabla 5. Resultados KPIs O1 38
Tabla 6. Requerimientos arquitectónicamente significativos 40
Tabla 7. Topología despliegue 44
Tabla 8. Observaciones de la matriz 47
Tabla 9. Resultados KPIs O2 48
Tabla 10. Aplicación de la Definición de Terminado por sprint 54
Tabla 11. Resumen de los cambios respecto al diseño 55
Tabla 12. Resultado del KPIs O3 57
Tabla 13. Tipos de prueba y casos del PPS 58
Tabla 14. Detección por tipo de alteración 68
Tabla 15. Consistencia entre el ledger y la base de datos 69
Tabla 16. Latencia de registro y verificación 69
Tabla 17. Defectos encontrados y su cierre 71
Tabla 18. Resultado de los KPI del O4 73

LISTA DE FIGURAS

Figura 1. Visualización de la metodología 28
Figura 2. Ciclo de vida del software 29
Figura 3. Repositorio del proyecto en GitHub 50
Figura 4. README del repositorio 50
Figura 5. Pruebas automatizadas 51
Figura 6. Aceptación de la política de tratamiento de datos en el primer ingreso 51
Figura 7. Registro de intentos fallidos de inicio de sesión en MongoDB 51
Figura 8. Contenedores del sistema en ejecución 52
Figura 9. Documentación de la API en formato OpenAPI 53
Figura 10. Llamadas reales a la API 53
Figura 11. Contraste directo en el ledger 54
Figura 12. Latencia del endpoint de verificación en 50 peticiones consecutivas 56
Figura 13. Carga de un contrato en PDF 59
Figura 14. Carga aceptada de un PDF 60
Figura 15. Rechazo de un archivo DOCX 60
Figura 16. Rechazo de un PDF de 25 MB 60
Figura 17. Sección mis documentos 61
Figura 18. Sección mi trazabilidad 61
Figura 19. Tratamiento de datos 62
Figura 20. Cambio de contraseña 62
Figura 21. Confirmación antes de cerrar la sesión 62
Figura 22. Selección en dos pasos 63
Figura 23. Comparación con el mismo archivo 64
Figura 24. Identificador inexistente 64
Figura 25. Listado general de documentos con su identificador 64
Figura 26. Panel del Administrador 65
Figura 27. Roles, creación de un usuario y contraseñas 65
Figura 28. Usuario desactivado no puede ingresar al portal 66
Figura 29. Trazabilidad administrativa 66
Figura 30. Alteración de texto en el contrato C01 67
Figura 31. Alteración de firma en el contrato C02 67
Figura 32. Alteración solo de metadatos en el contrato C03 67
Figura 33. Ejecución de INT-01 contra Azure 68
Figura 34. Trazabilidad de C02 69
Figura 35. Pruebas de infraestructura y seguridad sobre el despliegue 70
Figura 36. Resumen de la Matriz de Cumplimiento de Requerimientos 72

LISTA DE ANEXOS

Anexo A. Tabla Kpis, sección de detalle Smart y leyenda 82
Anexo B. Backlog del producto 82
Anexo C. Requerimientos Funcionales y No funcionales 82
Anexo D. Benchmarking Comparativo con GCP 82
Anexo E. Matriz Stakeholders 82
Anexo F. Matriz Normativa 82
Anexo G. Tabla detallada de instrumentos y herramienta 82
Anexo H. Seguridad por módulo en el sistema 82
Anexo I. Presupuesto 82
Anexo J. Cronograma, resumen por semana y Leyenda 82
Anexo K. Entrevista Empresa Preguntas 82
Anexo L. UML: casos de uso, componentes, despliegue, MER, secuencias 82
Anexo M. Wireframes baja fidelidad y Prototipo Alta fidelidad 82
Anexo N. Matriz Infraestructura 82
Anexo O. Matriz Asrs y Utility tree 83
Anexo P. Matriz_Trazabilidad_HU_R 83
Anexo Q. registro-sprints 83
Anexo R. integracion-fabric 83
Anexo S. INSTALACION 83
Anexo T. Manual de usuario 83
Anexo U. Registro cambios del diseño 83
Anexo V. Pruebas QA-QC 83
Anexo W. Informe de resultados 83
Anexo X. Matriz cumplimiento de requerimientos 83

GLOSARIO

HASH CRIPTOGRÁFICO

Resultado de aplicar una función matemática a un conjunto de datos, generando una cadena única de longitud fija que representa su contenido. Se utiliza para garantizar la integridad de la información.

SHA-256

Algoritmo de función hash perteneciente a la familia SHA-2, que genera una huella digital de 256 bits. Es ampliamente utilizado en sistemas de seguridad y blockchain para verificar la integridad de los datos.

SAL (SALT)

Valor aleatorio que se añade a una contraseña antes de aplicar una función hash, con el fin de aumentar la seguridad y evitar ataques.

BCRYPT

Algoritmo de hashing diseñado para el almacenamiento seguro de contraseñas, que incorpora automáticamente el uso de sal y múltiples iteraciones para dificultar ataques de fuerza bruta.

TOKEN JWT (JSON WEB TOKEN)

Mecanismo de autenticación basado en tokens que permite transmitir información segura entre cliente y servidor mediante un objeto firmado digitalmente.

AUTENTICACIÓN

Proceso mediante el cual un sistema verifica la identidad de un usuario o entidad antes de permitir el acceso a recursos.

AUTORIZACIÓN

Proceso posterior a la autenticación que determina los permisos y niveles de acceso que posee un usuario dentro de un sistema.

CERTIFICADO DIGITAL X.509

Documento electrónico que vincula una identidad con una clave pública, utilizado en infraestructuras de clave pública (PKI) para autenticar entidades en redes seguras.

BLOCKCHAIN

Tecnología de registro distribuido que permite almacenar información de forma descentralizada, segura e inmutable mediante una cadena de bloques enlazados criptográficamente.

RED BLOCKCHAIN PERMISIONADA

Tipo de red blockchain en la que el acceso y la participación están restringidos a entidades previamente autorizadas, permitiendo mayor control sobre identidad y gobernanza.

HYPERLEDGER FABRIC

Plataforma blockchain empresarial de código abierto, diseñada para redes permisionadas, que permite definir políticas de acceso, identidad y ejecución de contratos inteligentes en entornos organizacionales.

LEDGER (LIBRO MAYOR)

Registro distribuido donde se almacenan de forma cronológica e inmutable todas las operaciones o eventos realizados dentro de una red blockchain.

INMUTABILIDAD

Propiedad de un sistema que impide la modificación o eliminación de datos una vez han sido registrados, garantizando su integridad a lo largo del tiempo.

TRAZABILIDAD

Capacidad de rastrear y reconstruir el historial de eventos o acciones asociadas a un elemento, permitiendo conocer su origen, modificaciones y estado a lo largo del tiempo.

CHAINCODE

Programa o contrato inteligente desplegado en Hyperledger Fabric que define la lógica de negocio y las operaciones que pueden ejecutarse sobre el ledger.

NODO

Entidad participante dentro de una red blockchain que mantiene una copia del ledger y contribuye a la validación y ejecución de operaciones.

INTEGRIDAD DOCUMENTAL

Propiedad que garantiza que un documento no ha sido alterado desde su creación o registro, manteniendo su contenido original de forma verificable.

DOCUMENTO ELECTRÓNICO

Archivo digital que contiene información estructurada y que puede ser almacenado, transmitido o procesado mediante sistemas informáticos.

FIRMA DIGITAL

Mecanismo criptográfico que permite validar la identidad del firmante y garantizar la integridad de un documento electrónico mediante el uso de claves públicas y privadas.

METADATOS

Datos descriptivos asociados a un documento, como fecha de creación, autor o identificador, que permiten su gestión, clasificación y trazabilidad.

HUELLA DIGITAL DEL DOCUMENTO

Representación única de un documento generada mediante una función hash, utilizada para verificar su integridad sin necesidad de almacenar el archivo original.

API REST
Estilo de arquitectura para el diseño de servicios web que permite la comunicación entre sistemas mediante solicitudes HTTP y el uso de recursos identificados por URL.

BACKEND
Componente de un sistema que gestiona la lógica de negocio, el procesamiento de datos y la comunicación con bases de datos o servicios externos.

FRONTEND
Parte del sistema con la que interactúa el usuario, encargada de la presentación visual y la experiencia de uso.

DOCKER
Plataforma de virtualización ligera que permite empaquetar aplicaciones y sus dependencias en contenedores, facilitando su despliegue y ejecución en distintos entornos.

BENCHMARKING
Proceso de evaluación comparativa que consiste en analizar el rendimiento, características o resultados de un sistema frente a otros referentes o estándares, con el fin de identificar mejoras o validar su desempeño

RESUMEN

Hoy en día es cada vez más común que los contratos laborales se manejen de forma digital, sobre todo en el sector de TI. Sin embargo, varios de estos documentos son administrados por sistemas antiguos o tradicionales los cuales no pueden garantizar que la integridad, autenticidad y trazabilidad del documento sean confiables. Esto hace que se generen riesgos relacionados con la alteración de la información haciendo que este pierda su valor como evidencia digital.

En el sector colombiano, el marco normativo reconoce la validez jurídica de los documentos electrónicos, pero estos deben cumplir con las condiciones de integridad y verificabilidad que estos proponen. En algunos casos, estas condiciones no se cumplen dentro de los mecanismos de gestión documental, generando un escenario evidente; la necesidad de implementar nuevas soluciones tecnológicas que ayuden a fortalecer la confiabilidad de los registros digitales.

Ante esta problemática se realizó el presente proyecto, el cual se basa en el desarrollo de un MVP de un sistema web integrado a una red permisionada blockchain basada en Hyperledger Fabric, con el objetivo de verificar la integridad y la trazabilidad de contratos laborales en una empresa de tecnología en Bogotá. Esta solución tuvo como base la generación de huellas criptográficas, o también conocidos como "hash", de los documentos. Estas huellas se registraron en la red junto con los eventos necesarios para su gestión sin necesidad de almacenar directamente el archivo en la red blockchain.

El sistema se compuso de tres partes: un frontend desarrollado en React, un backend en Node.js y un contrato inteligente (chaincode) en Go, desplegados sobre una infraestructura de contenedores Docker. El MVP se evaluó con un plan de pruebas de 62 casos sobre un despliegue en Microsoft Azure, donde detectó el 100 % de las alteraciones en 20 documentos de prueba y aprobó el 100 % de los casos funcionales, lo que demostró la viabilidad de la red blockchain como mecanismo de verificación de la integridad documental. Así mismo, se buscó contribuir con la mejora de la seguridad, trazabilidad y confiabilidad de los contratos laborales en entornos digitales.

PALABRAS CLAVE: Blockchain, Cryptography, Data integrity, Smart contracts, Web sites, Prototyping, Access control, Traceability, Distributed databases, Computer security, Digital signatures, Application software.

ABSTRACT

Nowadays, it is increasingly common for employment contracts to be managed digitally, especially in the IT sector. However, many of these documents are managed through outdated or traditional systems that cannot guarantee the integrity, authenticity, and traceability of the documents. This creates risks related to the alteration of information, potentially causing the document to lose its value as digital evidence.

In Colombia, the legal framework recognizes the legal validity of electronic documents; however, these documents must comply with the integrity and verifiability requirements established by the applicable regulations. In some cases, these conditions are not fully met by existing document management mechanisms, creating a clear need to implement new technological solutions that can strengthen the reliability of digital records.

In response to this issue, this project developed a Minimum Viable Product (MVP) consisting of a web-based system integrated with a permissioned blockchain network based on Hyperledger Fabric, with the objective of verifying the integrity and traceability of employment contracts at a technology company in Bogotá. The solution was based on generating cryptographic fingerprints, also known as "hashes," for the documents. These fingerprints were recorded on the network along with the events required for their management, without storing the files on the blockchain network.

The system consisted of three components: a frontend developed with React, a backend developed with Node.js and a smart contract (chaincode) written in Go, deployed on a Docker-based containerized infrastructure. The MVP was evaluated with a 62-case software test plan on a Microsoft Azure deployment, where it detected 100% of the alterations in 20 test documents and passed 100% of the functional test cases, demonstrating the technical feasibility of using a permissioned blockchain network to verify document integrity. Furthermore, the project contributed to improving the security, traceability, and reliability of employment contracts in digital environments.

KEYWORDS: Blockchain, Cryptography, Data Integrity, Smart Contracts, Websites, Prototyping, Access Control, Traceability, Distributed Databases, Computer Security, Digital Signatures, Application Software.

INTRODUCCIÓN

El sector de TI en Colombia ha experimentado un crecimiento constante el cual genera que los procesos contractuales se digitalicen, haciendo que toda la gestión de contratos laborales y acuerdos de confidencialidad migren a entornos más digitales. Pero esto también tiene sus desventajas, este crecimiento puede presentar errores estructurales importantes cuando los documentos son gestionados como archivos digitales convencionales, por ejemplo: cuando los documentos son compartidos por correo electrónico o almacenados en repositorios en la nube sin mecanismos técnicos que garanticen su integridad, autenticidad ni trazabilidad a lo largo de su ciclo de vida [1]. Para este tipo de contextos existen leyes cómo: la Ley 527 de 1999 exige que la validez jurídica de un mensaje de datos dependa de la demostrabilidad de su integridad [2]. El Decreto 2364 de 2012 establece que los mecanismos de firma electrónica deben permitir identificar al firmante y asociarlo con el contenido suscrito [3]. A pesar de esto, muchas organizaciones continúan utilizando firmas digitalizadas que no ofrecen las garantías requeridas [3], creando una brecha técnica y jurídica que incrementa el riesgo de alteraciones documentales no detectables, disputas contractuales y debilidad probatoria [4].

Ante esta problemática, el presente trabajo de grado propone el desarrollo de un Producto Mínimo Viable (MVP) de un sistema web integrado con una red blockchain permisionada Hyperledger Fabric orientado a la verificación de integridad y trazabilidad documental en contratos laborales y acuerdos de confidencialidad en formato PDF, en el contexto de una empresa del sector de servicios tecnológicos en Bogotá [5, 6]. El objeto de estudio es el proceso de gestión contractual digital de dicha organización, donde se identificaron deficiencias específicas en el control de versiones, la verificación de integridad y la capacidad probatoria de los documentos. Una vez se tengan los datos analizados, se hace levantamiento de requerimientos funcionales, normativos y técnicos, entrevistas semiestructuradas y un benchmarking comparativo para evaluar la competencia. Una vez se tenga esa información completa, se construye un stack donde está la base del proyecto: React, Node.js, Hyperledger Fabric, MongoDB y Docker. para luego ser evaluado mediante un plan de pruebas dividida en 4 partes para evaluar la modificación en documentos sintéticos, pruebas funcionales de caja negra, verificación de la trazabilidad e inmutabilidad de red permisionada, así como su medición de latencias operativas [7, 8].

Finalmente, se espera que el proyecto genere un impacto positivo haciendo que el sistema de la empresa de objeto de estudio fortalezca la gestión documental contractual al desplazar su modelo tradicional a un modelo más técnico y objetivo, mejorando así la capacidad de verificar la integridad en contratos, reduciendo la necesidad de análisis posteriores y aportando una base técnica replicable para modernizar los procesos de gestión documental de esta organización [9, 10].

1. GENERALIDADES

1.1 ANTECEDENTES DEL PROBLEMA

A nivel global, la digitalización ha convertido a los documentos electrónicos en el centro de las organizaciones, impulsando el uso de tecnologías como blockchain para rastrear, certificar y gestionar datos de forma segura frente a los riesgos de alteración. En Colombia, el crecimiento del sector TI [5], obliga a digitalizar masivamente los contratos laborales y, aunque la Ley 527 de 1999 les otorga validez jurídica, exige obligatoriamente demostrar la integridad de la información desde su creación [2]. Sin embargo, en la práctica, muchas empresas administran estos contratos como archivos digitales convencionales (usualmente PDFs) que carecen de mecanismos criptográficos para garantizar su autenticidad y trazabilidad a lo largo de su ciclo de vida.

Al aterrizar esta problemática a un escenario local, se evidencia el caso de una empresa de servicios tecnológicos ubicada en Bogotá, donde los contratos se siguen guardando mediante imágenes de firmas digitalizadas [6]. Esta práctica no ofrece las garantías técnicas ni jurídicas requeridas por la normatividad colombiana para comprobar quién firmó el documento o garantizar que su contenido no haya sido alterado. Esta falta de mecanismos confiables imposibilita llevar un control de versiones adecuado y dificulta detectar a tiempo modificaciones no autorizadas, generando una debilidad operativa y jurídica que compromete el valor probatorio de los documentos ante posibles auditorías o procesos judiciales.

1.2 PLANTEAMIENTO DEL PROBLEMA

El sector de TI en Colombia a través del tiempo se ha convertido en uno de los ámbitos con una gran importancia en la parte nacional de la economía digital, pues esto se debe al crecimiento que se ha evidenciado en el aumento de la demanda de servicios tecnológicos, la transformación en organizaciones y la necesidad de contratar personas capacitadas [5]. A partir de esto, ha incrementado la rotación laboral, teniendo en consecuencia una gran cantidad de documentos contractuales gestionados por las organizaciones; como lo son contratos laborales o acuerdos de confidencialidad (NDA), registros esenciales que sirven para formalizar relaciones laborales, regular obligaciones y proteger información estratégica.

En este contexto, la manera en que se manejan los documentos ha cambiado a un entorno mayormente digital, en donde desde su elaboración hasta su consulta se realiza principalmente mediante archivos electrónicos, usualmente en formato PDF, compartidos por correo electrónico o que están protegidos en servicios de almacenamiento en la nube [2]. Sin embargo, este proceso genera limitaciones a pesar de sus beneficios, pues el control documental queda vulnerado, especialmente en lo relacionado con la verificación de integridad, autenticidad y trazabilidad [1].

En diversas empresas de tecnología los contratos y acuerdos de confidencialidad se siguen guardando como archivos digitales tradicionales que incluyen una foto o imagen de la firma de la persona, en lugar de una firma digital segura y verificable. Ante la visión jurídica, se considera que este proceso no cumple con las condiciones técnicas de autenticidad e integridad documental. De acuerdo con la Ley 527 de 1999, una firma digital corresponde a un

valor numérico asociado a un mensaje de datos y vinculado a un procedimiento criptográfico que permite identificar al firmante y detectar alteraciones posteriores en la información [2]. Por otra parte, el Decreto 2364 de 2012 reglamenta la firma electrónica en Colombia y establece las condiciones bajo las cuales puede utilizarse como mecanismo de autenticación en entornos digitales [3]. En consecuencia, una firma digitalizada no brinda la seguridad necesaria para comprobar quién firmó el documento, ni garantizar que su contenido no haya sido modificado.

Esta limitación se convierte en un punto débil dentro del proceso de gestión documental de los contratos, ya que los documentos pueden ser replicados, reenviados, modificados o sustituidos sin que exista una prueba confiable para verificar cuál corresponde a la versión original, si su contenido se alteró o si el archivo aún conserva las condiciones iniciales [4]. Cuando un contrato se comparte entre diferentes personas puede resultar difícil saber cuál es la versión más reciente o si el documento ha sido modificado. La falta de control de versiones, registros de fecha verificables y mecanismos de seguridad aumenta la incertidumbre sobre la autenticidad, validez e integridad del documento [11, 12].

Desde una perspectiva operativa, las dificultades aparecen cuando se necesita detectar a tiempo modificaciones no autorizadas en cláusulas, fechas, obligaciones o condiciones contractuales. Al ser documentos digitales convencionales, pueden modificarse sin que exista un registro de los cambios realizados. Esto representa un problema significativo en un entorno donde el aumento de incidentes relacionados con ciber fraude y manipulación documental incrementa los riesgos para la seguridad y confiabilidad [13].

Desde el punto de vista legal, la falta de mecanismos técnicos de verificación dificulta demostrar que un documento es auténtico y que no ha sido modificado. Esto puede generar problemas en disputas contractuales, conflictos laborales, auditorías o procesos judiciales [4]. En estas situaciones, suele ser necesario recurrir a revisiones manuales o análisis informáticos especializados para validar el documento, lo que aumenta los costos, demora la resolución de los casos y reduce la confianza en el documento como evidencia digital [14].

Todo este problema cobra relevancia cuando se trata de documentos que están asociados a acuerdos de confidencialidad y cláusulas de propiedad intelectual, donde si ocurre una alteración sin que se genere un aviso puede generar dilemas de incumplimientos contractuales, disputas sobre obligaciones pactadas o exposición indebida de información sensible para la organización. Teniendo eso en cuenta, la ausencia de un proceso confiable no es lo único que compromete la integridad, sino también un riesgo operativo, jurídico y estratégico [14].

Si bien esta problemática se presenta en diferentes organizaciones del sector TI colombiano, el presente proyecto centra su análisis en una empresa de servicios tecnológicos ubicada en Bogotá. En esta organización se identificaron dificultades relacionadas con la gestión de contratos digitales, particularmente en la verificación de la integridad de los documentos, el control de versiones y el seguimiento de los cambios realizados sobre los registros contractuales.

En este sentido, se identifica la necesidad de una solución tecnológica que permita fortalecer

la gestión documental contractual de la organización objeto de estudio mediante mecanismos que garanticen la integridad del contenido, la trazabilidad de los eventos asociados al documento y la posibilidad de verificar su estado durante su ciclo de vida digital.

1.3 PREGUNTA PROBLEMA

¿En qué medida una red blockchain permisionada (Hyperledger Fabric) integrada con un sistema web permite verificar la integridad y trazabilidad de contratos laborales electrónicos en el contexto normativo colombiano?

1.4 JUSTIFICACIÓN

Como se expuso en el planteamiento del problema, la creciente dependencia de documentos digitales para formalizar relaciones laborales, gestionar acuerdos de confidencialidad y conservar evidencia contractual ha puesto en evidencia la necesidad de garantizar condiciones de integridad, trazabilidad y verificabilidad sobre la información contenida en dichos archivos [15]. Las dificultades para verificar si un documento ha sido modificado después de su emisión, especialmente cuando se administra mediante repositorios convencionales sin mecanismos criptográficos de validación [1, 10], justifican la búsqueda de una solución técnica que aborde estas carencias.

Al momento de tratar la problemática, se necesita sustentar tomando en cuenta el marco normativo vigente de Colombia para documentos electrónicos. La Ley 527 de 1999 reconoce la validez jurídica y probatoria de los mensajes de datos, siempre que sea posible demostrar la integridad de la información desde su creación [2]. Asimismo, el Decreto 2364 de 2012 regula la firma electrónica como un mecanismo válido de autenticación en entornos digitales, mientras que la Ley 1581 de 2012 establece lineamientos para el tratamiento y la protección de los datos personales [3, 16]. Sin embargo, la falta de una adecuada gestión documental provoca que muchas organizaciones no cuenten con los mecanismos necesarios para garantizar la integridad, la trazabilidad y el control de modificaciones de los documentos electrónicos [9, 10]. Esto evidencia una brecha entre el reconocimiento normativo de estos documentos y la capacidad técnica para verificar su autenticidad y confiabilidad.

Ante este escenario, las tecnologías de registro distribuido surgen como una alternativa para cerrar esa brecha [17]. Blockchain, la tecnología seleccionada, permite mantener un registro cronológico, auditable e inmutable, a partir de estructuras criptográficas enlazadas, logrando una ventaja mayor cuando se requiere tener evidencia de integridad, trazabilidad y control de cambios [9, 10, 18]. En particular, diversos estudios señalan que la aplicación de blockchain en sistemas documentales fortalece la verificación de la integridad de los documentos, la trazabilidad de los cambios realizados y los procesos de auditoría. Esto se debe a que reduce la dependencia de mecanismos centralizados y aumenta la confiabilidad de los registros almacenados [1, 9, 19]. Por esta razón, su implementación en la gestión documental no solo responde a una tendencia tecnológica, sino también a la necesidad de garantizar mayor confianza, transparencia y seguridad en la evidencia digital [10, 15].

Para el desarrollo de la solución se necesita seleccionar una alternativa tecnológica pertinente dentro de blockchain que se ajuste a los requisitos del proyecto y que permita garantizar un adecuado tratamiento de los documentos contractuales. Entre las alternativas disponibles,

Hyperledger Fabric resultó ser la opción más adecuada para este proyecto, es una plataforma blockchain permisionada, modular y orientada a entornos empresariales [20]. Cuenta con servicios como mecanismos de control de acceso, confidencialidad y trazabilidad entre actores autorizados [20, 21]. Además, cuenta con una arquitectura que permite construir redes privadas bajo control institucional, conservando propiedades de auditabilidad, trazabilidad, inmutabilidad sin exponer información sensible en infraestructuras públicas [20, 22].

Su arquitectura distribuida y robusta evita que se desarrollen desde cero ciertos mecanismos fundamentales como la gestión de identidades, el consenso, la replicación del libro mayor y la validación de transacciones [20]. Esta red favorece la viabilidad del proyecto y fortalece el control institucional sobre aspectos críticos como el acceso, la identidad de los participantes y el registro de eventos [20].

En este sentido, usar Hyperledger Fabric permite concentrar el desarrollo en las funcionalidades relacionadas con la verificación documental y la trazabilidad de contratos, manteniendo un alto nivel de rigurosidad técnica [20]. El entorno será más confiable debido al uso de estos mecanismos, y estará alineado con las necesidades organizacionales [1, 20].

Se propone el desarrollo de un MVP integrado a una red blockchain permisionada para registrar, validar y consultar las acciones realizadas sobre documentos contractuales. Cuando un documento sea cargado en la plataforma, se generará una huella criptográfica única (hash) [23] la cual será registrada en la red blockchain como evidencia de su estado e integridad. Cuando se quiera comprobar una copia del documento, se calculará de nuevo su huella y se comparará con la registrada: cualquier cambio, por mínimo que sea, producirá una huella distinta y dejará en evidencia la alteración [23, 24]. La solución no está en guardar los documentos en sí, sino en conservar evidencia verificable de su estado a lo largo del tiempo, sin comprometer la confidencialidad de su contenido [9, 24].

La idea de esta propuesta no es una herramienta tecnológica genérica, sino una respuesta a una necesidad identificada en una empresa del sector de servicios tecnológicos en Bogotá, donde se encontraron debilidades en la verificación de integridad, el control de versiones y la capacidad probatoria de los contratos manejados digitalmente [4, 6]. En este sentido, la solución se plantea para responder directamente a un problema organizacional observado en un contexto real de aplicación.

Desde el punto de vista técnico y operativo, la propuesta es factible en el contexto de una investigación aplicada, ya que Hyperledger Fabric puede ser desplegado y evaluado en entornos controlados mediante contenedores y redes de prueba, sin necesidad de recurrir a la infraestructura empresarial de producción para validar su funcionamiento [20]. Esto permite desarrollar una solución funcional, ejecutar pruebas de integridad, trazabilidad y rendimiento, y demostrar la viabilidad técnica del sistema en un entorno académico controlado [25].

Contribución a los Objetivos de Desarrollo Sostenible

ODS 8 Trabajo decente y crecimiento económico. Meta 8.8: proteger los derechos laborales y promover un entorno de trabajo seguro y sin riesgos para todos los trabajadores.

El contrato laboral es el documento que fija los derechos y obligaciones del trabajador. Si la empresa y el trabajador pueden comprobar de forma objetiva que las condiciones pactadas, como el salario, la vigencia o las cláusulas de confidencialidad, no se modificaron después de la firma, se reduce el riesgo de disputas por alteraciones y ambas partes cuentan con evidencia confiable de lo acordado.

ODS 9 Industria, innovación e infraestructura. Meta 9.5: aumentar la investigación científica y mejorar la capacidad tecnológica de los sectores industriales, fomentando la innovación.

El proyecto desarrolla capacidad tecnológica desde la universidad, aplicada a un problema real del sector de servicios tecnológicos en Colombia: la verificación de la integridad de documentos contractuales mediante una red blockchain permisionada, diseñada de acuerdo con el marco normativo del país en lugar de adoptar una solución externa.

ODS 16 Paz, justicia e instituciones sólidas. Meta 16.6: crear a todos los niveles instituciones eficaces y transparentes que rindan cuentas.

Un registro inmutable y auditable de las acciones sobre cada contrato fortalece la transparencia y la rendición de cuentas dentro de la organización, y aporta evidencia técnica que puede apoyar la resolución de conflictos laborales, auditorías o procesos judiciales, en línea con el valor probatorio que la Ley 527 de 1999 reconoce a los mensajes de datos [2].

En el ámbito nacional, el proyecto se alinea con la Guía de referencia para la adopción e implementación de proyectos con tecnología blockchain del MinTIC [1], que reconoce el potencial de las redes permisionadas para garantizar la integridad y la trazabilidad de los registros, y cuyos criterios técnicos también son aplicables a organizaciones privadas.

Por último, desde el punto de vista organizacional, el desarrollo de un sistema web para la verificación de integridad y trazabilidad documental en contratos laborales representa una oportunidad concreta para que las organizaciones dejen de depender de la buena fe o de revisiones manuales para confiar en sus propios documentos. En un contexto donde los contratos laborales son la base de múltiples decisiones jurídicas y operativas, contar con un mecanismo que permita demostrar que un documento no fue alterado tiene un valor que va más allá de lo técnico [4, 13]. En Colombia, donde la transformación digital avanza más rápido que la capacidad de muchas organizaciones para gestionarla con seguridad, propuestas como esta aportan algo necesario: no solo tecnología, sino criterio sobre cómo usarla [5

1.5 OBJETIVOS

1.5.1 Objetivo General.

Desarrollar un MVP de un sistema web basado en blockchain permisionada para la verificación de integridad y trazabilidad documental en contratos laborales.

1.5.2 Objetivos Específicos.

Identificar, mediante análisis documental y consulta con actores clave, los requerimientos funcionales, no funcionales y normativos necesarios para el desarrollo del MVP de un sistema web y la implementación de una red blockchain permisionada.

Diseñar la arquitectura del sistema contemplando el diseño UX/UI, la arquitectura de software y la arquitectura de infraestructura sobre una red Hyperledger Fabric desplegada en Docker.

Desarrollar el MVP del sistema web integrado con una red Hyperledger Fabric desplegada en Docker, que permita el registro, verificación de integridad, consulta y trazabilidad de contratos laborales entre personas y empresa mediante el almacenamiento de hashes documentales en el ledger.

Evaluar el MVP mediante un plan de pruebas que verifique el comportamiento funcional de los módulos, la detección de alteraciones en documentos registrados y el rendimiento operativo del sistema.

1.6 MARCO REFERENCIAL

1.6.1 Marco Conceptual

Para comprender el funcionamiento del sistema propuesto, es necesario definir una serie de conceptos que sustentan tanto su operación técnica como su propósito documental. Estos conceptos están relacionados entre sí y permiten entender cómo una aplicación web puede garantizar la integridad y la trazabilidad de contratos laborales digitales mediante tecnología blockchain, así como la relevancia jurídica que este enfoque tiene en el contexto colombiano.

El primer concepto fundamental es la integridad documental, entendida como la capacidad de demostrar que un documento conserva exactamente el mismo contenido con el que fue creado o registrado, sin modificaciones no autorizadas. En el ámbito digital no se puede garantizar que un documento sigue siendo original solamente porque al observarlo se ve auténtico, pues para el ámbito jurídico esto no es una evidencia clara, por lo que se necesita contar con mecanismos confiables que brinden pruebas de que la información no se alteró y es legítima [4, 13]. Para los contratos laborales esta situación cobra relevancia ya que cualquier modificación en algún dato sensible puede generar conflictos o disputas legales [26].

La integridad del documento es un factor esencial; sin embargo, también se necesita poder demostrar cuándo fue registrado originalmente y conocer si hubo algún cambio sobre él, identificando que ocurrió, cuando ocurrió y quien realizó la acción. Este proceso se conoce como trazabilidad documental, haciendo referencia a la capacidad de reconstruir de forma verificable el historial de un documento a lo largo de todo su ciclo de vida [9, 21]. En consecuencia, un sistema que preserve tanto la integridad como la trazabilidad no solo protege la información, sino que mantiene un registro auditable de los eventos [21].

La verificación de la integridad se apoya en el uso de funciones hash criptográficas, que transforman información de cualquier tamaño en una huella única de longitud fija [23]. Esta huella actúa como una representación única del contenido: mientras el archivo permanezca intacto, el hash será el mismo; cualquier modificación producirá un resultado diferente, haciendo evidente la alteración [4, 27]. Este mecanismo está estandarizado [23] y es ampliamente utilizado en soluciones de gestión documental basadas en blockchain [26, 28].

La plataforma seleccionada es Hyperledger Fabric, cuya arquitectura modular separa los procesos de ejecución, ordenamiento y validación para ofrecer un desempeño eficiente en entornos empresariales [20]. Dentro de esta infraestructura, el chaincode ejecuta las reglas de negocio sobre el ledger, el cual tendrá el registro permanente de todas las operaciones, almacenando únicamente las huellas criptográficas y eventos verificables para proteger información sensible. Estos registros no se pueden modificar fácilmente ya que cada operación recibirá un identificador único (txID) que se encargará de mantener un rastreo. Además, las identidades de los usuarios son gestionadas por el Membership Service Provider (MSP), el componente encargado de verificar quiénes pueden acceder a la red y qué acciones tienen permiso para realizar [20].

Como conclusión, los elementos anteriormente mencionados son fundamentales para fortalecer el valor probatorio documental dentro del marco jurídico colombiano. Al aplicar estos aspectos se puede demostrar evidencia verificable, la autenticidad, la integridad y trazabilidad de la información registrada [2]. En este contexto, la firma electrónica permite saber quién realizó una acción o firmó un documento [3], mientras que el registro inmutable ayuda a demostrar que la información no ha sido modificada con el tiempo, demostrando que es transparente y confiable [29].

1.6.2 Marco Teórico.

1.6.2.1 Verificación de integridad documental mediante criptografía

El problema de demostrar que un documento no ha sido alterado desde su creación se viene abordando desde la criptografía aplicada. En 1991, Haber y Stornetta propusieron un mecanismo basado en sellos de tiempo y encadenamiento de resúmenes criptográficos que partía de la siguiente idea: la prueba de integridad no puede depender del propio documento ni del sistema que lo almacena, sino de un registro externo verificable de forma independiente [11]. Esa idea, que en su momento era teórica, terminó convirtiéndose en el fundamento de los sistemas de registro inmutable que hoy se conocen [11].

La verificación de la integridad de un documento puede realizarse mediante funciones hash criptográficas, las cuales generan una huella digital única para cada archivo. Esta huella siempre será la misma mientras el contenido permanezca sin cambios. Sin embargo, si el documento es modificado, incluso de manera mínima, la huella generada será completamente diferente. Gracias a esta característica es posible detectar alteraciones de forma rápida y automática [23].

Las funciones hash poseen características que las hacen útiles para verificar la integridad de los documentos, entre ellas el determinismo, la resistencia a colisiones y el efecto avalancha [30]. Para el sistema propuesto, la propiedad más relevante es el efecto avalancha, ya que permite detectar automáticamente cualquier modificación realizada en el documento, incluso si se trata de un cambio mínimo [24].

Para generar esas huellas, el proyecto utiliza SHA-256, algoritmo estandarizado por el NIST dentro de la familia SHA-2 [23]. Se eligió por dos razones prácticas: es el algoritmo de hash que Hyperledger Fabric emplea internamente en su cadena de bloques, lo cual evita inconsistencias entre la huella calculada por la aplicación y la que registra la red [20]. A la fecha no se conocen ataques que comprometan su resistencia a colisiones, lo que lo mantiene como una opción segura para este tipo de aplicaciones [23].

Ahora bien, también importa poder demostrar cuándo se registró, por lo que el protocolo RFC 3161 resuelve eso al definir cómo asociar una marca temporal a un dato de forma criptográficamente verificable [31]. Con este mecanismo, la evidencia no solo prueba que el contenido permanece intacto, sino que ancla ese estado a un momento específico en el tiempo [11, 31]. Este principio orienta el diseño del sistema: en el MVP, la fecha de cada registro la aporta la transacción confirmada en el ledger, y la integración con un servicio de estampado cronológico RFC 3161 se plantea como trabajo futuro [11, 31].

La combinación de las funciones hash y las marcas de tiempo permite generar evidencia confiable sobre los documentos digitales [11, 31]. Mientras que las huellas digitales ayudan a verificar que el contenido no ha sido modificado, las marcas de tiempo permiten demostrar cuándo fue registrado. De esta manera, es posible garantizar la integridad, autenticidad y trazabilidad de los documentos [31]

1.6.2.2 Blockchain permisionada como infraestructura de registro inmutable

En su forma más básica, una blockchain es un registro distribuido donde la información se agrupa en bloques y cada bloque incluye el hash del bloque anterior formando una cadena. Esa estructura tiene una consecuencia práctica directa, pues si alguien modifica un registro antiguo los hashes de todos los bloques posteriores dejan de coincidir y la alteración queda expuesta de forma matemática [9, 17]. La tecnología nació en redes públicas abiertas, pero con el tiempo aparecieron variantes pensadas para organizaciones que necesitan más control sobre quién participa y qué información se comparte [21].

Esas variantes son las blockchains permisionadas. A diferencia de una red pública, donde

cualquier persona puede unirse y ver todo, en una permisionada solo participan los que han sido identificados y autorizados previamente por una entidad administradora [21]. Eso es lo que las hace viables para manejar información contractual sensible: se conserva la inmutabilidad del registro, pero sin exponer documentos privados a quienes no deberían verlos [27].

En el ámbito de la gestión documental y legal, estas redes aportan registros que no se pueden modificar una vez escritos, un historial transparente de todo lo que ocurrió y la posibilidad de que varios participantes autorizados verifiquen la información por su cuenta [21, 22, 32]. Díaz Michel y Llamas Covarrubias [15] declaran que blockchain no sirve para reemplazar la redacción legal de un contrato; su valor real está en generar evidencia técnica de que ese contrato existe y no ha sido alterado.

Esa distinción es la que define el alcance de este proyecto. No se busca reemplazar el contrato laboral como instrumento jurídico ni crear documentos con autonomía legal propia. Lo que se hace es usar blockchain como una capa de confianza adicional que genera evidencia verificable sobre contratos que ya existen, reforzando así las garantías de integridad y trazabilidad en la relación laboral entre personas y empresas [9, 27].

1.6.2.3 Hyperledger Fabric como plataforma de implementación

Hyperledger Fabric es una plataforma blockchain de código abierto, mantenida por la Hyperledger Foundation bajo la Linux Foundation, y diseñada específicamente para redes permisionadas en entornos empresariales [20]. Lo que la diferencia de otras plataformas es su modelo de procesamiento de transacciones, llamado execute-order-validate. Inicialmente los peers ejecutan la transacción, luego un servicio de ordenamiento la agrupa con otras en un bloque, y finalmente se valida antes de escribirla en el ledger [20].

La arquitectura de la red está formada por varios componentes que trabajan juntos. Los peers almacenan una copia de la información registrada y ejecutan las reglas del sistema. El servicio de ordenamiento se encarga de organizar las transacciones y agruparlas antes de agregarlas a la cadena de bloques. Por su parte, el MSP administra la identidad de los usuarios y controla los permisos de acceso dentro de la red [20]. Además, existen los canales, que permiten que cierta información solo sea visible para grupos específicos de participantes, protegiendo así los datos sensibles.

Para que una aplicación pueda interactuar con la red blockchain, se utiliza el Fabric SDK. Este componente permite enviar transacciones, consultar información almacenada y gestionar identidades sin que el desarrollador tenga que trabajar directamente con los elementos internos de la red. En la práctica funciona como un intermediario entre la aplicación y la blockchain.

Que todo esto funcione fuera del laboratorio ya se ha probado. Rehan et al. [25] documentaron una implementación con contenedores Docker y CouchDB como base de datos de estado, y mostraron que esa combinación permite hacer consultas avanzadas sobre los datos almacenados y mantener comunicaciones seguras entre nodos. El presente proyecto adopta una arquitectura similar, lo que hace de ese estudio un referente directo.

El uso de Docker permite que todos los componentes de la red blockchain se ejecuten en entornos aislados. Gracias a ello, la solución puede replicarse fácilmente en distintas máquinas y resulta más sencilla de administrar, ya que los problemas en un componente no afectan directamente a los demás [25]. Para un proyecto académico, esto facilita las pruebas y el desarrollo en diferentes equipos.

1.6.2.4 Chaincode y automatización de lógica documental

Chaincode es el programa que contiene las reglas de negocio dentro de Hyperledger Fabric. Se instala en los peers de la red y define qué operaciones se pueden hacer sobre los datos del ledger y en qué condiciones [20]. Como todos los peers ejecutan el mismo código las reglas se aplican de manera uniforme sin importar cuál nodo procese la transacción.

En blockchain, el chaincode o contrato inteligente es un programa que ejecuta automáticamente determinadas acciones cuando se cumplen ciertas condiciones. Además de automatizar procesos, registra cada acción realizada, lo que permite conocer qué ocurrió y en qué momento sucedió [33, 34].

En este proyecto el chaincode no almacena contratos laborales completos, lo que guarda son las huellas criptográficas de esos documentos y los eventos asociados a su ciclo de vida; el registro inicial y las verificaciones posteriores. Cada una de esas acciones genera una transacción con un identificador único (txID), lo cual permite reconstruir con precisión qué se hizo, cuándo y en qué orden; las consultas, en cambio, leen el ledger sin generar transacciones [20].

Mier Bello et al. [24] probó esta idea en un sistema de integridad de datos científicos sobre Hyperledger Fabric y concluyeron que usar blockchain como capa de evidencia criptográfica en vez de como repositorio de archivos, permite detectar alteraciones sin necesidad de guardar los documentos originales en la cadena. En sus pruebas obtuvieron una tasa de detección del 100 % frente a modificaciones controladas, lo que respalda la viabilidad del modelo que adopta este proyecto.

Según Bannikov et al. [35], la automatización basada en blockchain es más efectiva cuando las reglas pueden ser supervisadas y verificadas por varios participantes. En este sistema, esa función la cumple el chaincode, ya que se encarga de ejecutar automáticamente las reglas de registro y verificación, asegurando que siempre se apliquen de la misma forma sin importar quién realice la operación [35].

1.6.2.5 Patrones de diseño en sistemas documentales con blockchain

No todos los sistemas documentales con blockchain se diseñan igual, pero la literatura muestra un patrón que se repite en las soluciones mejor logradas. Precht, Hüllmann y Marx Gómez [9], a partir de una revisión sistemática, identifican tres capas que suelen estar presentes. La primera son los documentos originales, que se guardan fuera de la blockchain en repositorios convencionales. La segunda son los metadatos y la información operativa, que normalmente viven en una base de datos convencional; en este proyecto, MongoDB. Y la tercera son las pruebas criptográficas de integridad, que son las únicas que se registran en la blockchain [9].

Esta estrategia permite aprovechar los beneficios de la blockchain sin almacenar directamente los documentos en la red. Los contratos permanecen en un almacenamiento externo, mientras que en la blockchain solo se registra su huella digital. De esta manera, es posible verificar la integridad del documento sin afectar el rendimiento de la red ni exponer información sensible.

Siguiendo una arquitectura utilizada en diversos proyectos blockchain, el sistema incorpora un backend que actúa como intermediario entre la interfaz de usuario y la red Hyperledger Fabric. Este componente se encarga de la autenticación, el control de acceso y la validación de las operaciones, evitando que el frontend interactúe directamente con la infraestructura blockchain [9].

En materia de seguridad, el diseño del sistema se basa en buenas prácticas inspiradas en la norma ISO/IEC 27001:2022 [36]. Aunque el proyecto no implementa un sistema de gestión de seguridad completo ni busca una certificación, sí adopta controles relacionados con la gestión de accesos, el registro de actividades, la protección de la integridad de los datos y la reducción de la exposición de información sensible.

La incorporación de estas prácticas permite fortalecer la seguridad del prototipo y justificar que las decisiones de diseño adoptadas están alineadas con estándares reconocidos internacionalmente [36].

1.6.2.6 Plan de pruebas y evaluación del sistema

Para verificar el correcto funcionamiento del sistema, se definió una estrategia de pruebas enfocada en los aspectos más importantes de la solución [7]. Estas pruebas incluyen la validación del registro de documentos, la generación de huellas criptográficas y la consulta de la información almacenada. También se realizan pruebas de integridad, modificando deliberadamente documentos previamente registrados para comprobar que el sistema detecta cualquier alteración. Adicionalmente, se evalúa la comunicación entre la interfaz web, el backend y la red Hyperledger Fabric para asegurar que todos los componentes funcionen de manera integrada. Los casos de prueba más críticos se automatizan para poder ejecutarlos repetidamente y garantizar la confiabilidad del sistema ante futuras modificaciones [7].

Además de evaluar las funcionalidades del sistema, también es necesario medir el desempeño de la red blockchain. Para ello, se consideran indicadores como la cantidad de transacciones que la red puede procesar, el tiempo que tarda en registrar una transacción y la consistencia de la información almacenada entre los distintos nodos [37, 38, 39]. Estos indicadores se complementan con métricas del sistema, como la capacidad para detectar alteraciones en los documentos, el tiempo de respuesta de las operaciones y la disponibilidad del historial de trazabilidad. En conjunto, estas medidas permiten determinar de forma objetiva si la solución cumple con los objetivos planteados para el MVP [39].

1.6.3 Marco Jurídico.

Al ser un proyecto que maneja documentos digitales sensibles, se deben cumplir las normas

que regulan la seguridad y protección de los datos personales, para ello se tomarán tantos los estándares internacionales como los nacionales.

1.6.3.1 Estándares internacionales

A nivel internacional, la norma principal para estos casos es la ISO/IEC 27001:2022 la cual define como se debe gestionar la seguridad de la información [36]. Aunque el proyecto no está centrado en todo el estándar, si toma principios como guía de diseño para: acceso de usuarios, separación de responsabilidades, protección de los contratos con su hash, y entre otros.

El enfoque principal que toma el proyecto sobre la norma ISO/IEC 27001:2022, es el Anexo A, el cual contiene un conjunto de controles de seguridad conocidos mundialmente para los accesos, cifrados, auditorias y manejo de incidentes [36] los cuales son primordiales al implementar herramientas propias del Hyperledger Fabric. En los alcances y limitaciones del proyecto se habla más a profundidad sobre el anexo.

Por último, también es importante incluir la norma ISO 15489-1:2016, orientada a la gestión de documentos y registros, establece que los sistemas documentales deben garantizar cuatro propiedades fundamentales: autenticidad, integridad, fiabilidad y disponibilidad [40].

1.6.3.2 Marco normativo colombiano

Pasando ahora a nivel nacional, la principal norma colombiana que sustenta el proyecto es la Ley 527 de 1999, considerada un pilar importante para el derecho informático del país, ya que establece el marco jurídico para los mensajes de datos, el comercio electrónico y las firmas digitales [2]. Esta ley incorpora el principio de equivalencia funcional, según el cual los documentos electrónicos pueden producir los mismos efectos jurídicos que los documentos físicos, siempre que se garantice su confiabilidad e integridad. En particular, los artículos 6 al 12 tienen una relación directa con el sistema propuesto, pues establecen que un mensaje de datos cumple con el requisito de constar por escrito cuando la información puede ser consultada posteriormente, y que su integridad debe evaluarse verificando que la información haya permanecido completa e inalterada, independientemente de los cambios de formato derivados de su transmisión, almacenamiento o presentación. En este sentido, el sistema satisface dicho requisito mediante el registro de la huella criptográfica SHA-256 en el ledger de Hyperledger Fabric, ya que cualquier modificación realizada sobre el documento genera un hash diferente, permitiendo detectar alteraciones y comprobar la integridad del contenido original en cualquier momento.

Por otro lado, el Decreto 2364 de 2012 reglamenta el artículo 7 de la Ley 527 de 1999 en lo referente a la firma electrónica, reconociendo la validez de distintos mecanismos tecnológicos para identificar a una persona y asociarla con una manifestación de voluntad sobre un documento digital [3]. El sistema desarrollado no pretende sustituir estos mecanismos de firma, sino complementarlos. En este contexto, cada usuario se autentica con su contraseña y un código de un solo uso enviado a su correo, y el sistema registra en el ledger, junto con cada acción, el usuario que la ejecutó. Por su parte, el Membership Service Provider (MSP) de Hyperledger Fabric controla qué organizaciones y nodos participan en la red y firman las transacciones. Así es posible determinar quién ejecutó cada acción del historial de trazabilidad

documental, siguiendo principios similares a los establecidos por la norma para la atribución verificable de actuaciones electrónicas.

Para la protección de datos personales, Colombia cuenta con la Ley 1581 de 2012 [16]. Los contratos laborales contienen información sensible de los trabajadores, como nombres, números de identificación, salarios o cargos, cuyo tratamiento debe ajustarse a principios como la finalidad, la necesidad y la minimización de datos. Para responder a estas exigencias normativas, el sistema evita almacenar el contenido de los contratos tanto en la base de datos (MongoDB) como en la red blockchain. En su lugar, únicamente registra la huella criptográfica SHA-256 de cada documento, permitiendo verificar su integridad sin exponer ni procesar directamente los datos personales contenidos en él.

Para terminar con la parte normativa, se toma en cuenta la guía de referencia para la adopción e implementación de proyectos con tecnología blockchain para el Estado colombiano elaborada por el MinTIC [1]. Esta guía destaca el potencial de las plataformas blockchain permisionadas para garantizar la integridad, trazabilidad y transparencia de los registros en entornos institucionales, promoviendo su adopción en escenarios donde la inmutabilidad de la información y el control de acceso son requisitos esenciales. Estas características coinciden directamente con los objetivos del sistema propuesto. Aunque la guía está orientada principalmente a entidades del sector público, los criterios técnicos y las recomendaciones que plantea también resultan aplicables a organizaciones privadas interesadas en implementar soluciones basadas en blockchain dentro del contexto colombiano.

1.7 DISEÑO METODOLÓGICO

La estrategia metodológica de este proyecto se define bajo un enfoque mixto, integrando la investigación aplicada para la fundamentación técnica-normativa y un ciclo de vida de software para la construcción del MVP. Esta combinación garantiza que el desarrollo técnico no solo sea funcional, sino que esté rigurosamente alineado con el marco legal colombiano y las necesidades de integridad documental.

Figura 1. Visualización de la metodología



Fuente: Desarrollado mediante Inteligencia Artificial

1.7.1 Diseño de la Investigación

Se adopta una investigación aplicada con alcance descriptivo y enfoque cualitativo.

Investigación Aplicada: Busca utilizar el conocimiento tecnológico de blockchain para resolver el problema específico de la alteración de contratos laborales [20], [36].

Alcance Descriptivo: Permite caracterizar las propiedades de integridad y trazabilidad requeridas, detallando cómo la arquitectura de Hyperledger Fabric satisface estas necesidades [28], [20].

1.7.2 Ciclo de Vida del Software (SDLC): Prototipado Evolutivo

Dado que el equipo está conformado por dos personas, se ha seleccionado el modelo de Prototipado Evolutivo, este modelo permite:

Reducción de riesgos: Validar la conexión con la red blockchain (la parte más compleja) de forma temprana.

Refinamiento continuo: Ajustar la interfaz de usuario (UX/UI) y la lógica del chaincode conforme se estabiliza la integración.

Esta metodología es perfecta para el enfoque del MVP ya que Prioriza las funcionalidades críticas (registro de hash y consulta de integridad) para asegurar un producto funcional.

Figura 2. Ciclo de vida del software



Fuente: Desarrollado mediante Inteligencia Artificial

Fases del Proyecto y Correspondencia con Objetivos

A continuación, se detallan las fases metodológicas, las actividades clave y los entregables, vinculándolos directamente con los Objetivos Específicos (OE).

Fase 1. Análisis, Benchmarking y Definición de Requerimientos (Relacionado con OE1)

En esta fase se recolecta la información base para la viabilidad del sistema.

Procedimiento:

Análisis Normativo: Revisión cualitativa de la Ley 527 de 1999 y Ley 1581 de 2012 y los controles de integridad de la información y requisitos de criptografía establecidos en el Anexo A de la norma ISO/IEC 27001:2022. para definir los criterios de integridad y privacidad (no almacenamiento de datos sensibles en el ledger).

Benchmarking Metodológico: Se realiza un análisis comparativo entre la solución propuesta (Hyperledger Fabric + Docker) frente a soluciones comerciales (ej. Google Cloud Platform). Se evalúan criterios como costo, inmutabilidad criptográfica, control de datos y valor jurídico en Colombia. Revisar Anexo C.

Levantamiento de Requerimientos: Traducción de necesidades en historias de usuario priorizadas. Entregables: Matriz de trazabilidad normativa, Cuadro comparativo de benchmarking y Backlog del MVP. Revisar anexo A para el backlog.

Fase 2. Diseño de Arquitectura y Modelado del Sistema (Relacionado con OE2)

Traducción de los requerimientos a especificaciones técnicas detalladas.

Procedimiento

Diseño de Arquitectura: Definición de la estructura de capas (Presentación, Aplicación, Blockchain y Persistencia).

Modelado de Red Blockchain: Configuración de la topología de Hyperledger Fabric (Organizaciones, Peers, Orderers y Canales) en entornos Docker.

Diseño UX/UI: Creación de prototipos de baja y alta fidelidad enfocados en la simplicidad de la verificación documental.

Entregables: Diagramas UML (Componentes, Secuencia y Despliegue), Diseños de interfaz y Especificación técnica del chaincode.

Fase 3. Desarrollo e Integración del MVP (Relacionado con OE3)

Construcción del sistema siguiendo el enfoque de prototipado evolutivo.

Procedimiento

Configuración de Infraestructura: Despliegue de la red permisionada mediante Docker Compose. Lógica de Negocio (Chaincode): Desarrollo de las funciones en Go para el registro y consulta de hashes SHA-256.

Desarrollo Frontend y Backend: Construcción de la aplicación web e integración con el

Fabric SDK para comunicar la interfaz con la red blockchain.

Entregables: Código fuente del MVP, Script de despliegue de la red y API de integración funcional.

Fase 4. Evaluación, Pruebas de Integridad y Rendimiento (Relacionado con OE4)

Esta fase tiene como objetivo validar la calidad, seguridad y eficiencia del MVP. No se limita a la ejecución aleatoria de tareas, sino que se rige por un Plan de Pruebas de Software (PPS) formalmente estructurado, que garantiza que cada requerimiento definido en la Fase 1 sea verificado.

Procedimiento:

Diseño del Plan de Pruebas: Se definen los casos de prueba, los datos de entrada, los resultados esperados y los criterios de aceptación para cada módulo.

Pruebas de comportamiento funcional (Caja Negra): Validación de que cada función del sistema web (carga, registro, consulta) se ejecute según lo definido en las historias de usuario. Se verifica que el flujo entre el usuario, la API y el SDK de Hyperledger Fabric sea consistente.

Pruebas de Integridad y Seguridad (Mutación Documental): Ejecución de pruebas de estrés sobre la integridad de los datos. Se introducen modificaciones intencionadas en archivos PDF ya registrados (cambios en el texto, firmas o metadatos) para confirmar que el sistema detecte la modificación al comparar el hash generado contra el hash almacenado en el ledger.
Pruebas de Trazabilidad e Inmutabilidad: Verificación técnica de que el historial de transacciones recuperado desde la blockchain coincida cronológicamente con las acciones realizadas sobre el documento, asegurando la propiedad de norepudio.

Pruebas de Rendimiento Operativo: Medición sistemática de latencias (tiempos de respuesta) en dos momentos críticos: la escritura del hash en la red permisionada y la consulta de verificación por parte del usuario final.

Entregables: Documento del Plan de Pruebas de Software. Especificación de entornos, herramientas y casos de prueba. Informe de Resultados de Pruebas y Defectos: Resumen de éxitos, fallas encontradas y acciones correctivas tomadas. Matriz de Cumplimiento de Requerimientos: Documento que cruza los requerimientos iniciales (Fase 1) con los resultados de las pruebas, demostrando la cobertura total del proyecto. Manual de Usuario y Operación: Guía técnica para el despliegue en Docker y uso del sist

2. DESARROLLO DEL PRIMER OBJETIVO

El primer objetivo específico se completó siguiendo la guía de la fase 1 de la metodología planteada. Esta fase está soportada por seis entregables que conforman la base de requerimientos validados del MVP: la matriz de trazabilidad normativa, el guion de entrevista aplicado a la empresa, la matriz de actores, el cuadro de benchmarking, el documento de especificación de requerimientos (SRS v2.0) y el backlog de historias de usuario.

2.1 Planteamiento del objetivo y correspondencia metodológica

El propósito del primer objetivo es identificar y definir los requerimientos que servirán como base para desarrollar el sistema. De esta manera, las decisiones que se tomen durante el proyecto estarán respaldadas por necesidades reales del negocio y por las normas que aplican al sistema.

Este objetivo se desarrolló durante la Fase 1 de la metodología, mediante cuatro actividades:

1. **Análisis normativo:** Se revisaron la Ley 527 de 1999, el Decreto 2364 de 2012 y la Ley 1581 de 2012, junto con los controles de criptografía e integridad del Anexo A de ISO/IEC 27001:2022. Esto permitió identificar los aspectos legales y de seguridad que debían tenerse en cuenta para el diseño del sistema, especialmente en relación con la integridad de los documentos y la protección de los datos personales.
2. **Consulta con actores clave:** Se diseñó y realizó una entrevista semiestructurada a la empresa objeto de estudio, con el fin de conocer cómo se realiza actualmente la gestión de contratos, cuáles son los principales problemas y qué necesidades existen frente a la seguridad y trazabilidad de los documentos. Además, se elaboró una matriz de actores para identificar a las personas y grupos interesados en el proyecto.
3. **Benchmarking metodológico:** Se realizó una comparación entre la solución propuesta y una alternativa comercial como Google Cloud Platform (GCP), teniendo en cuenta aspectos como los costos, la seguridad, la infraestructura y el cumplimiento normativo. Esto permitió justificar las decisiones tomadas para el diseño del sistema.
4. **Levantamiento y especificación de requerimientos:** A partir de la información recopilada, se definieron los requerimientos funcionales y no funcionales del MVP. También se elaboraron las historias de usuario y el backlog, que reúne las funcionalidades, mejoras y tareas necesarias para desarrollar el sistema, organizadas según su prioridad.

El objetivo 1 se midió en base a dos indicadores clave de desempeño. El KPI-1.1, correspondiente a la cobertura de requerimientos documentados en el SRS, establece que se debe formalizar al menos el 90 % de los requerimientos identificados. Por su parte, el KPI-1.2, relacionado con la cobertura de las disposiciones normativas mapeadas al SRS, exige cubrir al menos el 90 % de las disposiciones aplicables de las normas analizadas.

2.2 Análisis normativo y matriz de trazabilidad

Para realizar el análisis normativo, se tuvo en cuenta que el sistema no busca reemplazar los mecanismos jurídicos existentes, como la firma electrónica, sino generar evidencia que permita identificar y hacer seguimiento a los cambios o modificaciones que pueda tener un documento. Esto permite orientar el diseño del sistema de acuerdo con el marco legal colombiano relacionado con los documentos electrónicos.

Para relacionar las normas con el funcionamiento del sistema, se elaboró una matriz de

trazabilidad normativa, en la que se identifican las disposiciones aplicables y los requerimientos que permiten darles cumplimiento.

Se analizaron tres normas y un estándar, cada uno con un aporte diferente al proyecto:

- **Ley 527 de 1999** (mensajes de datos, comercio electrónico y firmas digitales): aporta los criterios de equivalencia funcional, integridad del mensaje de datos (Art. 8º y 9º) y fuerza probatoria (Art. 10 y 11). Es la norma que da sustento jurídico a tratar el contrato en PDF como evidencia válida y a que la huella SHA-256 constituya la "garantía confiable" de conservación de la integridad.
- **Decreto 2364 de 2012** (firma electrónica): reglamenta el requisito de firma. Varias de sus disposiciones quedaron declaradas fuera del alcance funcional del MVP, por cuanto el sistema no emite ni valida firmas, aunque sí cubre parcialmente el criterio de detección de alteraciones posteriores (Art. 4º).
- **Ley 1581 de 2012** (protección de datos personales): aporta los principios de finalidad, acceso restringido, seguridad y confidencialidad. Es la norma que justifica el patrón *off-chain* del sistema —registrar únicamente la huella y metadatos mínimos, nunca el contenido del contrato con datos personales.
- **ISO/IEC 27001:2022** (Anexo A, controles A.8.24 (uso de criptografía) y A.8.11 (enmascaramiento de datos)): se adoptaron como criterios de diseño, no como objeto de certificación ni auditoría.

La matriz registra la norma, el artículo o control correspondiente, un resumen de la disposición, los requerimientos funcionales o no funcionales relacionados, la forma en que se contempla su cumplimiento en el sistema y el estado de cobertura, que puede ser Cubierto, Cubierto parcialmente o Fuera de alcance. De esta manera, es posible identificar la relación entre las disposiciones revisadas y los requerimientos definidos para el MVP.

Tabla 1. Resultados del KPI-1.2

Estados de Cobertura	Disposiciones	Proporción
Cubierto	13	62%
Cubierto Parcialmente	4	19%
Fuera de alcance (firma electrónica, documentado en Alcances y	4	19%
Total Analizado	21	**100%**

Fuente: Desarrollado por las autoras

Para calcular este indicador, se excluyeron las disposiciones relacionadas con la firma electrónica que quedaron fuera del alcance funcional del MVP. De esta manera, el universo normativo aplicable quedó conformado por 15 disposiciones: 12 cubiertas totalmente y 3 cubiertas parcialmente.

Esto representa una cobertura del 100 % del universo normativo aplicable, por lo que se supera el umbral del 90 % establecido para el KPI-1.2. Los controles A.8.24 y A.8.11 de ISO/IEC 27001:2022 se registraron por separado, debido a que se utilizaron como criterios de diseño.

2.3 Consulta con actores clave

La consulta con los actores clave se realizó mediante una entrevista semiestructurada a la empresa objeto de estudio. Esta entrevista se organizó en cinco bloques: proceso actual de gestión de contratos, actores y roles, cuantificación del problema, normativa y seguridad, y

necesidades del sistema. La información recopilada permitió conocer mejor el proceso actual de la empresa, identificar sus principales dificultades y definir los requerimientos del negocio que debía contemplar el MVP.
A partir de la entrevista, se confirmó que uno de los principales problemas es la gestión manual de los contratos. Actualmente, el asesor jurídico redacta el documento, el asesor de reclutamiento se encarga de gestionar las firmas manuscritas y, posteriormente, el documento se escanea y se almacena en SharePoint. Sin embargo, no existe un mecanismo técnico que permita comprobar si el documento fue modificado después de la firma. Por esta razón, cuando se necesita identificar cuál es la versión válida de un contrato, es necesario realizar una revisión manual que puede tardar horas o incluso días.
La empresa gestiona aproximadamente 20 documentos al año entre contratos de proveedores, nómina y prestación de servicios. Además, antes de llegar a la versión final, los documentos suelen pasar por dos o tres versiones. Estos datos sirvieron como referencia para definir el conjunto de 20 documentos sintéticos utilizado en la validación del sistema, conformado por 12 contratos y 8 acuerdos de confidencialidad (NDA).
La entrevista también permitió identificar la necesidad de manejar diferentes roles dentro del sistema, separando las funciones del administrador, los empleados y el verificador. Asimismo, se estableció la necesidad de contar con un inicio de sesión mediante correo corporativo y un código de verificación para reforzar la seguridad del acceso.

2.3.1 Matriz de Actores (stakeholders): Después de la entrevista y de revisar el alcance del proyecto, se elaboró una matriz de actores para organizar a los interesados e identificar su relación con los requerimientos del sistema. Esta organización también permitió asignar cada requerimiento a un único actor responsable. Se identificaron seis actores:

Tabla 2. Stakeholders

Actor / Stakeholder	Rol frente al sistema	RF	RNF	Total
Todos los usuarios autenticados	Transversal a los 3 roles	11	11	22
Empleado (firmante)	Carga y consulta de sus documentos	15	2	17
Verificador	Auditoría de integridad documental	18	3	21
Administrador	Gobierno del sistema y de los usuarios	11	1	12
Actor normativo	Impone restricciones de diseño (no opera el sistema)	0	7	7
Equipo de desarrollo y evaluación	Construye, despliega y evalúa el MVP	0	6	6
Total		**55**	**30**	**85**

Fuente: Desarrollado por las autoras

Uno de los aspectos más importantes de esta matriz es que incluye tanto a las personas que utilizarán el sistema como a aquellos interesados que, aunque no lo operan directamente, influyen en su desarrollo. Por ejemplo, el actor normativo y el equipo de desarrollo establecen

condiciones que deben tenerse en cuenta, especialmente en temas de seguridad, arquitectura y cumplimiento.
De esta manera, la matriz de actores se relaciona con la matriz de trazabilidad normativa, ya que permite conectar las necesidades legales y del negocio con los interesados y los requerimientos que se definieron para responder a ellas.
Adicionalmente, la matriz incluye una clasificación de cada interesado según su nivel de interés e influencia, lo que permite establecer una estrategia para gestionar su participación durante el proyecto.

2.4 Benchmarking metodológico

Con el propósito de justificar las decisiones tomadas para el diseño del sistema, se realizó un benchmarking entre la solución propuesta y una alternativa comercial basada en Google Cloud Platform (GCP).
La solución propuesta utiliza Hyperledger Fabric, Docker y Azure for Students, con un backend desarrollado en Node.js y contratos inteligentes (*chaincode*) en Go. Por su parte, la alternativa de GCP contempla servicios como Cloud Storage con Bucket Lock, Cloud KMS, Cloud Run y Document AI. La comparación se realizó teniendo en cuenta cinco aspectos: costo total de propiedad, infraestructura y despliegue, seguridad e integridad documental, cumplimiento normativo colombiano y dependencia tecnológica. Este análisis se utilizó como una herramienta para tomar decisiones de diseño, no como una auditoría de cumplimiento.

Tabla 3. Benchmarking

Dimensión	MVP Hyperledger Fabric + Docker	Google Cloud Platform
Costo total estimado (4 meses)	$23.270.112 COP (alineado al presupuesto aprobado)	~$29.408.832 COP (escenario de máxima funcionalidad)
Inmutabilidad	Matemática y distribuida (hash SHA-256 encadenado)	Contractual y centralizada (Bucket Lock WORM)
Control de acceso	Permisionado, 3 roles definidos en chaincode	IAM por servicio; requiere desarrollo adicional de roles
Soberanía de datos	Colombia (Azure Student), sin transferencia internacional	Variable (Document AI procesa en EE. UU.)
Dependencia del proveedor (vendor lock-in)	Nula (open source Apache 2.0, portable con Docker)	Alta (APIs propietarias, migración bloqueada por retención)
Curva de implementación	Alta (certificados X.509, YAML, ciclo de vida del chaincode)	Baja (integración vía REST/SDK, red abstraída)

Fuente: Desarrollado por las autoras

A partir de la comparación, se determinó que Hyperledger Fabric se ajusta mejor a las necesidades actuales del proyecto por tres razones principales. Primero, permite desarrollar la solución dentro del presupuesto disponible. Segundo, facilita mantener el control sobre la

infraestructura y los datos, según la configuración del despliegue. Tercero, permite registrar evidencias de forma que los cambios posteriores puedan detectarse mediante la comparación de las huellas digitales de los documentos.
También se identificó que GCP podría ser una alternativa conveniente si el sistema llega a una etapa de producción con un mayor volumen de documentos, debido a los servicios administrados que ofrece y a las facilidades de escalabilidad.
Por lo tanto, el benchmarking permitió justificar por qué se eligió Hyperledger Fabric para el MVP y dejó documentada una alternativa que podría evaluarse si las necesidades del sistema cambian en el futuro.

2.5 Levantamiento y especificación de requerimientos

Las necesidades identificadas se registraron en un documento de especificaciones de requerimientos (SRS) elaborado conforme a la norma ISO/IEC/IEEE 29148:2018 (ingeniería de requisitos) y con los atributos de calidad de ISO/IEC 25010:2023. Cada requisito está planteado en función a nueve criterios de calidad (necesario, apropiado, no ambiguo, completo, singular, factible, verificable, correcto y conforme), lo que permite que cada enunciado posteriormente pueda comprobarse mediante las pruebas del sistema.
El SRS registra 55 requerimientos funcionales (RF) y 30 requerimientos no funcionales (RNF), para un total de 85 requerimientos. Los funcionales se organizan por módulos que reflejan los roles del sistema:

- **Autenticación y acceso (RF-001 a RF-007):** inicio de sesión con correo corporativo, segundo factor mediante token con vigencia de 5 minutos, asignación automática de rol único (Administrador, Empleado o Verificador) y cierre de sesión seguro.
- **Rol Empleado (RF-008 a RF-022):** carga de documentos PDF (máximo 20 MB, uno por operación), generación de la huella SHA-256, registro de la huella y los eventos en la red blockchain (nunca el archivo), y consulta del estado (Pendiente, Verificado o Alterado) y de la trazabilidad, incluido el identificador de transacción (txID) de Hyperledger Fabric.
- **Rol Verificador (RF-023 a RF-040):** selección de un documento registrado, carga de la versión a comparar, generación y comparación de huellas, determinación automática del resultado (integridad confirmada o comprometida), actualización del estado y registro de la verificación en la trazabilidad.
- **Rol Administrador (RF-041 a RF-051):** panel con el resumen documental consolidado, gestión del ciclo de vida de usuarios (creación, edición y desactivación lógica sin perder la trazabilidad) y consulta de la trazabilidad administrativa.
- **Gestión de credenciales y seguridad de acceso (RF-052 a RF-055):** cambio de contraseña, bloqueo temporal por intentos fallidos y aceptación registrada de la política de tratamiento de datos.

Los 30 requerimientos no funcionales cubren seguridad (hashing con *salt*, RBAC, no almacenamiento de contenido en el ledger, inmutabilidad de la evidencia), rendimiento (percentil 95 de latencia ≤ 3.000 ms en las operaciones principales), disponibilidad (≥ 95 % durante el mes de pruebas), confiabilidad (detección del 100 % de las alteraciones sobre los 20 documentos), usabilidad, mantenibilidad y arquitectura (modular, contenerizada, con la comunicación a Fabric a través del backend) y auditoría. Varios requerimientos incorporan

notas de cambio respecto de la versión anterior, por ejemplo, la incorporación del campo txID en la trazabilidad (RF-022, RF-035, RF-051) o la referencia explícita a MongoDB y Hyperledger Fabric en la consistencia de información (RNF-013), lo que evidencia la naturaleza evolutiva del levantamiento.

2.6 Backlog del MVP

A partir del SRS, se elaboró un backlog conformado por 24 historias de usuario, organizadas en cinco épicas y estimadas mediante puntos de historia. Cada historia sigue la estructura: Como [rol], quiero [acción], para [beneficio], lo que permite identificar quién necesita una funcionalidad, qué espera realizar y para qué la necesita.

Además, cada historia se relaciona con los requerimientos funcionales y no funcionales correspondientes, se prioriza mediante el método MoSCoW y se asigna a un sprint. En total, el backlog suma 107 puntos de historia.

Tabla 4. Backlog

Épica	Historias	Enfoque
EP-01 Autenticación y control de acceso	6	Inicio de sesión, doble factor, cierre, cambio de contraseña, bloqueo y política de datos
EP-02 Registro y gestión documental (Empleado)	5	Carga, generación de huella, registro en blockchain y consultas
EP-03 Verificación de integridad (Verificador)	7	Selección, comparación de huellas, resultado y trazabilidad
EP-04 Administración del sistema	5	Panel, gestión de usuarios y trazabilidad administrativa
EP-05 Infraestructura y despliegue	1	Despliegue de la red Fabric y el entorno con Docker Compose

Fuente: Desarrollado por las autoras

El MVP se concentra en 22 historias "Must" y 2 "Should", priorizadas mediante MoSCoW. La planificación y el control de calidad del proyecto se estructuran de la siguiente manera:

2.6.1

Cronograma de Sprints:

- **Sprint 0:** Preparación de la infraestructura de la red permisionada, correspondiente a la HU-021, con una estimación de 13 puntos de historia.
- **Sprint 1:** Desarrollo de la autenticación y el acceso al sistema.
- **Sprint 2:** Implementación del registro documental.
- **Sprint 3:** Desarrollo de las funciones de verificación de documentos.
- **Sprint 4:** Implementación de las consultas y funciones administrativas.

Control de Calidad:

- **Criterios:** Evaluación bajo la metodología INVEST.
- **Definición de Terminado (DoD):** Revisión de pares, integración en GitHub, pruebas aprobadas (PASS), SRS actualizado, despliegue en Docker y aceptación de la dirección.
- **Trazabilidad:** 51 escenarios de aceptación en Gherkin que conectan las historias con las pruebas funcionales.

2.7 Resultados frente a los indicadores

Tabla 5. Resultados KPIs O1

Indicador	Meta	Resultado obtenido	Estado
KPI-1.1 — Cobertura de requerimientos documentados en el SRS	≥ 90 %	85/85 requerimientos formalizados en el SRS v2.0 (100 %), asignados a 6 actores sin doble conteo	Cumplido
KPI-1.2 — Cobertura de disposiciones normativas mapeadas al SRS	≥ 90 %	15/15 disposiciones aplicables al MVP mapeadas (100 %): 12 totales y 3 parciales	Cumplido

Fuente: Desarrollado por las autoras

Finalmente, los resultados de los dos indicadores muestran que se superó el umbral del 90 % establecido para el primer objetivo específico. El KPI-1.1 refleja que todos los requerimientos identificados quedaron documentados en el SRS y relacionados con un actor responsable. Por su parte, el KPI-1.2 muestra que todas las disposiciones normativas incluidas dentro del alcance del sistema fueron relacionadas con al menos un requerimiento.

En conjunto, estos resultados evidencian que se logró definir y documentar el alcance del MVP, relacionando las necesidades identificadas en la empresa con los requerimientos del sistema y las disposiciones normativas aplicables. Esto proporciona una base para continuar con las siguientes fases del proyecto y verificar, mediante las pruebas correspondientes, que la solución desarrollada cumpla con lo establecido.

3. DESARROLLO DEL SEGUNDO OBJETIVO

El segundo objetivo específico se desarrolló a través de la Fase 2 de la metodología, correspondiente al diseño de arquitectura y modelado del sistema. En esta fase, los requerimientos definidos en el SRS v2.0 se utilizaron como base para elaborar los artefactos de diseño del MVP, organizados en cuatro aspectos principales: el modelado funcional y los drivers de arquitectura, la arquitectura de software, la arquitectura de infraestructura sobre Hyperledger Fabric desplegada en Docker y el diseño UX/UI.

3.1 Planteamiento del objetivo y correspondencia metodológica

El propósito de este objetivo es definir como se organiza el sistema antes de comenzar su desarrollo, tomando como base los 85 requerimientos identificados en el primer objetivo. Para esto, se elaboraron diferentes documentos que permiten visualizar el funcionamiento del sistema, su estructura interna, la infraestructura necesaria para ejecutarlo y la forma en que los usuarios interactúan con él.

Este objetivo se desarrolló durante la Fase 2 de la Metodología, el cual se organizó en cuatro actividades:

1. **Modelado funcional y drivers de arquitectura:** Se elaboró el diagrama de casos de uso, la matriz de requerimientos arquitectónicamente significativos (ASR) y el utility tree. Estos permiten identificar las funcionalidades del sistema y los atributos de calidad que deben tenerse en cuenta al momento de diseñar su arquitectura.
2. **Arquitectura de software:** Se diseñaron el diagrama de componentes por capas, el modelo entidad-relación (MER) y los diagramas de secuencia para cada rol. Con estos elementos se definió cómo se organiza internamente el sistema y cómo se relacionan sus componentes.
3. **Arquitectura de infraestructura:** Se elaboró el diagrama de despliegue y el diseño de la red Hyperledger Fabric sobre Docker, con el fin de establecer los componentes necesarios para ejecutar el sistema y la forma en que se comunican entre sí.
4. **Diseño UX/UI:** Se realizaron los wireframes de baja fidelidad y el prototipo de alta fidelidad, los cuales definen la distribución de las pantallas y la forma en que los usuarios interactúan dentro del sistema.

El cumplimiento de este objetivo se evaluó mediante dos indicadores. El KPI-2.1, relacionado con los artefactos de diseño aprobados por el tutor, establece que los cuatro dominios deben estar aprobados antes de iniciar el desarrollo, previsto para la semana 7.
Por su parte, el KPI-2.2, correspondiente a la tasa de observaciones críticas durante la revisión del diseño, establece que estas no deben superar el 10 % de los componentes revisados, tomando como referencia un máximo de dos observaciones críticas sobre aproximadamente 20 componentes.

3.2 Modelado funcional y drivers de arquitectura

Se definieron las funcionalidades que debe tener el sistema y los aspectos de calidad que influyen en las decisiones de arquitectura, permitiendo establecer una base para el diseño antes de definir cómo se organizarían internamente los componentes.

3.2.1 Diagrama de casos de uso

Se elaboró un diagrama de casos de uso para representar las funcionalidades del sistema y la relación que tienen con cada uno de los roles. Se definieron tres roles principales: Empleado, Verificador y Administrador. Los tres roles comparten funcionalidades como iniciar y cerrar sesión, aceptar la política de tratamiento de datos y cambiar la contraseña. Además, el diagrama representa las relaciones entre los casos de uso, incluyendo las acciones que requieren que el usuario haya iniciado sesión previamente.
También se dejó establecido que cada usuario tiene un rol asignado a su cuenta, por lo que no puede seleccionarlo libremente al momento de ingresar. Finalmente, los casos de uso se relacionaron con los requerimientos funcionales del SRS, desde el RF-001 hasta el RF-055, para mantener la trazabilidad entre las funcionalidades definidas y el diseño del sistema.

3.2.2 Matriz de requerimientos arquitectónicamente significativos (ASR)

Se elaboró una matriz de requerimientos arquitectónicamente significativos (ASR), con el propósito de identificar cuáles requerimientos tienen mayor influencia en las decisiones de arquitectura.
Para esto, se establecieron 27 escenarios de calidad, cada uno descrito mediante seis elementos: la fuente, el estímulo, el entorno, el artefacto afectado, la respuesta esperada y la medida que permite comprobar dicha respuesta. Esto facilita que los aspectos de calidad no se queden únicamente como características generales, sino que puedan evaluarse mediante criterios verificables. Además, cada requerimiento se evaluó teniendo en cuenta dos aspectos: el Valor de Negocio (VN), que representa su importancia para el proyecto, y el Impacto Arquitectónico (IA), que permite identificar qué tan complejo sería resolverlo o modificarlo posteriormente.
A partir de esta evaluación, se asignó un nivel de prioridad y se definió cómo se verificaría cada requerimiento, de esta manera la matriz permitió identificar los aspectos que debían tenerse en cuenta desde el diseño de la arquitectura, evitando dejar decisiones importantes para etapas posteriores del desarrollo.

Tabla 6. Requerimientos arquitectónicamente significativos

ASR	Escenario	Medida de la respuesta	(VN, IA)
ASR-07	Registro en blockchain sin almacenar el PDF	100 % de 30 transacciones solo con hash y metadatos; 0 PDF en el ledger	(Alta, Alta)
ASR-15	Consistencia entre MongoDB y Hyperledger Fabric	0 inconsistencias en 50 operaciones con 10 fallos inyectados; reconciliación ≤ 60 s	(Alta, Alta)
ASR-16	Inmutabilidad de la evidencia en blockchain	0 modificaciones sobre 20 intentos; orden y txID verificables	(Alta, Alta)
ASR-17	Detección confiable de alteraciones	100 % de alteraciones detectadas sobre 20 documentos; 0 falsos negativos	(Alta, Media)
ASR-19	Arquitectura modular por capas	Un cambio tipo toca ≤ 1 módulo y no obliga a redesplegar el chaincode	(Alta, Alta)

Fuente: Desarrollado por las autoras

3.2.3 Utility Tree

Se elaboró un Utility Tree para organizar los escenarios de calidad que fueron aprobados, teniendo en cuenta cuatro niveles: utilidad, característica de calidad según ISO/IEC 25010, subcaracterística y escenario medible, permitiendo organizar los aspectos de calidad que debe cumplir el sistema y relacionarlos con criterios que se puedan verificar.

Para definir el orden en que se trabajarían estos aspectos se tuvo en cuenta la priorización realizada en la matriz ASR según el Valor de Negocio (VN) y el Impacto Arquitectónico (IA). Por esta razón, los escenarios clasificados como “Alta-Alta” se abordaron primero, ya que tienen una gran importancia para el proyecto y sería más complejo realizar cambios sobre ellos posteriormente. Entre estos se encuentran la consistencia de los datos (ASR-15), la inmutabilidad (ASR-16) y la modularidad (ASR-19).

De esta manera, el Utility Tree permite relacionar las decisiones tomadas durante el diseño de la arquitectura con los atributos de calidad que se busca cumplir y con los requerimientos de los que se originan.

3.3 Arquitectura de software

En este dominio se definió cómo está organizado internamente el sistema, tomando como referencia el escenario ASR-19, relacionado con la modularidad por capas. Para representar esta estructura y el funcionamiento de sus componentes, se elaboraron tres tipos de diagramas UML: el diagrama de componentes, el modelo entidad-relación (MER) y los diagramas de secuencia.

3.3.1 Diagrama de componentes

Se diseñó una arquitectura organizada en cuatro capas, cada una con responsabilidades específicas:

- **Presentación (Frontend React SPA):** Corresponde a la interfaz con la que interactúan los usuarios. Se divide en cuatro módulos: autenticación, empleado, verificador y administrador (de acuerdo con las funcionalidades de cada rol).
- **Aplicación (Backend Node.js + Express):** Se encarga de procesar las solicitudes del sistema y coordinar las operaciones. Incluye los controladores de autenticación, documentos, verificación y administración, además de servicios para generar las huellas SHA-256, comunicarse con Hyperledger Fabric, gestionar usuarios y documentos, validar el acceso mediante JWT y roles, y enviar los códigos de verificación por correo electrónico.
- **Persistencia (MongoDB):** Se utiliza para almacenar la información de los usuarios, los metadatos de los documentos y los índices necesarios para consultar la trazabilidad.
- **Blockchain (Hyperledger Fabric):** Contiene los componentes encargados de registrar y consultar las huellas de los documentos, actualizar sus estados y guardar los eventos de trazabilidad. Para esto, se utiliza Fabric Gateway, el chaincode desarrollado en Go y el ledger donde quedan registrados los datos correspondientes.

En este diseño se establecieron dos decisiones importantes. La primera es que el frontend no se comunica directamente con la blockchain, sino que todas las solicitudes pasan por el backend, de acuerdo con el RNF-020. La segunda es que el PDF original no se almacena en el ledger; únicamente se registran las huellas digitales y los eventos de trazabilidad, como se establece en el RNF-004. Así, Hyperledger Fabric conserva el registro utilizado para comprobar

la integridad de los documentos, mientras que MongoDB permite almacenar los metadatos y facilita las consultas necesarias para el funcionamiento del sistema.

3.3.2 Modelo entidad-relación (MER)

Se elaboró el modelo entidad-relación para definir cómo se organiza la información que necesita el sistema. Para esto, se establecieron tres entidades principales: Usuario, Documento y EventoTrazabilidad.

La entidad Usuario contiene la información necesaria para la autenticación, el estado de la cuenta, el control de intentos fallidos y el consentimiento relacionado con el tratamiento de datos personales. La entidad Documento registra información como el nombre del archivo, el responsable, la fecha, la huella digital SHA-256 y el estado del documento, que puede ser Pendiente, Verificado o Alterado.

Por su parte, la entidad EventoTrazabilidad permite registrar las acciones realizadas, junto con la fecha, el resultado y el identificador de transacción (txID) correspondiente.

En cuanto a las relaciones, un usuario puede registrar varios documentos y generar múltiples eventos de trazabilidad. De igual forma, un documento puede tener varios eventos asociados, lo que permite consultar las acciones realizadas sobre él a lo largo del tiempo.

El modelo también permite diferenciar la información que se almacena en MongoDB de la que se registra en la blockchain. En la blockchain se guardan las huellas digitales y los eventos correspondientes, junto con su txID, pero no el contenido de los documentos.

3.3.3 Diagramas de secuencia

Se elaboraron cuatro diagramas de secuencia para representar cómo interactúan los componentes del sistema durante las principales operaciones. Estos diagramas muestran el recorrido de las solicitudes desde el frontend hasta el backend y los servicios correspondientes, incluyendo la comunicación con Hyperledger Fabric y MongoDB, así como las respuestas y los posibles errores.

Los flujos representados corresponden a los siguientes procesos:

- **Autenticación:** Se necesita en el inicio de sesión mediante credenciales y un segundo factor de verificación. También hay situaciones como el bloqueo de una cuenta, el vencimiento del token, el reenvío del código, la aceptación de la política de tratamiento de datos y el cierre de sesión por inactividad.
- **Rol Empleado:** Se muestra el proceso de carga y registro de un documento, desde su validación y la generación de la huella digital hasta su registro en Hyperledger Fabric y el almacenamiento de los metadatos en MongoDB. También se contempla el caso en que la blockchain confirme el registro, pero ocurra un error al guardar la información en MongoDB. Para esta situación se considera un mecanismo de reconciliación relacionado con el ASR-15. Además, está incluido la consulta de documentos y de su trazabilidad.
- **Rol Verificador:** Se representa la consulta de la huella registrada, la comparación con la huella del documento que se quiere verificar y la determinación del resultado. Según la comparación, el sistema establece si la integridad del documento se mantiene o si fue alterado, actualiza su estado y registra el evento de verificación.
- **Rol Administrador:** Se muestra el acceso al panel general, la gestión de usuarios y las consultas administrativas. También hace parte la desactivación lógica de las cuentas, relacionada con el ASR-11, y la validación del token JWT y del rol del usuario en cada solicitud.

Una decisión importante que se refleja en el diagrama de secuencia del empleado es el orden en que se registra la información en los dos sistemas. Primero se realiza el registro en Hyperledger Fabric, y después de su confirmación se guarda la información correspondiente en MongoDB, con el fin de evitar que la base de datos tenga un registro de un documento que no cuente con la evidencia correspondiente en la blockchain, de acuerdo con lo establecido en el RNF-013.

3.3.4 Patrones de diseño y estilos arquitectónicos

El sistema se construyó siguiendo el patrón de arquitectura en capas, en el que cada capa cumple una responsabilidad y solo se comunica con la inmediatamente inferior. Se eligió para cumplir el ASR-19 (modularidad por capas) y el RNF-020, que impide que el frontend acceda directamente a la blockchain. Dentro del backend, esta organización se complementa con cuatro patrones:

Capa de servicios (Service Layer): la lógica de negocio se concentra en servicios como authService, documentService y userService, y no en las rutas. Así, las reglas de registro, verificación y gestión de usuarios se escriben una sola vez.
Repositorio (Repository): el acceso a MongoDB está aislado en la carpeta repositories/ (por ejemplo, userRepository). Los servicios no consultan la base de datos directamente.
Pasarela (Gateway): el módulo fabric/gateway.ts es el único punto de contacto con la red Hyperledger Fabric. Usa Fabric Gateway, que mantiene una sola conexión con el peer, y este reenvía la transacción al orderer.
Cadena de middlewares (Chain of Responsibility): cada solicitud pasa por requireAuth, que valida el JWT, la sesión y el rol, y después por la validación del archivo, antes de llegar al servicio. Si un paso falla, la cadena se corta con un 401 o un 403.

Además, el chaincode en Go aplica el patrón de contrato inteligente: las reglas de negocio se ejecutan dentro de la red. Por ejemplo, RecordVerification cambia el estado del documento y agrega el evento de verificación en una sola transacción.

Estilos arquitectónicos

- Despliegue. El sistema se despliega como contenedores con Docker Compose en un único host. El backend se despliega como un solo contenedor y funciona como un monolito modular. En cambio, la red blockchain está formada por nodos independientes: dos organizaciones con sus peers, un orderer Raft y tres autoridades certificadoras. En total son 11 contenedores. Esta decisión se evidencia en el diagrama de despliegue, en compose.app.yml y en los scripts docker/up.sh y down.sh, que levantan todo con una sola orden. La persistencia es híbrida: el ledger guarda las huellas y los eventos, y MongoDB guarda usuarios, sesiones y metadatos.

- Estructura del código. El código está organizado en capas y, dentro de ellas, en módulos funcionales: identidad, documentos y ledger, verificación, y administración. El backend sigue el flujo routes/ -> middlewares/ -> services/ -> repositories/, más fabric/gateway.ts. El frontend se organiza en páginas por rol con rutas protegidas. El chaincode es un proyecto aparte en Go. Esta estructura se ve en las carpetas backend/src/, frontend/src/ y

chaincode/contract/, y en la matriz de trazabilidad HU-RF-código, que relaciona cada historia con su pantalla, endpoint, servicio y función del chaincode.

- Comunicación. La comunicación entre el frontend y el backend es cliente-servidor, síncrona (petición y respuesta), mediante una API REST con mensajes JSON y autenticación por JWT. Entre el backend y Fabric se usa Fabric Gateway, y la transacción solo se da por confirmada cuando queda registrada en un bloque. El backend se comunica con MongoDB mediante su controlador, y con el servicio de correo (Brevo) mediante SMTP para el código de verificación. El orden de escritura es una regla de comunicación importante: primero se registra en el ledger y, después de su confirmación, en MongoDB (RNF-013).

3.4 Arquitectura de infraestructura

En este aspecto se definió cómo están desplegados los componentes del sistema y qué infraestructura se necesita para que puedan funcionar de manera conjunta. Para documentarlo, se elaboró un diagrama de despliegue y una matriz de infraestructura, tomando como referencia la configuración del repositorio, los archivos de Docker Compose y la test-network de Hyperledger Fabric 2.5.16.

Además, se revisó la configuración con los componentes en ejecución, con el fin de comprobar que el diseño correspondiera con la infraestructura utilizada en el proyecto.

3.4.1 Topología de despliegue

Se definió una topología en la que los componentes del sistema se ejecutan en contenedores Docker y se coordinan mediante Docker Compose sobre una misma máquina, de acuerdo con el RNF-021.

Para documentar esta infraestructura, se elaboró una matriz que identifica 13 componentes, clasificados desde INF-01 hasta INF-13.

Tabla 7. Topología despliegue

Capa	Componente	Tecnología	Puerto host
Presentación	Frontend	React 19 + Vite, servido por nginx	127.0.0.1:5173
Aplicación	Backend API	Node.js 20 + Express + Fabric Gateway SDK	127.0.0.1:8080
Persistencia	MongoDB 7.0	Base de datos off-chain (metadatos, usuarios, OTP)	127.0.0.1:27017
Blockchain	Orderer + 2 Peers (Org1, Org2)	Hyperledger Fabric 2.5.16	7050 / 7051 / 9051
Identidad	3 CA (Org1, Org2, Orderer)	Hyperledger Fabric CA 1.5.22	7054 / 8054 / 9054
Lógica	Chaincode contracts	Go 1.22 (5 funciones)	invocado por el peer
Externo	Correo transaccional (2FA)	Brevo SMTP relay	saliente 587

Fuente: Desarrollado por las autoras

3.4.2 Red Hyperledger Fabric

Para la infraestructura de la blockchain se definió una red Hyperledger Fabric conformada por dos organizaciones (Org1MSP y Org2MSP), cada una con un *peer*, y un nodo *orderer* que utiliza el mecanismo de consenso Raft (etcdraft). La red trabaja con un único canal llamado *contratoschannel*, en el que se registran las transacciones relacionadas con los documentos.

Para almacenar la información del estado de la red se utiliza *LevelDB*, ya que las consultas necesarias para el proyecto se pueden realizar mediante claves compuestas y rangos, sin requerir las consultas más complejas que ofrece CouchDB.

También se configuró el *chaincode* contracts en su versión 1.0, cuya definición es aprobada por ambas organizaciones. La política de endoso establecida es MAJORITY, por lo que se requiere la aprobación de la mayoría de las organizaciones participantes. En esta configuración, al estar conformada por dos organizaciones, se necesita el respaldo de ambas para validar las transacciones. El *Fabric Gateway* del backend se encarga de gestionar este proceso y recopilar las firmas necesarias.

Por otra parte, la aplicación se comunica con la red mediante una única conexión desde el backend hacia el *peer* de Org1, utilizando gRPC con TLS (grpcs). Esto permite mantener la comunicación con la blockchain controlada desde el backend y evita que el frontend acceda directamente a la red, de acuerdo con el RNF-020.

3.4.3 Separación de exposición

Otra decisión importante fue definir qué componentes pueden recibir conexiones desde fuera de sus redes internas. Para esto, los puertos del front-end, back-end y MongoDB se publican únicamente en 127.0.0.1 (*localhost*), mientras que los puertos internos de Hyperledger Fabric permanecen dentro de la red *bridge* de Docker.

Esta configuración limita el acceso directo a los servicios y reduce su exposición durante las pruebas del MVP. Además, las identidades y los certificados X.509 utilizados por la red se generan nuevamente en cada ciclo de despliegue y no se incluyen en el repositorio del proyecto. Esto facilita la preparación del entorno de pruebas y evita mantener estos archivos dentro del código versionado.

3.4.4 Trazabilidad de los RNF con los componentes de infraestructura

Para relacionar el diseño de infraestructura con los requerimientos no funcionales, se elaboró una matriz que identifica cada requerimiento, el componente encargado de cumplirlo y la evidencia utilizada para verificarlo.

Por ejemplo, el RNF-019, relacionado con la modularidad, se vincula con la separación de los componentes en contenedores. El RNF-020 se relaciona con la restricción de comunicación entre el front-end y Hyperledger Fabric, verificada mediante una prueba de humo. Por su parte, los RNF-005 y RNF-027, relacionados con la inmutabilidad, se vinculan con el registro de las transacciones en el *ledger* de los dos *peers* y con los eventos que conservan su identificador de transacción (*txID*).

Asimismo, el RNF-013, relacionado con la consistencia de la información, se relaciona con el orden definido para registrar los datos: primero en el *ledger* y después en MongoDB. Esta decisión busca mantener la relación entre la evidencia registrada en la blockchain y la información almacenada en la base de datos.

Finalmente, se definió un procedimiento de despliegue que permite levantar el entorno mediante una única orden. Esto facilita repetir la configuración de la infraestructura sin tener que ejecutar manualmente cada paso y permite verificar la reproducibilidad del despliegue, de

acuerdo con el escenario ASR-20.

3.5 Diseño UX/UI

Se diseñó la interfaz del sistema tomando como referencia el escenario ASR-18, relacionado con la facilidad de uso. El propósito es que los usuarios puedan realizar sus tareas sin necesidad de tener conocimientos sobre blockchain o criptografía.
El diseño se desarrolló bajo la identidad visual de TrazaDoc: Sistema de integridad documental, y se organizó en dos etapas: los wireframes de baja fidelidad y el prototipo de alta fidelidad.

3.5.1 Wireframes de baja fidelidad

Primero, se elaboraron los wireframes de baja fidelidad, que permiten definir la estructura de las pantallas, la ubicación de los elementos y el recorrido que debe seguir el usuario para completar cada tarea. Estos prototipos se realizaron en escala de grises, pues en esta etapa el objetivo era revisar la distribución de los elementos y el funcionamiento de los flujos antes de definir los colores y la identidad visual.
Se incluyeron las pantallas necesarias para el proceso de acceso, como el inicio de sesión, la verificación mediante un código de seis dígitos, la aceptación de la política de tratamiento de datos y el modal de cierre de sesión. También se diseñaron las pantallas correspondientes a los roles de Empleado, Verificador y Administrador.
Estos wireframes permitieron revisar la estructura general de la interfaz con el tutor y la empresa antes de avanzar al diseño más detallado.

3.5.2 Prototipo de alta fidelidad

Después de definir la estructura de las pantallas, se elaboró el prototipo de alta fidelidad en Figma, incorporando los colores, la tipografía y los elementos visuales de TrazaDoc.
Una de las decisiones de diseño fue utilizar colores diferentes según el rol, con el fin de ayudar a los usuarios a reconocer el contexto en el que se encuentran. Para esto, se utilizaron tonos azul oscuro en las interfaces del Empleado y el Administrador, y verde azulado para el Verificador.
Las pantallas diseñadas se organizaron de acuerdo con las funcionalidades definidas en los requerimientos:

- **Acceso:** Incluye el inicio de sesión, la verificación mediante el segundo factor de autenticación (2FA), con el rol del usuario visible y la opción de reenviar el código, el cambio de contraseña y el modal de cierre de sesión.
- **Empleado:** Permite cargar documentos PDF mediante una zona de arrastre, con validaciones de formato y tamaño, y el botón para cargar el archivo y generar su huella SHA-256. También incluye la consulta de los documentos registrados, sus estados (Verificado, Pendiente o Alterado) y sus huellas digitales, además de una sección de trazabilidad que muestra las acciones realizadas y sus respectivos identificadores de transacción.
- **Verificador:** Incluye la pantalla para comprobar la integridad de un documento, mediante la selección de un documento registrado o el ingreso manual de su huella, junto con la carga del archivo que se quiere comparar. También permite consultar todos los documentos y revisar la trazabilidad global mediante una tabla con información del documento, la acción realizada, el usuario, el *txID*, la fecha y el

resultado.

- **Administrador:** Cuenta con un panel general que presenta el total de documentos, los verificados, los pendientes y los alterados, junto con la actividad reciente. Además, incluye las pantallas para gestionar usuarios, consultar los documentos registrados y revisar el historial de trazabilidad.

En general, el diseño busca que las funciones principales sean fáciles de identificar y utilizar. Por esta razón, se presentan estados como Verificado y Alterado para comunicar el resultado, mientras que la huella digital se muestra como un dato de respaldo. De esta manera, el usuario puede utilizar el sistema sin tener que comprender los detalles técnicos de Hyperledger Fabric o del proceso criptográfico.

3.6 Decisiones de diseño y diferencias documentadas

Durante la Fase 2 también se documentaron las decisiones tomadas durante el diseño y las diferencias identificadas entre lo planteado inicialmente en el anteproyecto y lo definido para el MVP.

Este registro permite mostrar cómo evolucionó la propuesta a medida que se fueron concretando los requerimientos y las características técnicas del sistema, mostrando un proceso en el que se revisaron las decisiones iniciales y se ajustaron de acuerdo con las necesidades y condiciones actuales del proyecto.

Para organizar esta información, se incluyó una hoja de observaciones en la matriz de infraestructura, en la que se registraron siete puntos relacionados con las decisiones y diferencias identificadas.

Tabla 8. Observaciones de la matriz

ID	Tema	Diferencia documentada	Impacto
OBS-01	Política de endoso	Se planeó OR (basta una organización); se implementó MAJORITY (firman ambas)	Funciona; decidir si se corrige la documentación o se cambia la política
OBS-02	Base de estado	Se mencionó CouchDB; se implementó LevelDB	No se usan consultas enriquecidas; anotar el cambio
OBS-03	Topología	El benchmarking suponía 1 org / 1 peer; se implementó 2 orgs / 2 peers / 3 CA	Actualizar el cuadro o explicar la evolución
OBS-05	Entorno	Se planeó Docker local + VM de Azure; se trabajó solo en Docker local	Despliegue en Azure pendiente (Fase 4)
OBS-07	Correo	Se planeó correo corporativo; se usó Brevo con remitente Gmail verificado	Aceptable para el MVP, por indicación del tutor

Fuente: Desarrollado por las autoras

La diferencia más significativa se encuentra en la topología de la red (OBS-03). En el anteproyecto se había planteado una red mínima con una sola organización, pero en el diseño final se utilizó la *test-network* de Hyperledger Fabric, que cuenta con dos organizaciones. Esto permite que la información se replique en dos peers y que las transacciones requieran el endoso de ambas organizaciones, fortaleciendo la descentralización de la solución.

Ninguna de las observaciones registradas impide el funcionamiento del MVP, todas quedaron documentadas junto con una acción sugerida, lo que permite revisar las decisiones de diseño y realizar los ajustes necesarios sin afectar el desarrollo del sistema.

3.7 Resultados frente a los indicadores

Al finalizar la Fase 2, se obtuvieron los artefactos de diseño correspondientes a los cuatro dominios definidos, los cuales sirven como evidencia para evaluar los dos indicadores planteados.

Tabla 9. Resultados KPIs O2

Indicador	Meta	Resultado obtenido	Estado
KPI-2.1 — Artefactos de diseño aprobados por el tutor	4 de 4 dominios antes de S7	4 de 4 dominios con artefactos completos: casos de uso + ASR + utility tree; componentes + MER + 4 secuencias; despliegue + matriz de infraestructura; wireframes + prototipo de alta fidelidad	Cumplido (4 de 4 dominios aprobados)
KPI-2.2 — Observaciones críticas en revisión de diseño	≤ 10 % (máx. 2 de ~20 componentes)	Autoevaluación previa: 7 observaciones registradas, 0 críticas (ninguna bloquea el MVP, que está en ejecución)	Cumplido (0 observaciones críticas)

Fuente: Desarrollado por las autoras

Los cuatro dominios cuentan con sus respectivos artefactos de diseño, relacionados con los requerimientos del SRS y los escenarios de calidad definidos en la matriz ASR. Además, se aprobaron los cuatro dominios antes del fin de la semana 7, por lo que se cumple el KPI-2.1, correspondiente a la aprobación de los 4 de 4 dominios establecidos.

En cuanto al KPI-2.2, durante la revisión no se registraron observaciones críticas. Aunque se documentaron siete observaciones, estas corresponden a decisiones de diseño y ajustes menores que no impiden el funcionamiento del MVP. Por esta razón, el indicador se mantiene dentro del límite establecido del 10 % de observaciones críticas.

Finalmente, es importante mencionar que durante el diseño la arquitectura tuvo algunos cambios frente a lo planteado inicialmente en el anteproyecto, como la implementación de dos organizaciones en lugar de una. Estos cambios no representan un incumplimiento del alcance, sino ajustes realizados durante el proceso de diseño, que quedaron documentados para mantener la trazabilidad de las decisiones tomadas.

4. DESARROLLO DEL TERCER OBJETIVO

El tercer objetivo específico se desarrolló en la Fase 3 de la metodología, correspondiente al desarrollo e integración del MVP sobre la red Hyperledger Fabric diseñada en el segundo objetivo. En esta etapa, los artefactos de arquitectura, los diagramas UML y el prototipo de alta fidelidad se convirtieron en un sistema funcional de punta a punta, construido en cinco sprints bajo el modelo de Prototipado Evolutivo descrito en el diseño metodológico.

4.1 Planteamiento del objetivo y correspondencia metodológica

El propósito de este objetivo es construir el MVP que materializa el diseño de la Fase 2, integrando el frontend, el backend y la red blockchain, de forma que un Empleado pueda registrar un contrato laboral, un Verificador pueda comprobar su integridad y un Administrador pueda consultar la trazabilidad completa de cada documento mediante las huellas almacenadas en el ledger.

El desarrollo se organizó en cinco sprints que construyeron los seis módulos definidos en la arquitectura de software: infraestructura (Sprint 0), identidad (Sprint 1), documentos y ledger (Sprint 2), verificación (Sprint 3) y administración (Sprint 4). En cada sprint se siguió el mismo procedimiento:

- construcción del módulo en el backend y en el chaincode
- integración con el frontend
- verificación contra el sistema completo desplegado en Docker
- cierre con los criterios de la Definición de Terminado (DoD) del backlog.

El cumplimiento de este objetivo se evaluó con dos indicadores:

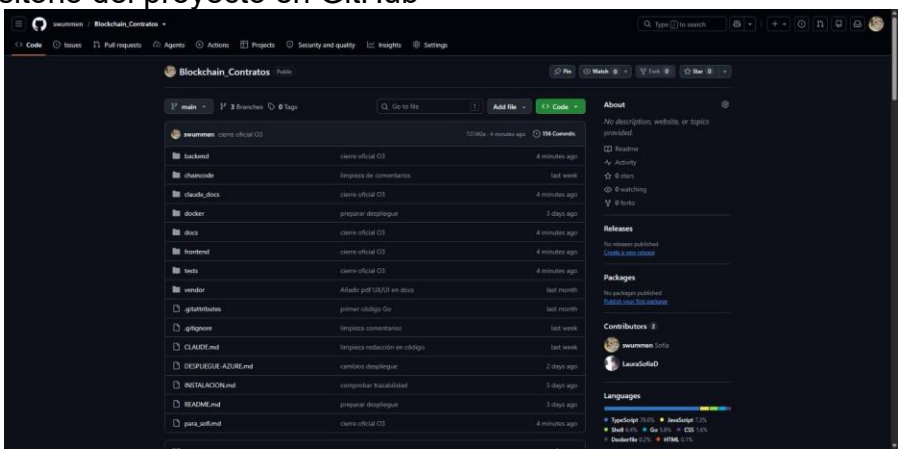
- KPI-3.1, latencia del endpoint de verificación: el percentil 95 debe ser menor o igual a 3.000 ms en 50 peticiones consecutivas, medido en Docker local.
- KPI-3.2, historias de usuario aprobadas por sprint: al menos el 80 % de las historias comprometidas en cada sprint deben quedar aprobadas al cierre.

4.2 Código fuente del MVP

El código quedó organizado en un único repositorio de GitHub, con un directorio por cada capa de la arquitectura definida en el segundo objetivo:

- frontend/, construido en React
- backend/, construido en Node.js con Express
- chaincode/, construido en Go bajo el modelo de contratos inteligentes de Hyperledger Fabric
- docker/, con los scripts de infraestructura y despliegue.

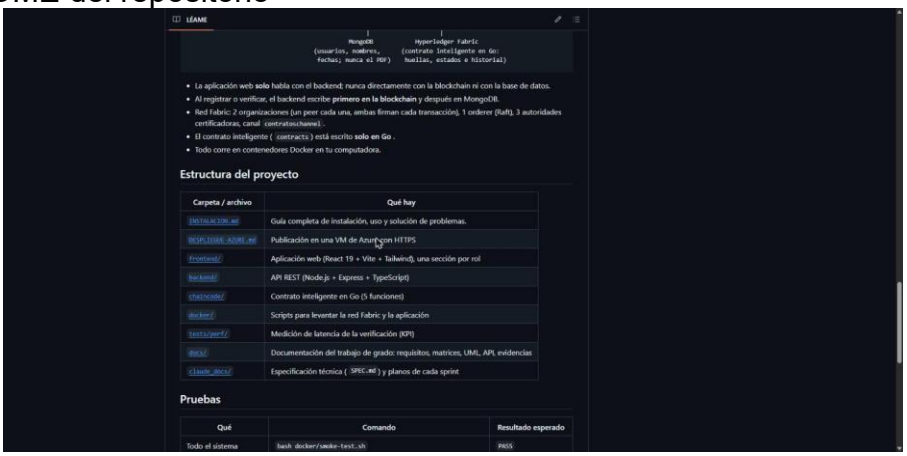
Figura 3. Repositorio del proyecto en GitHub



Fuente: Hecho por las autoras.

El historial de commits refleja el desarrollo incremental de los cinco sprints. El chaincode se desarrolló únicamente en Go, por ser el lenguaje con mejor soporte y documentación en Hyperledger Fabric; esta decisión quedó explícita en el README del repositorio.

Figura 4. README del repositorio

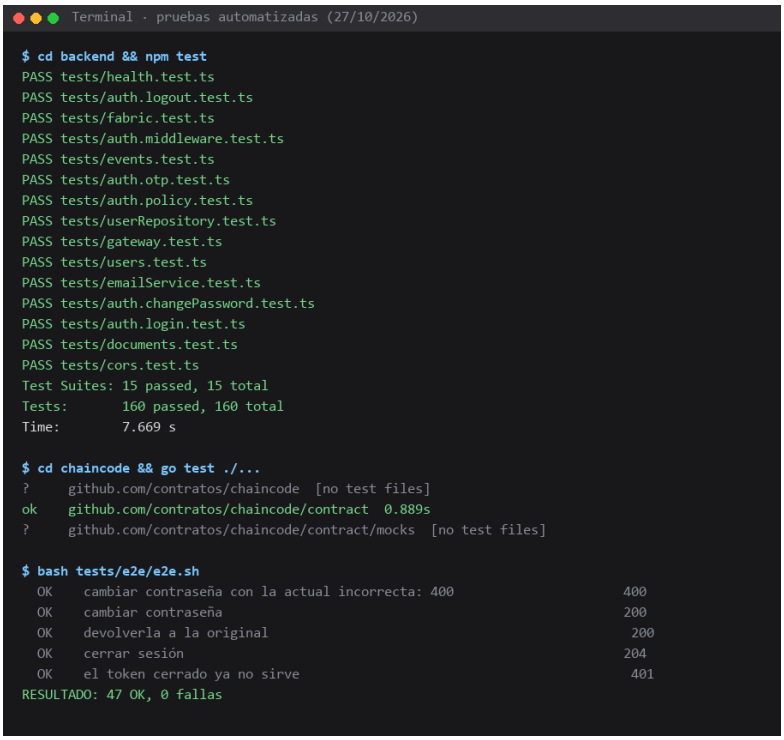


Fuente: Hecho por las autoras.

Al cierre del objetivo, los 55 requerimientos funcionales y las 24 historias de usuario del backlog quedaron implementados, incluidos los dos requerimientos incorporados en la versión 2.0 del backlog:

- El registro de los intentos fallidos de inicio de sesión (RF-054) y la aceptación de la política de tratamiento de datos personales exigida por la Ley 1581 de 2012 (RF-055).
- La calidad del código se respaldó con 160 pruebas automáticas del backend, las pruebas del chaincode y una prueba de punta a punta de 47 pasos sobre la API, todas aprobadas.

Figura 5. Pruebas automatizadas



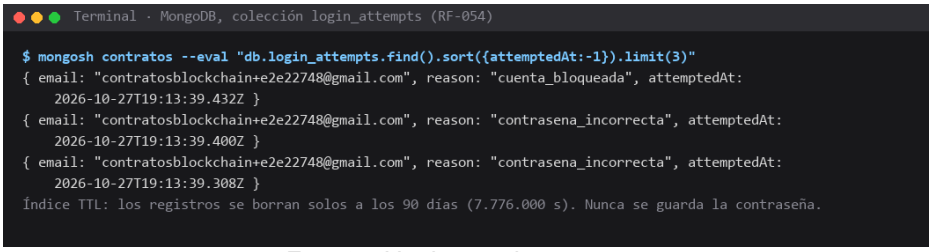
Fuente: Hecho por las autoras.

Figura 6. Aceptación de la política de tratamiento de datos en el primer ingreso



Fuente: Hecho por las autoras.

Figura 7. Registro de intentos fallidos de inicio de sesión en MongoDB



Fuente: Hecho por las autoras.

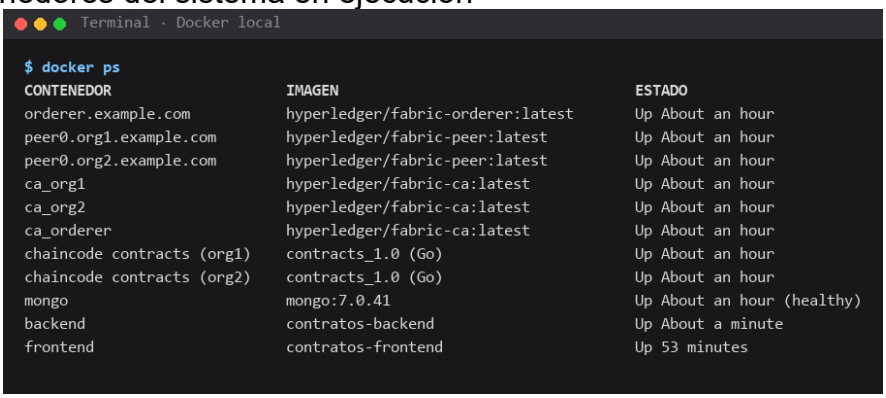
4.3 Script de despliegue de la red

El despliegue se automatizó con un conjunto de scripts que levantan la red desde cero:

- las tres autoridades certificadoras
- el orderer bajo consenso Raft
- los peers de las organizaciones Org1MSP y Org2MSP
- el canal contratoschannel
- el empaquetado, la instalación, la aprobación y la confirmación del chaincode contracts.

Un único comando (docker/up.sh) ejecuta toda la secuencia, levanta MongoDB, el backend y el frontend, y crea los usuarios de prueba de cada rol.

Figura 8. Contenedores del sistema en ejecución



Fuente: Hecho por las autoras.

El procedimiento completo quedó documentado en la guía de instalación del repositorio, de modo que el despliegue pueda reproducirse en un equipo distinto al de desarrollo. La guía cubre los programas requeridos, las variables de entorno, el encendido y apagado del sistema y la solución de los problemas más frecuentes. Con el mismo esquema, el sistema se desplegó el 25 de octubre de 2026 en una máquina virtual de Microsoft Azure con HTTPS, que sirve como entorno de validación para el cuarto objetivo.

4.4 API de integración funcional

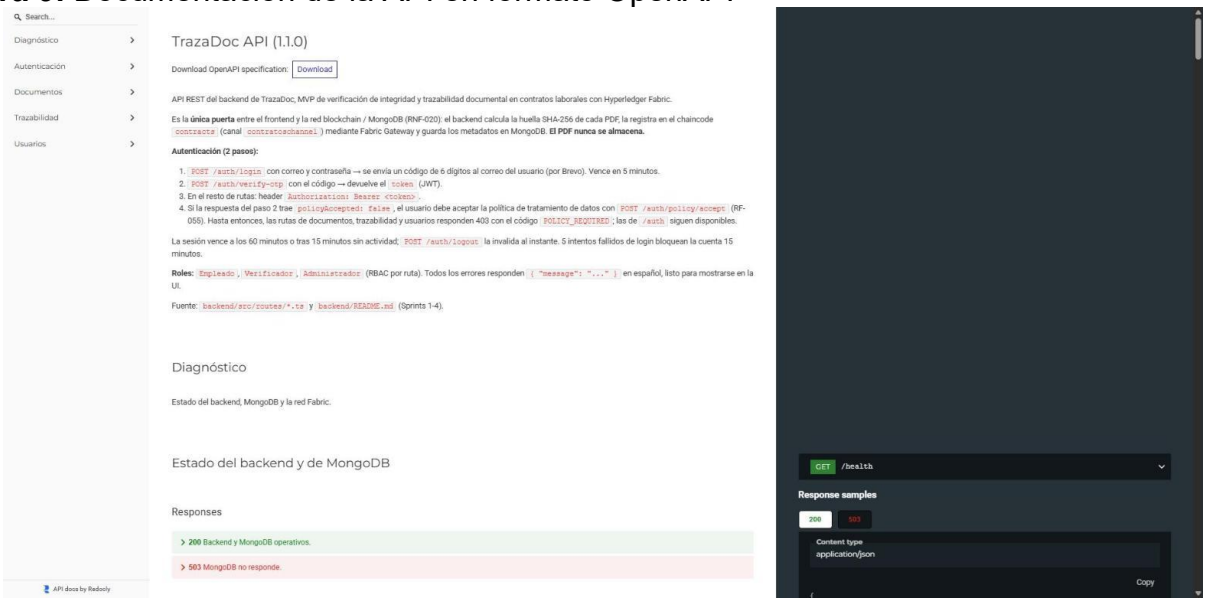
Sobre la infraestructura desplegada, el backend expone una API REST que conecta la interfaz con la red Fabric mediante el Fabric Gateway, como se planteó en el diseño de componentes del segundo objetivo. La API cubre:

- el registro de documentos, con el cálculo de la huella SHA-256 y su escritura en el chaincode mediante RegisterDocument
- la verificación de integridad, con la comparación de huellas y el registro del resultado mediante RecordVerification

- la consulta de documentos y de su trazabilidad mediante GetDocument, GetDocumentEvents y GetAllEvents
- la autenticación con doble factor, la aceptación de la política de datos y la gestión de usuarios con control de acceso por rol.

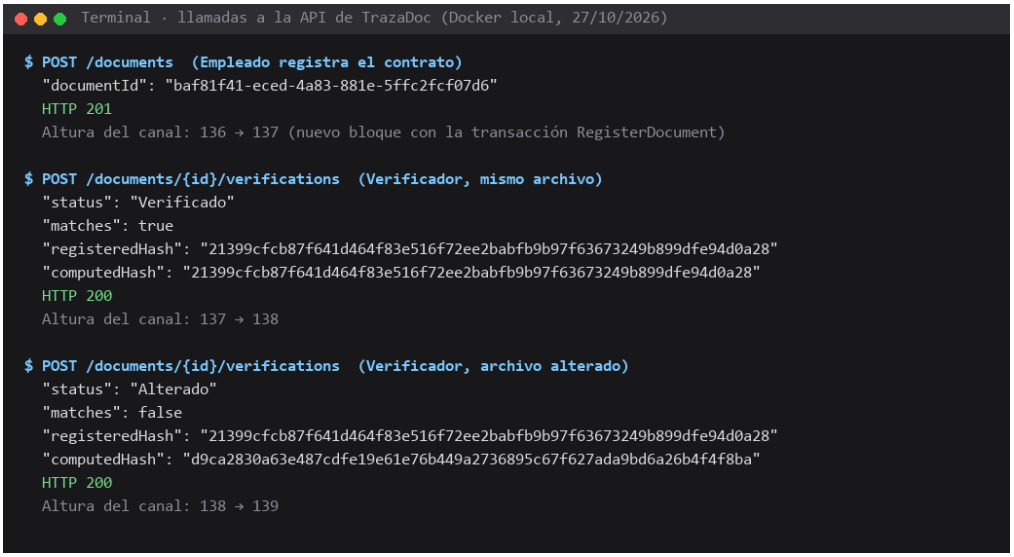
La API quedó documentada en formato OpenAPI, validada y contrastada con las rutas reales del backend, con una guía para importarla en Postman y ejecutar cada operación contra el sistema desplegado.

Figura 9. Documentación de la API en formato OpenAPI



Fuente: Hecho por las autoras.

Figura 10. Llamadas reales a la API



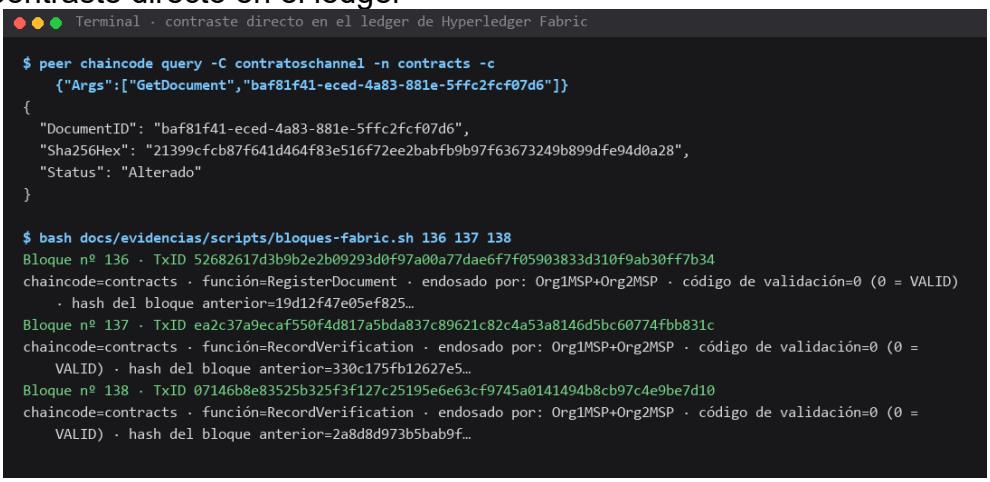
Fuente: Hecho por las autoras.

4.5 Evidencia de integración con Hyperledger Fabric

Para comprobar la integración del backend con la red Fabric de extremo a extremo, se ejecutó

y documentó un caso real de registro, consulta y dos verificaciones de un contrato sobre la red desplegada en Docker. Los identificadores de transacción (txID) de cada operación se localizaron en los bloques 9, 10 y 11 del ledger. Así se confirmó que cada transacción quedó endosada por ambas organizaciones, conforme a la política MAJORITY definida en el segundo objetivo.

Figura 11. Contraste directo en el ledger



Fuente: Hecho por las autoras.

El procedimiento quedó acompañado de scripts reproducibles, para que la evidencia pueda generarse de nuevo sobre cualquier despliegue del sistema.

4.6 Prototipado evolutivo y control de calidad por sprint

Cada sprint tuvo sus historias de usuario comprometidas, sus puntos de historia y su registro de lecciones aprendidas. El cierre de cada sprint se evaluó con los cinco criterios de la Definición de Terminado:

Tabla 10. Aplicación de la Definición de Terminado por sprint

Sprint	Historias	1. Revisión por pares	2. Pruebas PASS	3. Requerimientos actualizados	4. Docker Compose	5. Aprobación de la dirección
0. Infraestructura	HU-021	Parcial	Cumple	Cumple	Cumple	Cumple
1. Identidad	HU-001 a HU-004	Parcial	Cumple	Cumple	Cumple	Cumple
2. Documentos y ledger	HU-005 a HU-007	Parcial	Cumple	Cumple	Cumple	Cumple
3. Verificación	HU-010 a HU-014	Parcial	Cumple	Cumple	Cumple	Cumple
4. Administración	HU-008, HU-009, HU-015 a HU-020	Parcial	Cumple	Cumple	Cumple	Cumple

Fuente: Hecho por las autoras.

Los criterios 2 a 5 se cumplieron en los cinco sprints: las pruebas quedaron en PASS, el SRS se actualizó con cada nota de cambio, el sistema se verificó funcionando en Docker tras cada cierre y la dirección del trabajo de grado aprobó las historias de cada sprint. El criterio 1 se cumplió parcialmente, porque solo una parte de los cambios se integró mediante pull requests revisados; esto quedó registrado como una mejora del proceso en la auditoría de calidad del

cuarto objetivo.

4.7 Registro de cambios respecto al diseño

Durante la construcción del MVP se identificaron quince diferencias entre lo diseñado en el segundo objetivo y lo implementado, cada una documentada con su motivo, los requerimientos o diagramas que afecta y su evidencia, ocho de arquitectura e infraestructura y siete de funcionalidad:

- Arquitectura e infraestructura: destacan el uso de LevelDB en lugar de CouchDB como base de estado del peer, justificado por el tipo de consultas que requiere el sistema; la confirmación de la política de endoso MAJORITY; y la publicación del sistema a través de un único punto de entrada con HTTPS para el despliegue en Azure.
- Funcionalidad: destaca el paso de un identificador ingresado manualmente a una selección en dos pasos, primero persona responsable y luego contrato, con el fin de resolver un problema de usabilidad detectado en las pruebas del equipo.

Ninguno de los quince cambios elimina un requerimiento del SRS. Los dos requerimientos que estaban pendientes, la aceptación de la política de tratamiento de datos y el registro de intentos fallidos de inicio de sesión, quedaron implementados y probados al cierre de este objetivo.

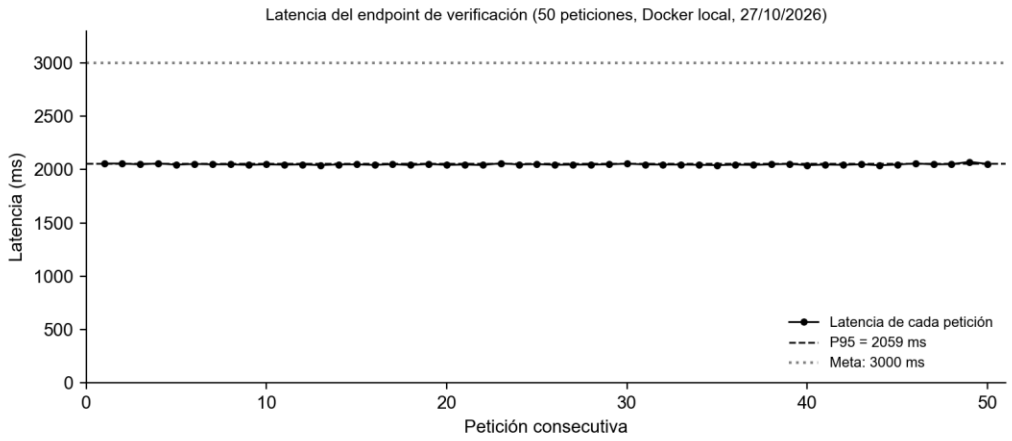
Tabla 11. Resumen de los cambios respecto al diseño

ID	Tema	Cambio implementado
C-01	Base de estado del peer	LevelDB en lugar de CouchDB
C-02	Topología de la red	2 organizaciones, 2 peers, 1 orderer (Raft) y 3 CA
C-03	Política de endoso	MAJORITY: firman ambas organizaciones
C-04	Conexión backend y Fabric	Fabric Gateway con una sola conexión al peer
C-05	Funciones del chaincode	Cinco funciones; registro y verificación escriben estado y evento en una transacción
C-06	Correo del código OTP	Proveedor externo (Brevo)
C-07	Entorno de despliegue	Docker local para desarrollo y VM de Azure para validación
C-15	Publicación del sistema	Un solo punto de entrada con HTTPS (proxy /api y Caddy)
C-08	Rol en el inicio de sesión	El rol se toma de la cuenta; no se elige
C-09	Reenvío del código OTP	Sin reenvío; agotados los intentos, se inicia sesión de nuevo
C-10	Selección del documento	Dos pasos: persona responsable y luego contrato
C-11	Visibilidad de la huella	Solo el Verificador ve la huella completa
C-12	Eventos de usuarios	El ledger solo guarda eventos de documentos
C-13	Política de datos	Implementada; el backend exige su aceptación

Fuente: Hecho por las autoras.

4.8 Resultados frente a los indicadores

Figura 12. Latencia del endpoint de verificación en 50 peticiones consecutivas



Fuente: Hecho por las autoras.

KPI-3.1: Un script automatizado ejecuta 50 verificaciones consecutivas contra el endpoint de verificación y calcula el percentil 95 del tiempo de respuesta. Se hicieron dos mediciones sobre Docker local:

- 15 de octubre de 2026 (ensayo): P95 de 2.040 ms.
- 27 de octubre de 2026 (medición oficial), con un contrato de 2,7 MB: P95 de 2.059 ms, con valores entre 2.041 y 2.072 ms y una media de 2.049 ms.

Ambas quedan por debajo del umbral de 3.000 ms, por lo que el indicador se cumple. La estabilidad de los tiempos se explica porque cerca de 2 segundos corresponden a la espera configurada del orderer para cortar cada bloque; el trabajo propio del sistema suma unos 30 ms.

Tabla 12. Resultado del KPIs O3

Medición	Fecha	Documento	P95	Media	Mínimo / máximo	Meta	Estado
Ensayo	15/10/2026	PDF de 0,1 KB	2.040 ms	N/D	2.028 / 2.047 ms	≤ 3.000 ms	Cumple
Oficial	27/10/2026	Contrato PDF de 2,7 MB	2.059 ms	2.049 ms	2.041 / 2.072 ms	≤ 3.000 ms	Cumple

Fuente: Hecho por las autoras

KPI-3.2: El registro de sprints muestra que el 100 % de las historias comprometidas (21 de 21) quedó terminado y aprobado por la dirección del trabajo de grado en cada uno de los cinco sprints, por encima del umbral del 80 %, por lo que el indicador se cumple. Además, las tres historias incorporadas en la versión 2.0 del backlog (HU-022, HU-023 y HU-024) también quedaron implementadas.

En conjunto, estos resultados muestran que el MVP quedó construido e integrado con la red Hyperledger Fabric, como exige el tercer objetivo específico. El registro, la verificación de integridad, la consulta y la trazabilidad de los contratos laborales funcionan de extremo a extremo mediante el almacenamiento de las huellas documentales en el ledger, con evidencia reproducible de la integración, los 55 requerimientos funcionales implementados y los dos indicadores cumplidos.

5. DESARROLLO DEL CUARTO OBJETIVO

5.1 Planteamiento del objetivo y correspondencia metodológica

El cuarto objetivo específico busca evaluar el MVP mediante un Plan de Pruebas de Software que verifique el comportamiento funcional de los módulos, la detección de alteraciones en documentos registrados y el rendimiento operativo del sistema. Corresponde a la Fase 4 de la metodología, Evaluación de integridad y rendimiento. Esa fase define cinco actividades: el diseño del plan de pruebas, las pruebas funcionales de caja negra, las pruebas de integridad por mutación documental, las pruebas de trazabilidad e inmutabilidad y las pruebas de rendimiento.

El objetivo se midió con tres indicadores. El primero es la tasa de detección de alteraciones documentales, con una meta del 100 % sobre 20 documentos alterados. El segundo es el porcentaje de casos funcionales con resultado PASS en la suite final, con una meta de al menos el 90 %. El tercero es el porcentaje de ejecución del plan completado en plazo, con una meta de al menos el 85 % de las actividades. Además, se volvió a medir la latencia de verificación definida en el tercer objetivo, esta vez como prueba de rendimiento del plan. Los entregables de la fase son cuatro: el documento del Plan de Pruebas de Software, el Informe de Resultados de Pruebas y Defectos, la Matriz de Cumplimiento de Requerimientos y el Manual de Usuario y Operación.

5.2 Plan de Pruebas de Software

El Plan de Pruebas de Software (PPS) define qué se prueba, cómo, dónde, quién lo hace, cuándo y con qué criterio se decide si el sistema pasa o no. Así, los resultados del informe final se pueden repetir y comprobar. El plan cubre las cinco épicas del backlog: autenticación y control de acceso, registro y gestión documental, verificación de integridad, administración del sistema e infraestructura. En total suma 62 casos de prueba: 51 funcionales, derivados de los escenarios de aceptación de las historias de usuario, y 11 no funcionales de integridad, trazabilidad y rendimiento.

Tabla 13. Tipos de prueba y casos del PPS

Tipo	Casos	Cómo se ejecuta
Funcional (caja negra)	51 (CP-01 a CP-51), en las 5 épicas	Manual en la interfaz, con captura
Integridad (mutación documental)	6 (INT-01 a INT-06)	INT-01 automatizada con los 20 documentos; INT-02 a INT-06 manuales
Trazabilidad e inmutabilidad	3 (TRZ-01 a TRZ-03)	Manual y automatizada
Rendimiento	2 (REN-01 y REN-02)	Automatizada (tests/perf y tests/integrity)
Total	62	

Fuente: Hecho por las autoras.

Cada caso indica sus precondiciones, los pasos, los datos de entrada, el resultado esperado y el requerimiento que prueba. El plan también define el procedimiento de reporte de defectos, con cuatro niveles de severidad y cuatro de prioridad, los criterios de entrada y salida de cada etapa, las herramientas y los riesgos de la ejecución.

5.3 Entornos y ciclos de prueba

Las pruebas se organizaron en tres ciclos. El Ciclo 1, de ensayo, se ejecutó en el entorno de desarrollo local con Docker entre el 11 y el 24 de octubre de 2026. Su propósito era comprobar que los casos, los datos y los scripts funcionaran antes de la ejecución oficial. El 25 de octubre el sistema se desplegó en una máquina virtual de Microsoft Azure, con HTTPS y un dominio público. Sobre ese despliegue se ejecutó el Ciclo 2, la ejecución oficial, cuyos resultados alimentan el informe. El Ciclo 3 se reservó para volver a probar los casos con defectos corregidos y repetir la regresión sobre la versión nueva.

Los datos de prueba son 20 documentos sintéticos en PDF con datos ficticios:

- 12 contratos laborales y 8 acuerdos de confidencialidad.

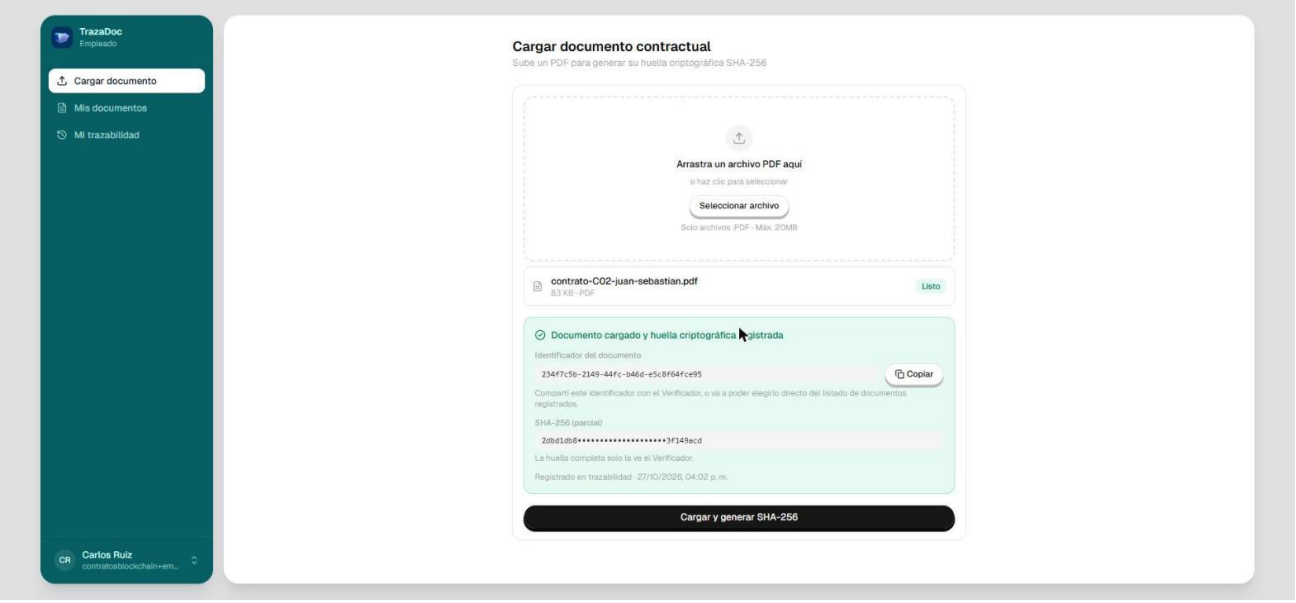
Los contratos se generaron a partir de la plantilla de contrato de trabajo del equipo y los acuerdos, a partir del modelo de acuerdo de confidencialidad de la OEPM, adaptado a la legislación colombiana. Cada documento tiene una copia alterada. También se prepararon archivos de límite: un PDF cercano a 10 MB, uno de 25 MB, uno de 30 MB y un archivo DOCX.

5.4 Pruebas funcionales de caja negra

Los casos funcionales se ejecutaron sobre la interfaz desplegada, con una cuenta de cada rol, comparando lo que hace el sistema con el resultado esperado de cada escenario de aceptación. En el rol Empleado se comprobó lo siguiente:

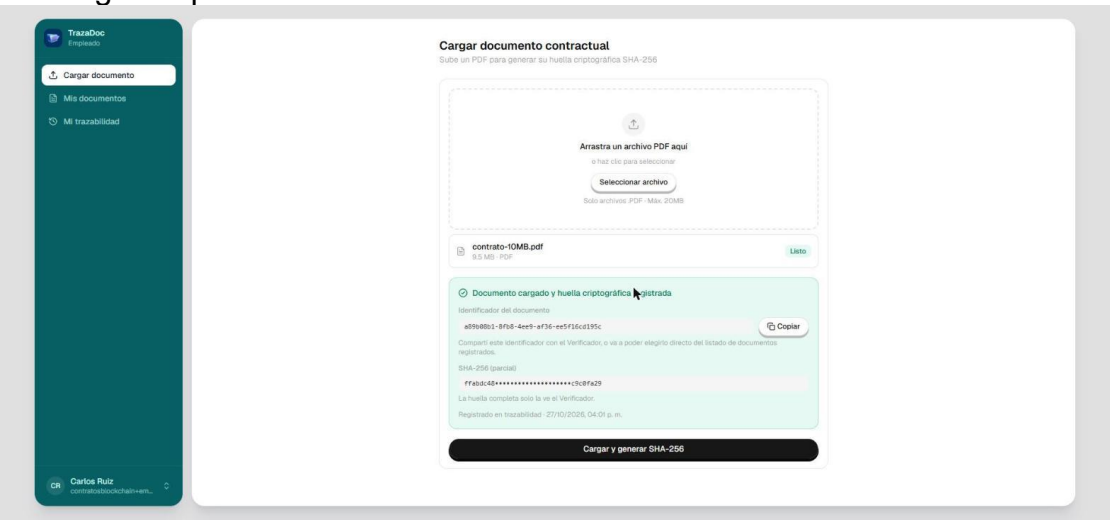
- la carga de un contrato en PDF genera automáticamente sus metadatos y su huella SHA-256 y lo registra en la blockchain con un identificador de transacción (Figura 13);
- el sistema acepta un PDF de 9,5 MB y rechaza tanto un archivo DOCX como un PDF de 25 MB

Figura 13. Carga de un contrato en PDF



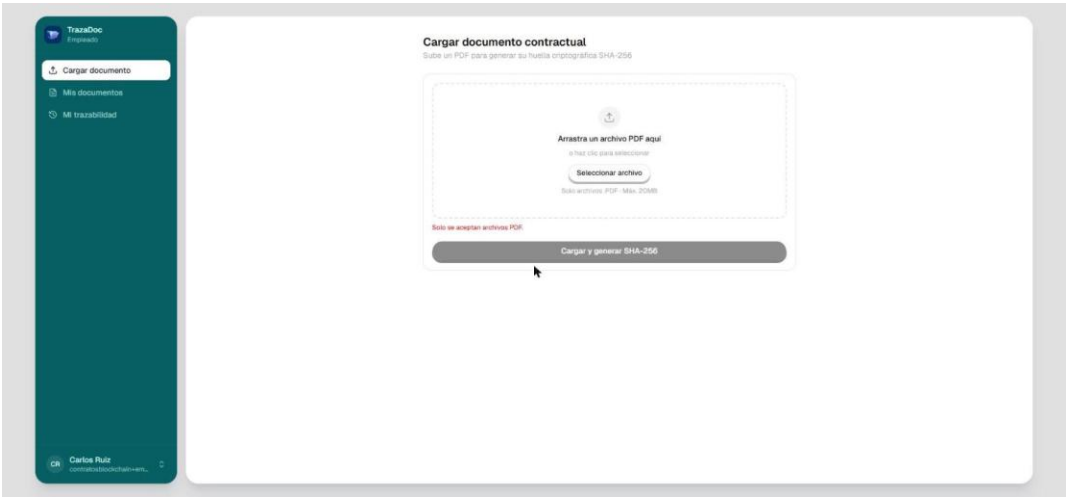
Fuente: Hecho por las autoras.

Figura 14. Carga aceptada de un PDF



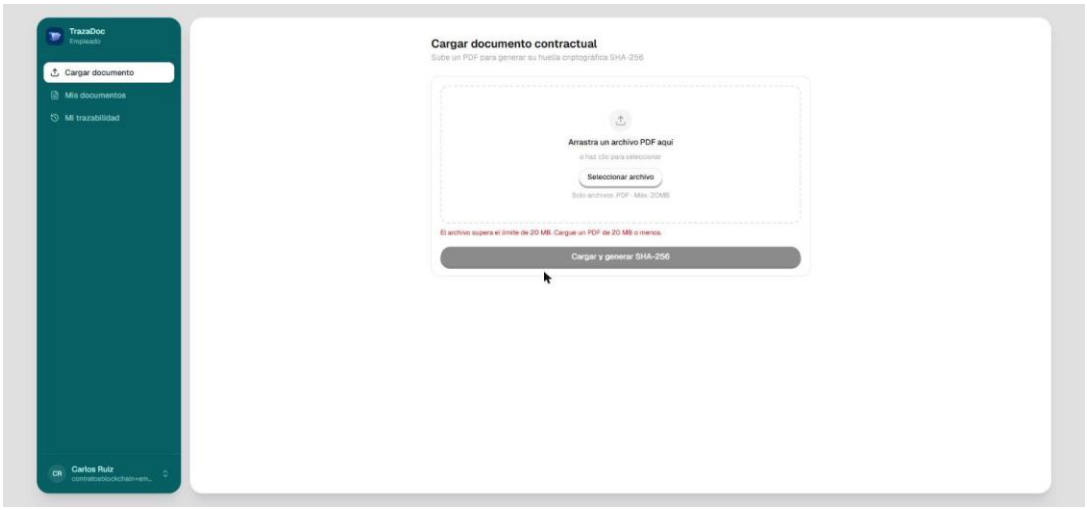
Fuente: Hecho por las autoras.

Figura 15. Rechazo de un archivo DOCX



Fuente: Hecho por las autoras.

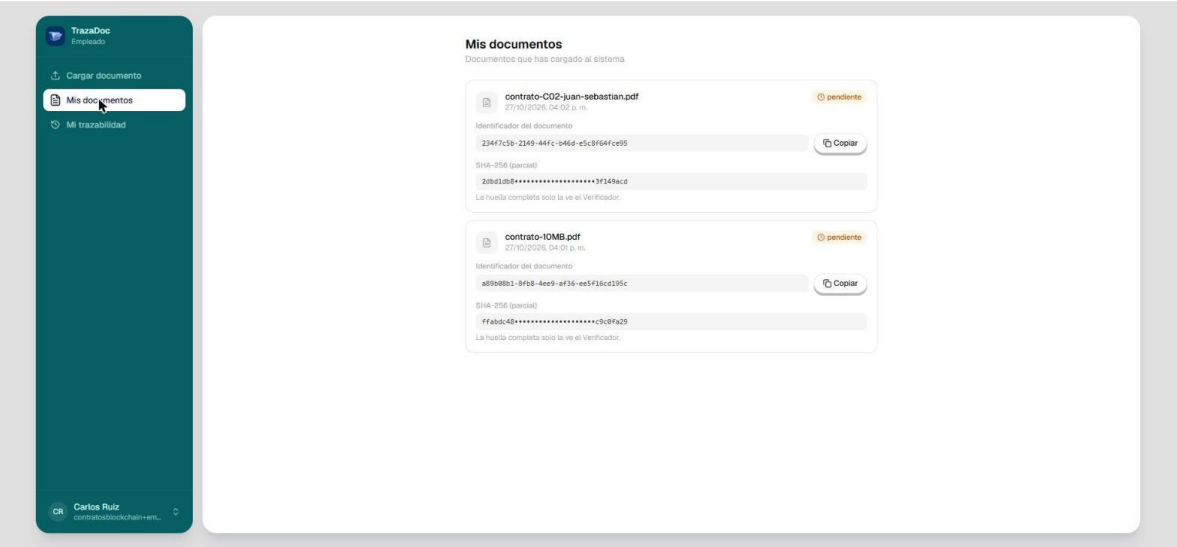
Figura 16. Rechazo de un PDF de 25 MB



Fuente: Hecho por las autoras.

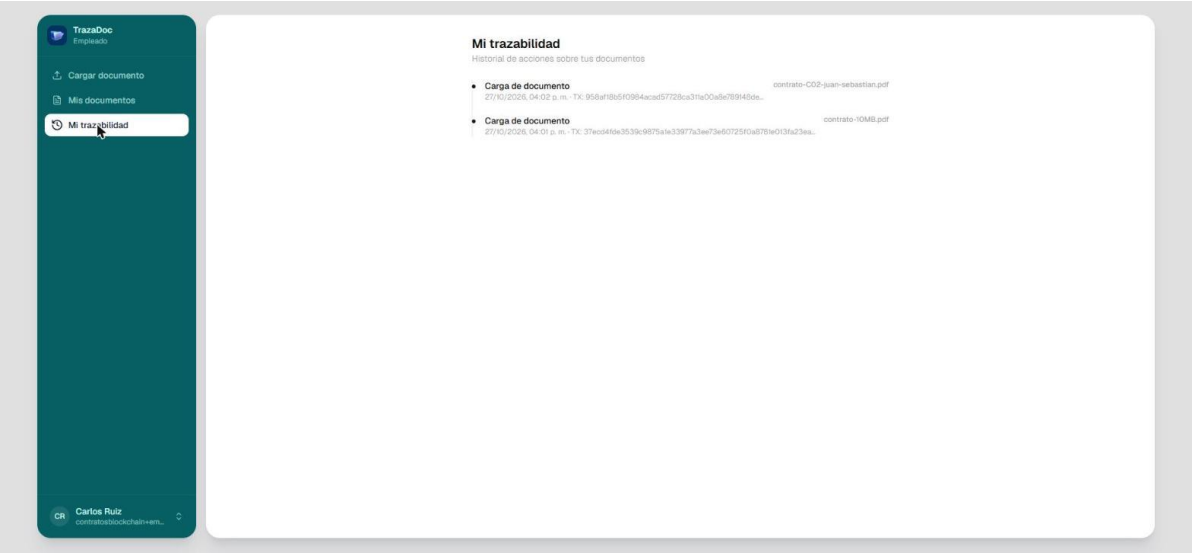
- la vista "Mis documentos" muestra únicamente los contratos del propio usuario, con estado inicial Pendiente, y su trazabilidad presenta cada acción con su transacción

Figura 17. Sección mis documentos



Fuente: Hecho por las autoras.

Figura 18. Sección mi trazabilidad



Fuente: Hecho por las autoras.

En el acceso al sistema se verificó lo siguiente:

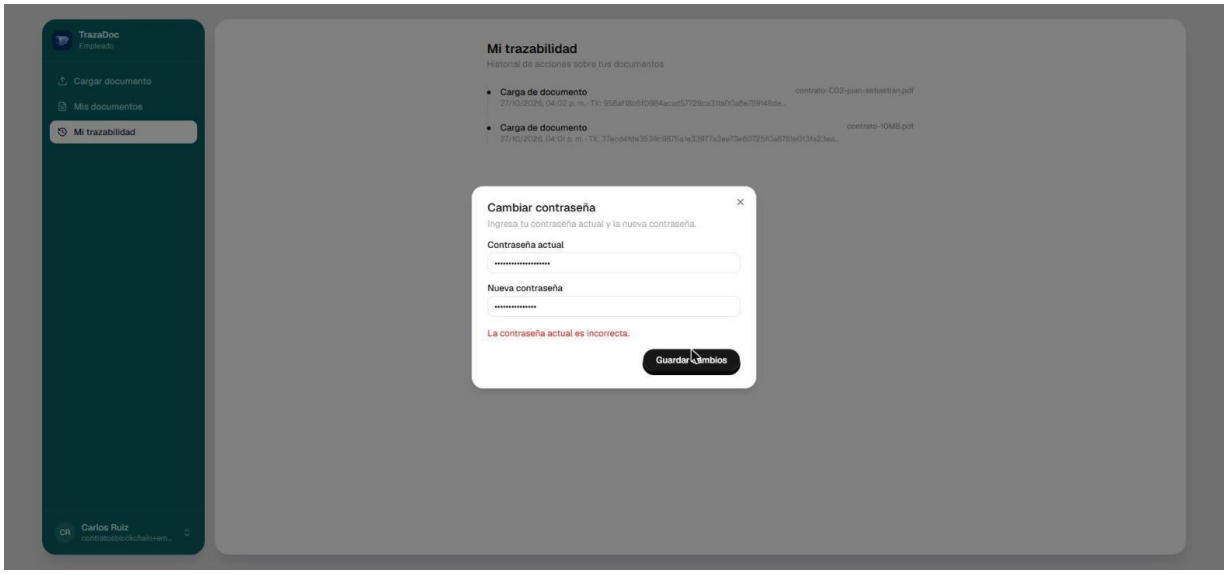
- la política de tratamiento de datos se presenta en el primer ingreso, antes de cualquier panel
- un usuario no puede abrir el panel de otro rol
- el cambio de contraseña rechaza una contraseña actual incorrecta
- después de cerrar sesión no es posible volver al panel con el botón Atrás del navegador

Figura 19. Tratamiento de datos



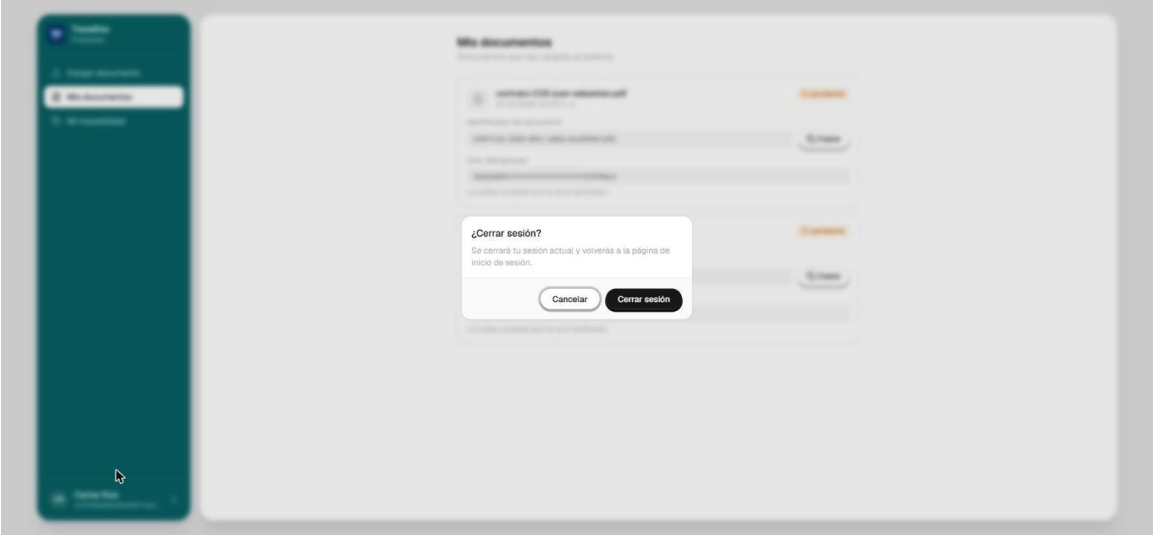
Fuente: Hecho por las autoras

Figura 20. Cambio de contraseña



Fuente: Hecho por las autoras

Figura 21. Confirmación antes de cerrar la sesión

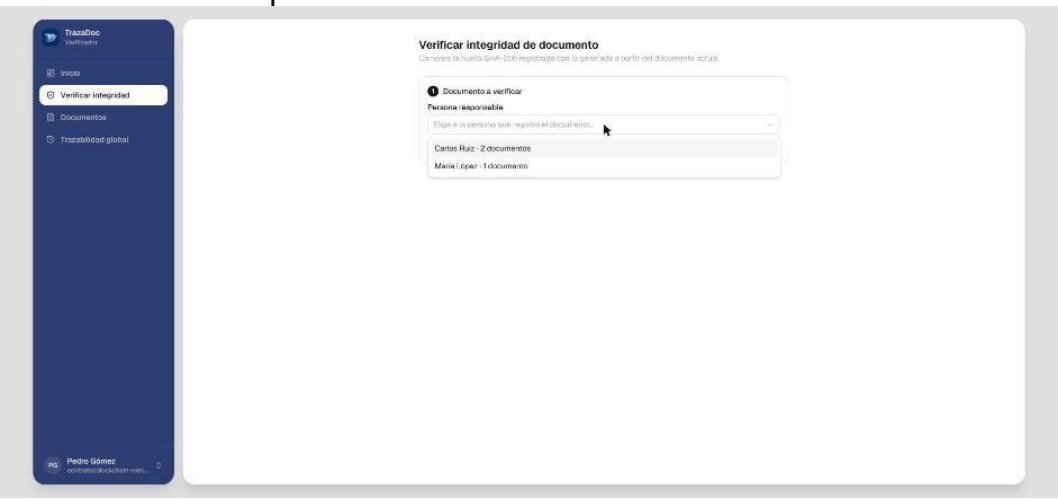


Fuente: Hecho por las autoras

En el rol Verificador se comprobó lo siguiente:

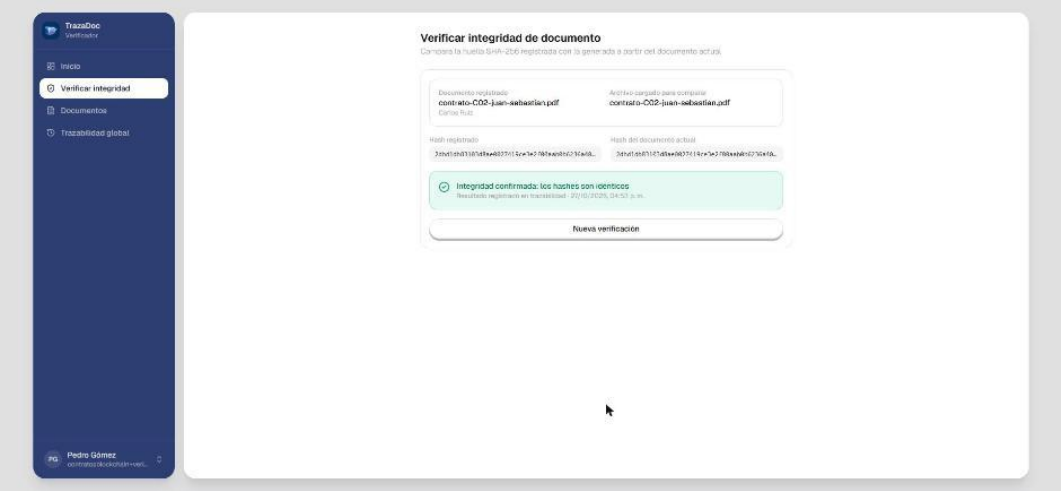
- la selección del documento se hace en dos pasos, primero la persona y luego su contrato, y se muestra la huella completa registrada
- la comparación con el mismo archivo confirma la integridad
- un identificador inexistente y un archivo de 30 MB se rechazan con su mensaje
- el listado general muestra todos los documentos con su estado.

Figura 22. Selección en dos pasos



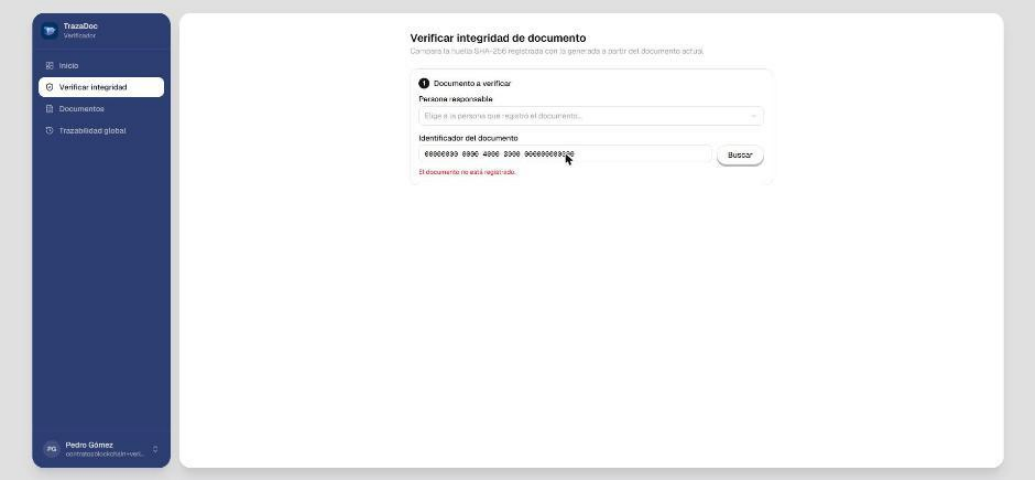
Fuente: Hecho por las autoras

Figura 23. Comparación con el mismo archivo



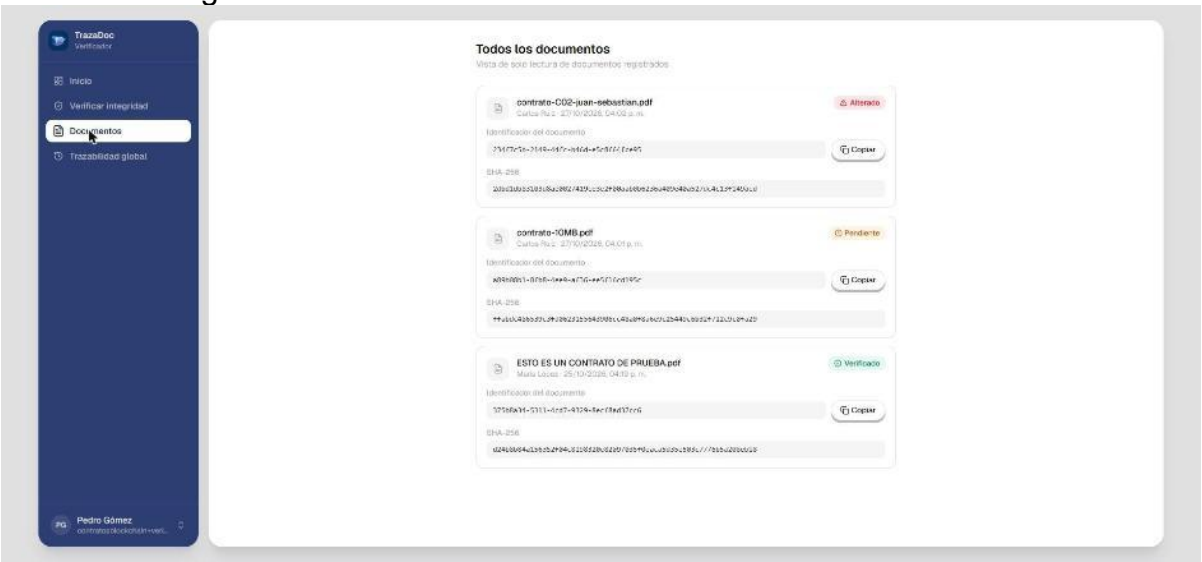
Fuente: Hecho por las autoras

Figura 24. Identificador inexistente



Fuente: Hecho por las autoras

Figura 25. Listado general de documentos con su identificador

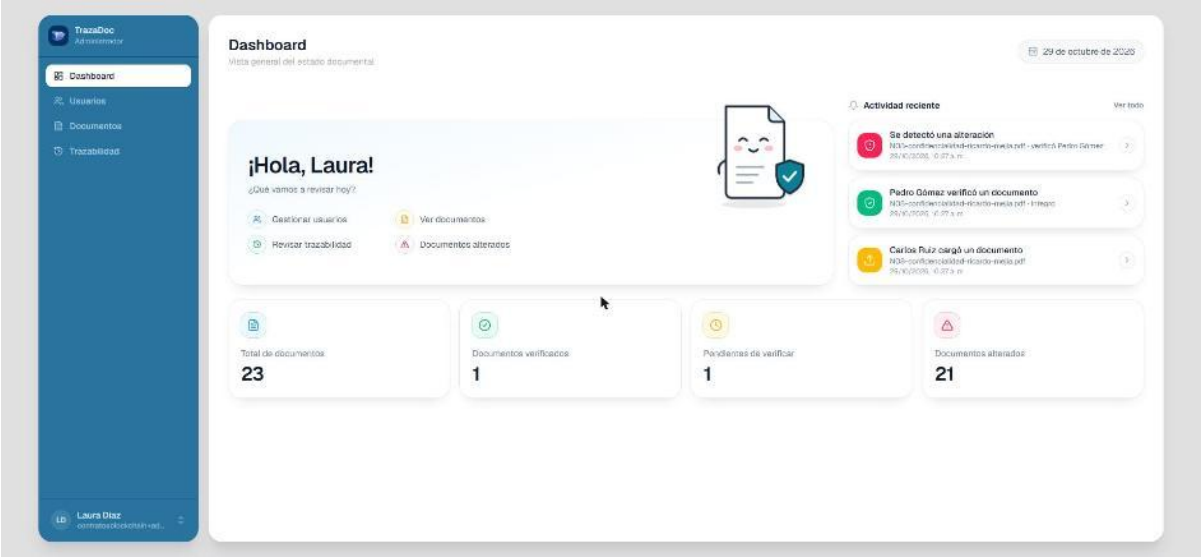


Fuente: Hecho por las autoras

En el rol Administrador se comprobó lo siguiente:

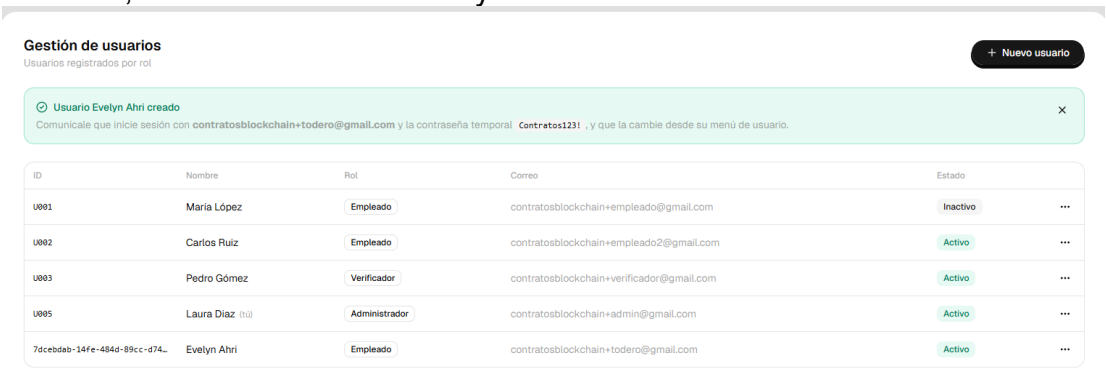
- los totales del panel suman lo mismo que el total de documentos
- se puede crear un usuario, cambiarle el rol y desactivarlo, y el usuario desactivado conserva su historial pero no puede iniciar sesión
- la huella completa solo es visible para el Verificador
- la trazabilidad administrativa muestra los mismos registros que la del Verificador

Figura 26. Panel del Administrador



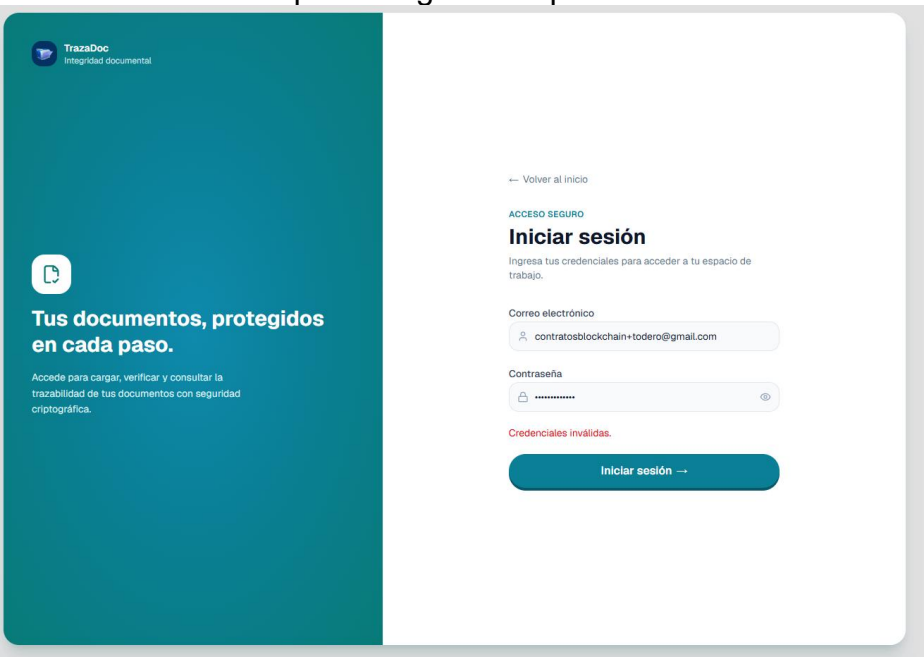
Fuente: Hecho por las autoras

Figura 27. Roles, creación de un usuario y contraseñas



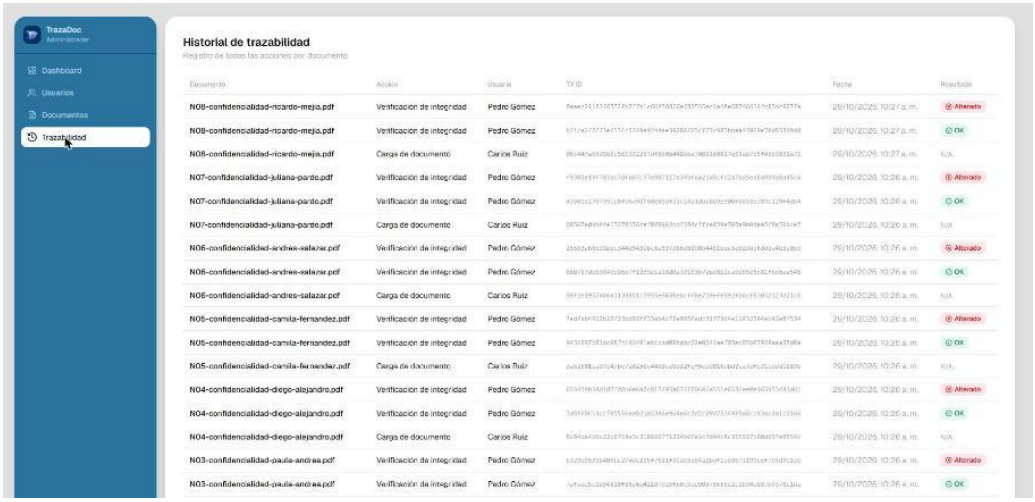
Fuente: Hecho por las autoras

Figura 28. Usuario desactivado no puede ingresar al portal



Fuente: Hecho por las autoras

Figura 29. Trazabilidad administrativa



Fuente: Hecho por las autoras

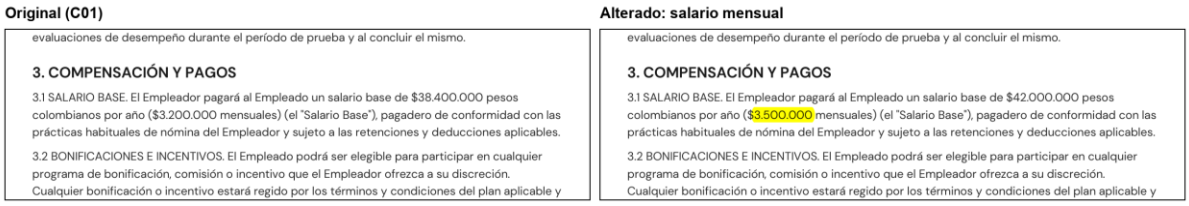
5.5 Pruebas de integridad por mutación documental

La prueba de mutación documental comprueba que el sistema detecte cualquier modificación de un documento después de registrado. Se aplicaron cinco tipos de alteración, cuatro documentos de cada tipo, repartidos entre contratos y acuerdos:

- Texto: el cambio de una condición, como el salario mensual o la vigencia de la confidencialidad.
- Firma: el reemplazo del firmante.
- Metadatos: el cambio del autor y de la fecha de modificación del PDF, sin alterar el contenido visible.
- Un solo carácter: un punto que pasa a ser coma.
- Anexo: una página agregada al final.

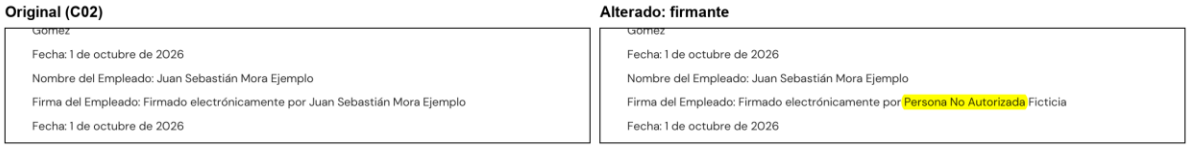
-

Figura 30. Alteración de texto en el contrato C01



Fuente: Hecho por las autoras

Figura 31. Alteración de firma en el contrato C02



Fuente: Hecho por las autoras

Figura 32. Alteración solo de metadatos en el contrato C03

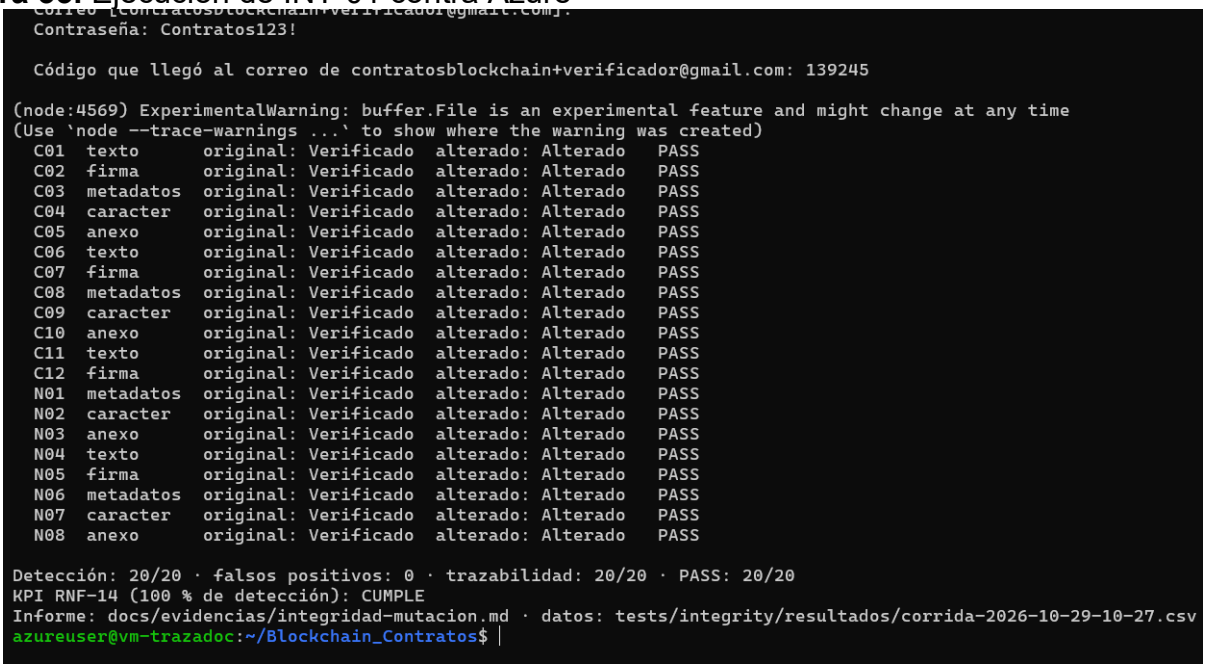
Campo	Original (C03)	Alterado
author		Usuario externo
creator	Microsoft® Word para Microsoft 365	Microsoft® Word para Microsoft 365
creationDate	D:20261027152610-05'00'	D:20261027152610-05'00'
modDate	D:20261027152610-05'00'	D:20261020183000-05'00'

Fuente: Hecho por las autoras

Los casos de metadatos y de un solo carácter son los más exigentes, porque a simple vista el documento alterado es idéntico al original.

La prueba INT-01 se automatizó con un script que usa la API del sistema igual que el navegador. Por cada documento, el Empleado registra el original. Luego, el Verificador lo compara primero con el mismo original, que debe dar Verificado, y después con la copia alterada, que debe dar Alterado. La primera comparación es el control de falsos positivos: sin ella, un sistema que respondiera "Alterado" a todo también obtendría el 100 %. El script comprueba además que las huellas devueltas por la API coincidan con las calculadas al generar los archivos y que el ledger registre las tres acciones de cada documento.

Figura 33. Ejecución de INT-01 contra Azure



Fuente: Hecho por las autoras

Tabla 14. Detección por tipo de alteración

Alteración	Documentos	Detectadas	Originales marcados como alterados
Texto	C01, C06, C11, N04	4 de 4	0
Firma	C02, C07, C12, N05	4 de 4	0
Metadatos	C03, C08, N01, N06	4 de 4	0
Un carácter	C04, C09, N02, N07	4 de 4	0
Anexo	C05, C10, N03, N08	4 de 4	0
Total	20	20 de 20 (100 %)	0

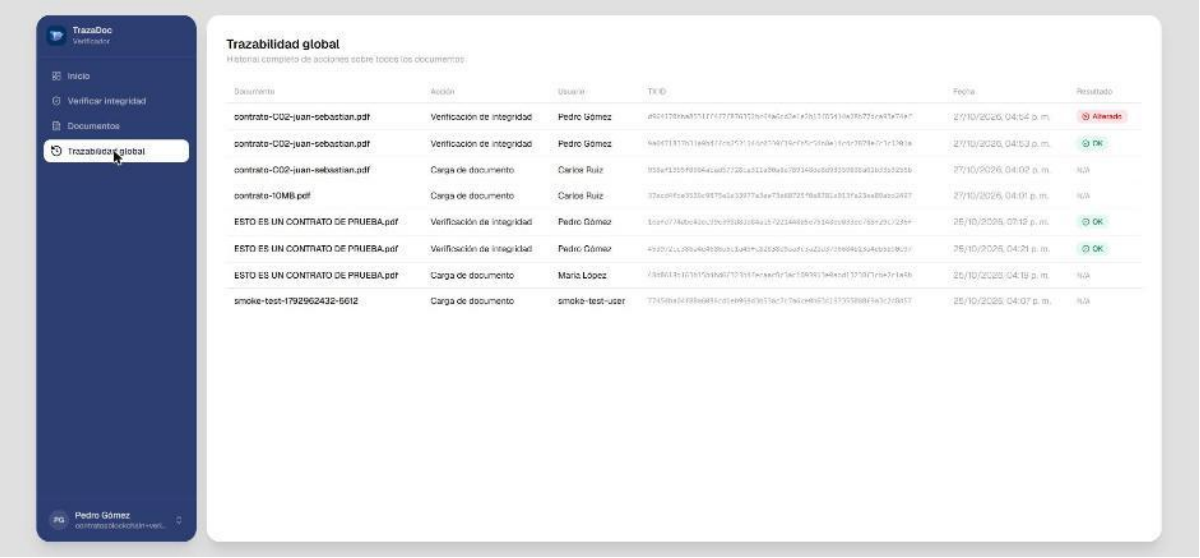
Fuente: Hecho por las autoras

Los casos INT-02 a INT-06 repitieron una alteración de cada tipo directamente en la interfaz del Verificador y todos mostraron "Integridad comprometida", incluidos los cambios de metadatos y de un solo carácter.

5.6 Pruebas de trazabilidad e inmutabilidad

En TRZ-01 se registró un documento y se verificó primero con el original y luego con una copia alterada. La trazabilidad mostró las tres acciones en el orden en que ocurrieron, cada una con su identificador de transacción.

Figura 34. Trazabilidad de C02



Fuente: Hecho por las autoras

En TRZ-02 se verificaron de nuevo documentos ya verificados. Cada verificación agregó un evento nuevo sin borrar los anteriores, y la huella registrada no cambió. Esto evidencia la propiedad de no repudio del registro.
En TRZ-03 se comparó, documento por documento, la huella y el estado guardados en la base de datos con los del ledger. Coincidieron en los 50 documentos del entorno de pruebas.

Tabla 15. Consistencia entre el ledger y la base de datos

Comprobación	Documentos	Resultado
Misma huella SHA-256 en MongoDB y en el ledger	50	50 de 50
Mismo estado en MongoDB y en el ledger	50	50 de 50

Fuente: Hecho por las autoras

5.7 Pruebas de rendimiento

Se midieron las dos operaciones críticas del sistema:

- Verificación (REN-01): con 50 peticiones consecutivas, el percentil 95 fue de 2.059 ms, por debajo de la meta de 3.000 ms. Sobre Azure, las 20 verificaciones de la prueba de integridad dieron un percentil 95 de 2.125 ms.
- Escritura del hash en la red (REN-02): se midió durante el registro de los 20 documentos en Azure, con una media de 2.112 ms por registro.

Tabla 16. Latencia de registro y verificación

Operación	Entorno	Muestras	Media	Mínimo / máximo	P95
Verificación (REN-01)	Docker local	50	2.049 ms	2.041 / 2.072 ms	2.059 ms
Registro del hash (REN-02)	Azure	20	2.112 ms	2.094 / 2.185 ms	N/D
Verificación	Azure	20	2.112 ms	2.096 / 2.131 ms	2.125 ms

Fuente: Hecho por las autoras

Los valores se mantuvieron alrededor de los dos segundos en ambos entornos, porque la red ordena las transacciones en bloques que se cortan cada dos segundos. La mayor parte del tiempo correspondió a la espera del bloque y no al procesamiento del sistema. La diferencia entre el entorno local y Azure fue de unos 70 ms.

5.8 Pruebas del despliegue en Azure

Sobre el despliegue se ejecutaron los casos de infraestructura y seguridad desde un equipo externo Se comprobó lo siguiente:

- el sitio usa un certificado HTTPS válido emitido por Let's Encrypt, y las peticiones por HTTP se redirigen a HTTPS
- el servidor envía las cabeceras de seguridad definidas
- las rutas de negocio responden 401 sin una sesión válida, y la ruta de diagnóstico interno de la red no es accesible desde internet
- los puertos internos de la base de datos, el backend y la red Fabric no responden desde fuera
- el inicio de sesión responde con un mensaje genérico ante credenciales inválidas y bloquea temporalmente una dirección IP después de diez intentos por minuto.

Figura 35. Pruebas de infraestructura y seguridad sobre el despliegue

```
### Fecha: 2026-10-27 15:08:14 -0500 · Entorno: https://trazadoc-tesis.eastus2.cloudapp.azure.com

### 1. Certificado HTTPS
subject=CN=trazadoc-tesis.eastus2.cloudapp.azure.com
issuer=C=US, O=Let's Encrypt, CN=YE1
notBefore=Oct 25 20:10:25 2026 GMT
notAfter=Jan 23 20:10:24 2027 GMT

### 2. HTTP redirige a HTTPS
GET http://trazadoc-tesis.eastus2.cloudapp.azure.com/ -> HTTP 308, Location: https://trazadoc-tesis.eastus2.cloudapp.azure.com/

### 3. Página de inicio y estado del backend
GET / -> HTTP 200
GET /api/health -> {"status":"ok","mongo":"connected"}

### 4. Cabeceras de seguridad
Content-Security-Policy: default-src 'self'; img-src 'self' data: blob:; style-src 'self' 'unsafe-inline'; font-src 'self' da...
Referrer-Policy: no-referrer
Server: nginx
Strict-Transport-Security: max-age=31536000
X-Content-Type-Options: nosniff
X-Frame-Options: DENY

### 5. Ruta de diagnóstico interno bloqueada
GET /api/fabric/ping -> HTTP 404

### 6. Rutas protegidas sin sesión
GET /api/documents sin token -> HTTP 401
GET /api/events sin token -> HTTP 401
GET /api/users sin token -> HTTP 401
GET /api/documents/summary sin token -> HTTP 401

### 7. Credenciales inválidas (cuenta inexistente)
{"message":"Credenciales inválidas."} HTTP 401

### 8. Puertos internos cerrados desde internet
puerto 8080 -> sin respuesta (cerrado)
puerto 5173 -> sin respuesta (cerrado)
puerto 27017 -> sin respuesta (cerrado)
puerto 7050 -> sin respuesta (cerrado)
puerto 7051 -> sin respuesta (cerrado)
puerto 9051 -> sin respuesta (cerrado)
puerto 7054 -> sin respuesta (cerrado)
puerto 8054 -> sin respuesta (cerrado)
puerto 9054 -> sin respuesta (cerrado)

### 9. Versión desplegada: ruta de la politica (RF-055)
{"message":"No autenticado."} HTTP 401
Pantalla /politica-datos incluida en el frontend: 1 coincidencia(s)

### 10. Límite de intentos de inicio de sesión por IP (cuenta inexistente)
401 401 401 401 401 401 401 401 401 401 401 429
{"message":"Demasiados intentos. Espera un minuto e intenta de nuevo."}
```

Fuente: Hecho por las autoras

5.9 Defectos y acciones correctivas

Durante los tres ciclos se registraron siete defectos: uno de severidad alta, tres de severidad media y tres de severidad baja, ninguno crítico. Ninguno impidió registrar ni verificar documentos.

Tabla 17. Defectos encontrados y su cierre

ID	Defecto	Severidad	Ciclo	Acción correctiva	Estado
DEF-01	La lista de contratos del Verificador no se actualizaba después de verificar	Baja	1	Respuestas de la API sin caché y recarga del listado	Cerrado
DEF-02	"Reenviar código" no enviaba un código nuevo	Media	1	Se quitó el botón: no hay reenvío por diseño (C-09)	Cerrado
DEF-03	No se registraban los intentos fallidos de inicio de sesión	Media	1	Registro de cada intento, sin la contraseña	Cerrado
DEF-04	No existía la aceptación de la política de tratamiento de datos	Alta	1	Política en el primer ingreso, exigida por el backend	Cerrado
DEF-05	Los estados se mostraban en minúscula	Baja	1	Estados con mayúscula inicial	Cerrado
DEF-06	Un Empleado podía consultar los datos de un documento ajeno	Media	1	Acceso denegado (403) a documentos ajenos	Cerrado
DEF-07	Al cerrar sesión no aparecía la pantalla "Sesión cerrada"	Baja	2	Ir a la pantalla de confirmación antes de cerrar la sesión	Cerrado

Fuente: Hecho por las autoras

Los más relevantes se encontraron en el Ciclo 1:

- DEF-04, severidad alta: faltaba la aceptación de la política de tratamiento de datos que exige la Ley 1581 de 2012.
- DEF-03: no se registraban los intentos fallidos de inicio de sesión.
- DEF-06: un Empleado podía consultar por la API los datos de un documento ajeno.

El Ciclo 1 también reveló tres defectos de interfaz:

- DEF-01: la lista de contratos del Verificador no reflejaba el estado nuevo después de verificar. Su causa real se identificó durante el Ciclo 2: el navegador reutilizaba respuestas guardadas de la API. Se corrigió desde el servidor, que ahora indica que esas respuestas no deben guardarse.
- DEF-02: un botón de reenvío de código no enviaba un código nuevo. Se eliminó, en coherencia con la decisión de diseño de no reenviar códigos.
- DEF-05: los estados se mostraban en minúscula.

En el Ciclo 2 apareció DEF-07: al cerrar sesión no se mostraba la pantalla de confirmación. Las siete correcciones se publicaron en Azure y se volvieron a probar en el Ciclo 3 con resultado PASS, por lo que todos los defectos quedaron cerrados.

5.10 Matriz de cumplimiento y manual de usuario

La Matriz de Cumplimiento de Requerimientos cruzó los 85 requerimientos del SRS con los casos de prueba que los cubren y su resultado final. Cumplieron los 55 requerimientos funcionales y 27 de los 30 no funcionales, para un cumplimiento del 96,5 %.

Figura 36. Resumen de la Matriz de Cumplimiento de Requerimientos

Matriz de Cumplimiento de Requerimientos · TrazaDoc

Cruza los requerimientos del SRS v2.0 (Fase 1) con los casos del Plan de Pruebas y su resultado final: el del Ciclo 2 (Azure) o, si el caso no se repitió allá, el del Ciclo 1 (local). Fecha de corte: 29/10/2026.

Tipo	Total	Cumple	Parcial	No cumple
Requerimientos funcionales (RF)	55	55	0	0
Requerimientos no funcionales (RNF)	30	27	3	0
Total	**85**	**82**	**3**	**0**
% que cumple (Cumple / Total)	**96,5%**			

Requerimientos en Parcial

RNF-012 Disponibilidad del sistema	No se midió la disponibilidad con una verificación automática cada 5 minutos durante un mes. Los contenedores reinician solos y el sistema estuvo arriba en todas las pruebas del 25 al 29/10/2026.
RNF-015 Facilidad de uso	No se hizo una prueba de usabilidad con usuarios de la empresa. El equipo completó todas las tareas de los tres roles sin ayuda; queda como recomendación una prueba piloto.
RNF-025 Compatibilidad web	Las pruebas se ejecutaron en Google Chrome. Edge y Firefox, previstos en el plan, no se probaron.

Fuente: Hecho por las autoras

Los tres no funcionales restantes quedaron como parciales, con su justificación:

- RNF-012: la disponibilidad no se midió de forma automática durante un mes, aunque el sistema estuvo disponible en todas las pruebas.
- RNF-015: la facilidad de uso no se evaluó con usuarios de la empresa.
- RNF-025: la compatibilidad se probó en Google Chrome y no en los demás navegadores previstos.

El Manual de Usuario y Operación tiene dos partes:

- Uso del sistema: explica, para cada rol, cómo usar el sistema con capturas del despliegue e incluye una sección de acceso para la evaluación.
- Guía técnica: cubre la instalación con Docker, la operación diaria en el servidor, la publicación de versiones nuevas, la reversión ante fallos y la solución de los problemas frecuentes

5.11 Resultados frente a los indicadores

El indicador de detección de alteraciones se cumplió: el sistema detectó, sobre el despliegue en Azure, las 20 alteraciones (100 %) sin ningún falso positivo, incluidas las que no se ven a simple vista. El indicador de casos funcionales alcanzó 51 de 51 casos con resultado PASS en la suite final (100 %), frente a una meta del 90 %.

El plan cerró sus 20 actividades en plazo (100 %), frente a una meta del 85 %. La latencia de verificación se mantuvo en 2.059 ms en el percentil 95, dentro de la meta de 3.000 ms.

Tabla 18. Resultado de los KPI del O4

KPI	Meta	Resultado	Estado
Tasa de detección de alteraciones	100 % de 20 documentos	100 % (20 de 20) y 0 falsos positivos, en Azure	Cumple
Casos funcionales PASS en la suite final	≥ 90 % (46 de 51)	100 % (51 de 51)	Cumple
Ejecución del PPS en plazo	≥ 85 % (17 de 20 actividades)	100 % (20 de 20)	Cumple
Latencia de verificación (REN-01)	P95 ≤ 3.000 ms	2.059 ms (local); 2.125 ms (Azure)	Cumple

Fuente: Hecho por las autoras

Con estos resultados se da por cumplido el cuarto objetivo.

6. CONCLUSIONES

El trabajo de grado respondió a la pregunta de en qué medida una red blockchain permisionada integrada con un sistema web permitía verificar la integridad y la trazabilidad de contratos laborales electrónicos en el contexto colombiano. Los resultados mostraron que lo permitió de forma efectiva: el sistema detectó todas las alteraciones del conjunto de prueba, registró cada acción de manera inmutable y respondió dentro de los tiempos definidos.

El alcance de esa respuesta fue preciso. El sistema demostró si un documento había cambiado, pero no identificó qué parte cambió ni reemplazó la firma electrónica regulada por el Decreto 2364 de 2012. Su aporte se ubicó en la evidencia de integridad que reconoce la Ley 527 de 1999.

El primer objetivo se cumplió con la formalización de 85 requerimientos, 55 funcionales y 30 no funcionales, y de un backlog de 24 historias de usuario, a partir del análisis normativo y de la consulta con la empresa. El 100 % de los requerimientos y de las 15 disposiciones normativas aplicables quedaron relacionados en el SRS, por encima de la meta del 90 %.

El análisis normativo evidenció que la ley no exigía guardar el contrato en la blockchain, sino demostrar su integridad. Por ello, en el ledger se registró únicamente la huella SHA-256 del documento y no su contenido, lo que protegió los datos personales conforme a la Ley 1581 de 2012. El benchmarking frente a Google Cloud Platform respaldó la elección de Hyperledger Fabric por costo y control de la infraestructura.

El segundo objetivo se cumplió con la aprobación de los cuatro dominios de diseño antes de la semana 7: modelado funcional, arquitectura de software, arquitectura de infraestructura y diseño UX/UI. No se registraron observaciones críticas. El diseño se mantuvo estable durante el desarrollo, y los cambios que surgieron fueron ajustes menores que quedaron documentados.

El tercer objetivo se cumplió con la construcción de un MVP que integró una interfaz web para los roles de Empleado, Verificador y Administrador, un backend y un contrato inteligente sobre una red Hyperledger Fabric con dos organizaciones. Se terminaron y aprobaron el 100 % de las historias comprometidas, y la verificación respondió en 2.059 ms en el percentil 95, por debajo de la meta de 3.000 ms.

El prototipado evolutivo resultó adecuado, porque la integración con la blockchain, que era el principal riesgo técnico, se resolvió desde el primer sprint. El despliegue en Azure con HTTPS demostró que el sistema funcionaba fuera del entorno de desarrollo.

El cuarto objetivo se cumplió mediante un plan de pruebas de 62 casos ejecutado en tres ciclos. Sobre el despliegue en Azure, el sistema detectó el 100 % de las alteraciones en 20 documentos, sin falsos positivos, y el 100 % de los casos funcionales obtuvo resultado PASS en la suite final, frente a una meta del 90 %.

Las pruebas también encontraron siete defectos, ninguno crítico, que se corrigieron y se volvieron a probar antes de la entrega. Entre ellos estaban la falta de aceptación de la política de datos y un acceso indebido a documentos ajenos. Esto demostró que el plan de pruebas aportó valor más allá de medir indicadores, al revelar fallas de seguridad y de cumplimiento normativo.

El hallazgo más relevante fue que se detectaron incluso las alteraciones invisibles a simple vista, como el cambio de un solo carácter o de los metadatos del archivo. Esto confirmó que la verificación dependió de la huella criptográfica y no del criterio de quien revisaba el documento.

Las pruebas de rendimiento mostraron que los tiempos de registro y de verificación, cercanos a dos segundos, estuvieron determinados por la configuración de la red y no por la aplicación. Por ello, el tiempo de respuesta resultó predecible.

En conjunto, se cumplieron los cuatro objetivos específicos y el objetivo general. El proyecto demostró que una huella criptográfica registrada en una blockchain permisionada ofreció a la empresa una prueba objetiva de que un contrato no fue modificado, sin depender de la confianza en quien custodiaba el archivo, y dejó una base validada para llevar la solución a producción

7. RECOMENDACIONES

Alcances y problemas no abarcados en este trabajo

El sistema determina si un contrato fue modificado, pero no indica qué parte cambió. Se recomienda agregar una comparación de contenido entre el documento registrado y el presentado. Para eso, el sistema tendría que conservar una copia del original en un almacenamiento seguro fuera de la cadena, con control de acceso y cifrado. Así, la verificación diría también dónde está la diferencia, y no solo que existe.

El MVP no emite ni valida firmas electrónicas, por lo que las disposiciones del Decreto 2364 de 2012 sobre firma quedaron fuera de su alcance. Se recomienda integrarlo con un proveedor de firma electrónica o digital certificado y con un servicio de estampado cronológico de una entidad de certificación acreditada. De esa forma, la evidencia de integridad que genera el sistema tendría mayor fuerza probatoria ante un tercero.

La evaluación de rendimiento se hizo con peticiones consecutivas y con pocos usuarios, en una sola máquina virtual. No se midió el comportamiento con usuarios simultáneos ni con volúmenes altos de documentos. Se recomienda hacer pruebas de carga y de estrés con concurrencia, para conocer el número de transacciones por segundo que soporta la red y el punto en que la latencia supera la meta.

Las pruebas mostraron que los tiempos de registro y verificación dependen del tiempo de corte de bloques de la red, fijado en dos segundos. Se recomienda estudiar cómo cambian la latencia y el rendimiento al ajustar el tamaño y el tiempo de corte de los bloques, para encontrar la configuración adecuada a la demanda real de la empresa.

La red se desplegó con un solo nodo ordenador y con todas las organizaciones en el mismo servidor. Esta configuración fue suficiente para un MVP, pero no ofrece tolerancia a fallos ni independencia real entre organizaciones. Para producción se recomienda:

- distribuir el servicio de ordenamiento en al menos tres nodos Raft
- ubicar los peers de cada organización en infraestructura separada
- custodiar las llaves criptográficas en un módulo de seguridad de hardware (HSM)
- incorporar monitoreo, copias de seguridad del ledger y un procedimiento de recuperación ante desastres.
-

El sistema protege los datos personales al registrar en la blockchain solo la huella del documento. Sin embargo, los derechos de los titulares que reconoce la Ley 1581 de 2012 (consulta, rectificación y supresión) no se cubrieron con un procedimiento completo. Se recomienda definir cómo atender esas solicitudes sobre los datos que sí se guardan en la base de datos, teniendo en cuenta que el registro en el ledger no se puede borrar.

Los controles de la norma ISO/IEC 27001:2022 se usaron como criterios de diseño, no como objeto de auditoría. Se recomienda una evaluación formal de seguridad, que incluya pruebas de penetración y una revisión de los controles por un tercero, antes de usar el sistema con contratos reales.

El MVP se validó con documentos sintéticos y con usuarios del equipo del proyecto. Se

recomienda una prueba piloto en la empresa, con contratos y usuarios reales, para medir la aceptación del sistema, su usabilidad y su efecto en el proceso actual de gestión de contratos.

El alcance se limitó a contratos laborales y acuerdos de confidencialidad en PDF de hasta 20 MB. Se recomienda evaluar la extensión a otros documentos de la relación laboral, como otrosíes, certificaciones y liquidaciones, y a otros formatos, además de integrar el sistema con los sistemas de nómina o de recursos humanos de la empresa.

Trabajos futuros sobre la misma temática

Se propone conformar un consorcio entre varias empresas, o entre empresas y entidades como notarías u órganos de control. Cada participante operaría su propia organización en la red, y la evidencia de integridad no dependería de una sola parte. Sería el escenario en que una blockchain permisionada aporta más valor frente a una base de datos tradicional.

Se sugiere un estudio comparativo entre Hyperledger Fabric y otras plataformas permisionadas, como Hyperledger Besu o R3 Corda, en latencia, costo de operación, facilidad de despliegue y cumplimiento normativo. Así se ampliaría el benchmarking hecho con Google Cloud Platform en este trabajo.

Se propone investigar técnicas criptográficas que permitan demostrar propiedades de un contrato sin revelar su contenido, como las pruebas de conocimiento cero. Por ejemplo, demostrar que un contrato tiene cierta cláusula o vigencia sin exponer los datos personales del trabajador.

Finalmente, se recomienda un trabajo con enfoque jurídico que analice cómo valora un juez colombiano la evidencia generada por el sistema dentro de un proceso laboral. Ese trabajo debería revisar si el registro en una blockchain permisionada cumple, por sí solo, los criterios de confiabilidad de la Ley 527 de 1999, o si necesita el acompañamiento de una entidad de certificación.

BIBLIOGRAFÍA

[1] MINISTERIO DE TECNOLOGÍAS DE LA INFORMACIÓN Y LAS COMUNICACIONES. Guía de Referencia para la adopción e implementación de proyectos con tecnología blockchain para el Estado colombiano. Versión 2. Bogotá: MinTIC, mayo de 2022. [en línea]. Disponible en: https://portalelearning.mintic.gov.co/

[2] CONGRESO DE LA REPÚBLICA DE COLOMBIA. Ley 527 de 1999: por medio de la cual se define y reglamenta el acceso y uso de los mensajes de datos, del comercio electrónico y de las firmas digitales. Bogotá: Función Pública, 18 de agosto de 1999. [en línea]. Disponible en: https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=4276

[3] PRESIDENCIA DE LA REPÚBLICA DE COLOMBIA. Decreto 2364 de 2012: por medio del cual se reglamenta el artículo 7° de la Ley 527 de 1999, sobre la firma electrónica. Diario Oficial n.º 48.622. Bogotá: Función Pública, 22 de noviembre de 2012. [en línea]. Disponible en: https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=50550

[4] ACOSTA-LEÓN, Carlos Alfonso. La prueba documental de fuentes informáticas basadas en documentos digitales con firma electrónica certificada. Revista Chilena de Derecho y Tecnología. 2023, vol. 12, n.º 1, e68567. DOI 10.5354/07192584.2023.68567.

[5] FEDESOFT. Caracterización del Sector TI en Colombia. Bogotá: Federación Colombiana de la Industria de Software y Tecnologías de la Información Relacionadas, 2023. [en línea]. Disponible en: https://fedesoft.org/

[6] TRUST4P. Manejo de contratos en TRUST4P: Riesgos Legales y Pruebas. [Documento técnico interno]. 2025.

[7] GÓMEZ GARCÍA, Anays; SOSA HERNÁNDEZ, Mercedes; VERONA MARCOS, Sandra y DELGADO DAPENA, Martha Dunia. Buenas prácticas de ingeniería de software: pruebas de software. Revista Cubana de Transformación Digital, 2023, vol. 4, n.° 2, pp. 1-13. ISSN 2708 3411.

[8] MALAGÓN CABALLERO, David Santiago y RAMÍREZ FRANCO, Cristian Felipe. Desarrollo de un prototipo de aplicación web para la verificación de autenticidad de títulos académicos mediante el uso de blockchain. Tesis de grado para optar al título de Ingeniero de Sistemas. Bogotá: Universidad Piloto de Colombia, 2022.

[9] PRECHT, Hauke, HÜLLMANN, Joschka Andreas y MARX GÓMEZ, Jorge. Paperless Everything: A Systematic Literature Review for the Design of Blockchainbased Document Management Systems. Distributed Ledger Technologies: Research and Practice. 2026, vol. 5, n.º 2, art. 19. DOI 10.1145/3737296.

[10] MOSCOSO URDANETA, Steve Eduardo, SUÁREZ CHACÓN, David Felipe y MARTIN VEGA, Daniela. Certificación digital de documentos académicos basada en la tecnología de blockchain. TIA. 2019, vol. 7, n.º 2, pp. 20-27. DOI 10.14483/23448288.12757.

[11] HABER, Stuart y STORNETTA, W. Scott. How to time-stamp a digital document. Journal of Cryptology. 1991, vol. 3, n.º 2, pp. 99-111. DOI 10.1007/BF00196791.

[12] HERMAN, Ivan, et al. Verifiable Credentials Data Model v2.0. W3C Recommendation. W3C, 15 de mayo de 2025. [en línea]. Disponible en: https://www.w3.org/TR/vc-data-model 2.0/. [Consulta: 1 mayo 2026].

[13]CASTAÑEDA-ORTEGA, Fabian Andrés. Ciberdelincuencia: análisis de tendencias y estrategias de mitigación, caso Colombia. Revista Portal de la Ciencia. 2024, vol. 5, n.º 3, pp. 298-308. DOI 10.51247/pdlc.v5i3.474.

[14] MENDOZA MENDOZA, Ingrid Esthela. Análisis de la Auditoría Forense: Gestor estratégico para la prevención del fraude financiero. Revisión Documental. Artículo científico previo a la obtención del título de Licenciada. Santo Domingo: Universidad Regional Autónoma de los Andes, 2023.

[15] DÍAZ MICHEL, Claudia y LLAMAS COVARRUBIAS, Jersain Zadamig. Contratos y blockchain: Contratos legales inteligentes. ¿Es necesario un marco legislativo? RITI Journal. 2023, vol. 11, n.º 23, pp. 42-58. DOI 10.36825/RITI.11.23.005.

[16] CONGRESO DE LA REPÚBLICA DE COLOMBIA. Ley 1581 de 2012: por la cual se dictan disposiciones generales para la protección de datos personales. Diario Oficial n.º 48.587. Bogotá: Función Pública, 17 de octubre de 2012. [en línea]. Disponible en: https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981.

[17] CASINO, Fran, DASAKLIS, Thomas K. y PATSAKIS, Constantinos. A systematic literature review of blockchain-based applications: current status, classification and open issues. Telematics and Informatics. 2019, vol. 36, pp. 55-81. DOI 10.1016/j.tele.2018.11.006.

[18] VACA, Pablo A. y DULCE-VILLARREAL, Edgar R. Blockchain para asegurar la integridad y trazabilidad en la cadena de custodia de evidencia digital en informática forense: un estudio de mapeo sistemático. TecnoLógicas. 2024, vol. 27, n.º 60, e3049. DOI 10.22430/22565337.3049.

[19] REMUZGO PINTO, Marcos. Blockchain para gobiernos digitales. Trabajo de fin de grado. Valencia: Universitat Politècnica de València, 2021.

[20] HYPERLEDGER FOUNDATION. A Blockchain Platform for the Enterprise — Hyperledger Fabric Docs main documentation. 2023. [en línea]. Disponible en: https://hyperledger fabric.readthedocs.io/

[21] SÁNCHEZ GÓMEZ, Pablo. Modelos de gestión documental con blockchain para verificación de integridad. Trabajo de fin de grado en Ingeniería Informática. Valladolid: Universidad de Valladolid, 2025.

[22] HYPERLEDGER FOUNDATION. Public networks: Besu documentation. 2023. [en línea]. Disponible en: https://besu.hyperledger.org/.

[23] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. NIST. *Secure Hash*

Standard (SHS). FIPS Publication 180-4. Gaithersburg: NIST, agosto de 2015. DOI 10.6028/NIST.FIPS.180-4. Disponible en: http://dx.doi.org/10.6028/NIST.FIPS.180-4

[24] MIER BELLO, Germán S., et al. A Scientific Data Integrity system based on Blockchain. Bucaramanga: Universidad Industrial de Santander, 2026. [en línea]. Disponible en: https://arxiv.org/abs/2601.13425.

[25] REHAN, Muhammad, et al. Supply chain management using an industrial Internet of Things Hyperledger Fabric network. Human-centric Computing and Information Sciences. 2023, vol. 13, art. 4. DOI 10.22967/HCIS.2023.13.004.

[26] NAJEEB, M., et al. Securing e-governance against shadow attacks with blockchain technology. Scientific Reports. 2025, vol. 15, art. 8456. DOI 10.1038/s41598-025-26326-0.

[27] MAURAS VELEZ, Agustín Leonel. Verificador de integridad de documentos con tecnología blockchain para la Universidad Católica de Córdoba. Trabajo final de grado para optar al título de Ingeniero en Sistemas. Córdoba: Universidad Católica de Córdoba, Facultad de Ingeniería, 2022.

[28] DE LA FUENTE MARTÍN, Alba. Aplicación de blockchain en sistemas de trazabilidad documental. Trabajo de fin de grado en Ingeniería Informática. Valladolid: Universidad de Valladolid, 2024.

[29] COLOMBIA. CORTE SUPREMA DE JUSTICIA. Sala de Casación Penal. Sentencia SP248-2025 (Rad. 58275) [en
línea]. M.P. Gerson Chaverra Castro. 12 de febrero de 2025. Disponible en: https://archivodigitalapi.cortesuprema.gov.co/share/2025/3/Boletines/SP248-2025(58275).pdf

[30] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. *Recommendation for Applications Using Approved Hash Algorithms*. Special Publication 800-107 Rev. 1. Gaithersburg: NIST, agosto 2012. Disponible en: https://doi.org/10.6028/NIST.SP.800-107r1

[31] INTERNET ENGINEERING TASK FORCE (IETF). RFC 3161: Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP). ZUCCHERATO, R., et al. 2001. [en línea]. Disponible en: https://tools.ietf.org/html/rfc3161.

[32] AGUDELO TARAZONA, Y. E. Prospectiva de los Smart Contracts para el derecho procesal civil en Colombia [en línea]. Revista Academia & Derecho. 2024. [Consulta: 3 de mayo de 2026]. https://dialnet.unirioja.es/descarga/articulo/9637062.pdf

[33] ROCHA, Ronaldo, LIMA, Gercina Ângela y ARAÚJO, Wálter José. Blockchain como tecnologia aplicada na elaboração de contratos inteligentes na Biblioteconomia e Ciência da Informação (BCI): uma revisión sistemática da literatura. AtoZ: novas prácticas em informação e conhecimento. 2025, vol. 14, pp. 1-9. DOI 10.5380/atoz.v14.91194.

[34] ZHENG, Zibin, et al. An overview on smart contracts: challenges, advances and platforms. Future Generation Computer Systems. 2020, vol. 105, pp. 475-491. DOI 10.1016/j.future.2019.12

[35] BANNIKOV, Valentyn, et al. Analysis of the use of blockchain technologies and smart contracts to automate management processes and ensure sustainability. Data and Metadata. 2024, vol. 3, art. 461. DOI 10.56294/dm2024461.

[36] INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. ISO/IEC 27001:2022 - Information security, cybersecurity and privacy protection — Information security management systems — Requirements. Ginebra: ISO, octubre 2022.

[37] CORRAL, Roberto. KPIs útiles: diseña indicadores operativos que realmente sirvan para mejorar. 1.ª ed. Barcelona: LEEXONLINE, 2017.

[38] LF DECENTRALIZED TRUST. Benchmarking Hyperledger Fabric 2.5 Performance [en línea]. de febrero de 2023. Disponible en: https://www.lfdecentralizedtrust.org/blog/2023/02/16/benchmarking-hyperledgerfabric-2-5 performance

[39] SIMROTH, Paul. Key Performance Indicators on the Blockchain: A Guide. Paul Simroth Blog, 2024. [en línea]. Disponible en: https://paulsimroth.at/

[40] INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. ISO 15489-1:2016 Información y documentación, Gestión de registros. Ginebra: ISO, 2016. [en línea]. Disponible en: https://www.iso.org/standard/62542.html.

ANEXOS

- Anexo A. Tabla Kpis, sección de detalle Smart y leyenda
  KPIs_SMART_MVP_Blockchain.xlsx

- Anexo B. Backlog del producto
  Requisitos_Historias_Usuario_(backlog incluido).xlsx

- Anexo C. Requerimientos Funcionales y No funcionales
  Requerimientos funcionales y no funcionales.docx

- Anexo D. Benchmarking Comparativo con GCP
  benchmarking_mvp_gcp.xlsx

- Anexo E. Matriz Stakeholders
  Matriz_Actores_Stakeholders_Proyecto.xlsx

- Anexo F. Matriz Normativa
  Matriz_Normativa.xlsx

- Anexo G. Tabla detallada de instrumentos y herramienta
  tabla_instrumentos_herramientas.xlsx

- Anexo H. Seguridad por módulo en el sistema
  Seguridad_Modulos_MVP_ConLeyenda.xlsx

- Anexo I. Presupuesto
  Presupuesto.xlsx

- Anexo J. Cronograma, resumen por semana y Leyenda
  Cronograma_16semanas_MVP_Blockchain_v2.xlsx

- Anexo K. Entrevista Empresa Preguntas
  Entrevista_empresa.docx

- Anexo L. UML: casos de uso, componentes, despliegue, MER, secuencias

- Anexo M. Wireframes baja fidelidad y Prototipo Alta fidelidad
  Wireframes baja fidelidad

- Anexo N. Matriz Infraestructura
  Matriz_Infraestructura.xlsx

- Anexo O. Matriz Asrs y Utility tree
  Matriz_ASRS y Utility tree.xlsx

- Anexo P. Matriz_Trazabilidad_HU_R
  Matriz_Trazabilidad_HU_RF.xlsx

- Anexo Q. registro-sprints
  registro-sprints-

- Anexo R. integracion-fabric
  integracion-fabric-

- Anexo S. INSTALACION
  Guia de Instalacion - TrazaDoc

- Anexo T. Manual de usuario
  Manual_Usuario_Operacion_TrazaDoc.docx

- Anexo U. Registro cambios del diseño
  registro-cambios-diseno-

- Anexo V. Pruebas QA-QC
  Pruebas_QA_QC_TrazaDoc.xlsx

- Anexo W. Informe de resultados
  Informe_Resultados_Defectos_TrazaDoc.xlsx

- Anexo X. Matriz cumplimiento de requerimientos
  Matriz_Cumplimiento_Requerimientos_TrazaDoc.xlsx